TEK-UP University

GLSI (Software Engineering and Information Systems)

End-of-Studies Project Report

Submitted in partial fulfilment of the requirements for the degree of

National Engineering Diploma in Computer Science (GLSI)

Inkwell.ai

An AI-powered writing marketplace connecting independent writers and magazine publishers

---

Carried out by: **Oussama Bejaoui**

Academic supervisor: **Mr. Bilel Zemzem**

Professional supervisor: **Mr. Ramzy Chridi**

Host organisation: **Intuitiv Group**

Project period: 1 February 2026 – 31 July 2026

Academic year 2025–2026

Acknowledgements

I would like to express my sincere gratitude to my academic supervisor, **Mr. Bilel Zemzem**, for his guidance, his availability and his valuable advice throughout this project, and for the rigour he encouraged in both the work and this report.

I am equally grateful to my professional supervisor, **Mr. Ramzy Chridi**, and to the whole team at **Intuitiv Group**, for welcoming me, for the trust they placed in me and for the time they gave to following the project and sharing their experience.

My thanks also go to the members of the jury for the honour they do me in examining this work, and to all the teaching staff of **TEK-UP University** for the knowledge and support they have given me throughout my studies.

Finally, I thank my family and my friends for their constant encouragement and patience, without which this work would not have been possible.

Abstract

Independent writers have tools to publish but few ways to be paid for quality, and magazines that want to commission them have no objective way to judge a writer before buying. This report presents **Inkwell.ai**, a two-sided web platform that addresses both sides. Writers draft in a rich-text editor beside an AI assistant grounded in their own published work and in reference documents they upload; it answers in a panel or writes directly into the document, cites its sources by page, and accepts spoken prompts. Every published article feeds an event-based analytics pipeline whose audience, content and quality signals, together with AI-generated portfolio insights, form an evaluation report that magazines use to decide. Magazines subscribe, receive monthly credits, preview an article for 10% of its price and buy the remainder for exclusive republication rights, every credit movement being recorded in a ledger whose invariants are tested after each operation.

The platform was built over nine two-week Scrum sprints grouped into four releases, with a Next.js 16 frontend, a NestJS 11 backend and worker, PostgreSQL 16 with pgvector, Redis and BullMQ, MinIO object storage and nginx, all containerised with Docker Compose, and with Groq and Gemini models orchestrated through the Vercel AI SDK. The backend suite counts 796 passing tests. Measured on a labelled set, retrieval returns a relevant passage in its top five for 89% of questions, hybrid search keeps every relevant article in its top ten even for paraphrased queries that defeat keyword search, and the publish-time moderation classifier reaches a precision of 0.93 and a recall of 1.00.

Keywords: writing marketplace, AI-assisted writing, retrieval-augmented generation, hybrid search, pgvector, analytics, article licensing, Scrum, NestJS, Next.js.

Résumé

Les rédacteurs indépendants disposent d'outils pour publier, mais de peu de moyens d'être rémunérés pour la qualité de leur travail, et les magazines qui souhaitent leur confier des articles n'ont aucun moyen objectif d'évaluer un rédacteur avant d'acheter. Ce rapport présente **Inkwell.ai**, une plateforme web biface qui répond aux deux besoins. Les rédacteurs écrivent dans un éditeur de texte enrichi, aux côtés d'un assistant d'intelligence artificielle ancré dans leurs propres articles publiés et dans les documents de référence qu'ils déposent ; l'assistant répond dans un panneau ou écrit directement dans le document, cite ses sources à la page près et accepte des consignes dictées à la voix. Chaque article publié alimente une chaîne d'analyse fondée sur des événements, dont les indicateurs d'audience, de contenu et de qualité, complétés par une synthèse générée par l'IA, constituent un rapport d'évaluation sur lequel les magazines s'appuient pour décider. Les magazines s'abonnent, reçoivent des crédits mensuels, débloquent un aperçu pour 10 % du prix et achètent le reste pour obtenir des droits de republication exclusifs ; chaque mouvement de crédits est inscrit dans un grand livre dont les invariants sont vérifiés après chaque opération.

La plateforme a été réalisée en neuf sprints Scrum de deux semaines, regroupés en quatre releases, avec un frontend Next.js 16, un backend et un worker NestJS 11, PostgreSQL 16 avec pgvector, Redis et BullMQ, le stockage objet MinIO et nginx, le tout conteneurisé avec Docker Compose, les modèles Groq et Gemini étant orchestrés par le Vercel AI SDK. La suite de tests du backend compte 796 tests réussis. Mesurée sur un jeu de données annoté, la recherche de passages renvoie un passage pertinent parmi les cinq premiers pour 89 % des questions, la recherche hybride conserve tous les articles pertinents parmi les dix premiers résultats, y compris pour des requêtes reformulées que la recherche par mots-clés ne trouve pas, et le classifieur de modération atteint une précision de 0,93 et un rappel de 1,00.

Mots-clés : place de marché d'écriture, rédaction assistée par l'IA, génération augmentée par la recherche, recherche hybride, pgvector, analytique, licence d'articles, Scrum, NestJS, Next.js.

Contents

Acknowledgements i

Abstract ii

Résumé iii

List of acronyms x

General introduction 1

1 Project context and requirements analysis 2
Introduction . . . 2
1.1 Project context . . . 2
1.1.1 Host organisation . . . 2
1.1.2 Context and motivation . . . 2
1.1.3 Study of existing solutions . . . 2
1.1.4 Problem statement and proposed solution . . . 3
1.2 Development methodology . . . 4
1.2.1 Sequential and agile approaches . . . 4
1.2.2 Scrum, and its adaptation to a one-person project . . . 4
1.2.3 Prioritisation and estimation . . . 5
1.2.4 Modelling language . . . 6
1.3 Identification of the actors . . . 6
1.4 Functional requirements . . . 6
1.5 Non-functional requirements . . . 7
1.6 Global use case diagrams . . . 9
1.7 Global class diagram . . . 12
1.8 Product backlog . . . 17
1.9 Release plan and schedule . . . 18
Conclusion . . . 19

2 Working environment and architecture **20**
Introduction . . . 20
2.1 Working environment . . . 20
2.1.1 Hardware environment . . . 20
2.1.2 Software environment . . . 20
2.1.3 Foundation and design track . . . 21
2.1.4 Repository organisation . . . 21
2.2 Technological choices . . . 21
2.2.1 Frontend . . . 21
2.2.2 Backend . . . 22
2.2.3 Artificial-intelligence services . . . 23
2.2.4 Infrastructure and quality . . . 23
2.3 Main parameters and their justification . . . 23
2.4 Logical architecture . . . 25
2.4.1 Frontend . . . 25
2.4.2 Backend . . . 25

2.5 Physical architecture . . . 26
2.6 Continuous integration and delivery . . . 27
Conclusion . . . 28

3 Release 1: functional foundation **29**
Introduction . . . 29
3.1 Sprint 1: Schema, authentication and article core . . . 29
3.1.1 Sprint backlog . . . 29
3.1.2 Analysis . . . 29
3.1.3 Design . . . 30
3.1.4 Realisation . . . 32
3.2 Sprint 2: Editor, AI assistance and event capture . . . 36
3.2.1 Sprint backlog . . . 36
3.2.2 Analysis . . . 36
3.2.3 Design . . . 37
3.2.4 Realisation . . . 39
Conclusion . . . 42

4 Release 2: social interactions, analytics and AI **44**
Introduction . . . 44
4.1 Sprint 3: Social interactions, notifications and analytics . . . 44
4.1.1 Sprint backlog . . . 44
4.1.2 Analysis . . . 44
4.1.3 Design . . . 45
4.1.4 Realisation . . . 49
4.2 Sprint 4: Retrieval, portfolio insights and hybrid search . . . 53
4.2.1 Sprint backlog . . . 53
4.2.2 Analysis . . . 54
4.2.3 Design . . . 54
4.2.4 Realisation . . . 58
Conclusion . . . 60

5 Release 3: marketplace, testing and deployment **61**
Introduction . . . 61
5.1 Sprint 5: Marketplace, subscriptions, premium and moderation . . . 61
5.1.1 Sprint backlog . . . 61
5.1.2 Analysis . . . 62
5.1.3 Design . . . 63
5.1.4 Realisation . . . 69
5.2 Sprint 6: Tests, demonstration and deployment . . . 76
5.2.1 Sprint backlog . . . 76
5.2.2 Analysis . . . 76
5.2.3 Design . . . 77
5.2.4 Realisation . . . 77
Conclusion . . . 78

6 Release 4: social depth, the reworked assistant and documents **79**
Introduction . . . 79
6.1 Sprint 7: Comment likes, lists, block, save and share . . . 79
6.1.1 Sprint backlog . . . 79
6.1.2 Analysis . . . 79
6.1.3 Design . . . 80
6.1.4 Realisation . . . 81

6.2 Sprint 8: Live cards, the assistant that writes, and voice . . . 83
6.2.1 Sprint backlog . . . 83
6.2.2 Analysis . . . 84
6.2.3 Design . . . 85
6.2.4 Realisation . . . 87
6.3 Sprint 9: Document library and citations . . . 90
6.3.1 Sprint backlog . . . 90
6.3.2 Analysis . . . 90
6.3.3 Design . . . 91
6.3.4 Realisation . . . 92
Conclusion . . . 94

7 Evaluation **95**
Introduction . . . 95
7.1 Evaluation protocol . . . 95
7.2 Retrieval behind the assistant . . . 95
7.3 Hybrid search . . . 96
7.4 Moderation classifier . . . 97
7.5 Performance and accessibility . . . 98
7.6 Test coverage . . . 99
7.7 Delivery: velocity and retrospective . . . 99
Conclusion . . . 100

General conclusion and perspectives **101**
Limitations . . . 101
Perspectives . . . 102

References **103**

A Functional requirements **106**

B Product backlog **110**

List of Figures

Figure 1.1 – Classic and agile approaches compared . . . 4
Figure 1.2 – The Scrum framework, as practised in this project . . . 5
Figure 1.3 – Global use case diagram: visitor and reader . . . 10
Figure 1.4 – Global use case diagram: writer . . . 11
Figure 1.5 – Global use case diagram: magazine and administrator . . . 12
Figure 1.6 – Class diagram: accounts and articles . . . 13
Figure 1.7 – Class diagram: social layer and moderation . . . 14
Figure 1.8 – Class diagram: AI and documents . . . 15
Figure 1.9 – Class diagram: marketplace . . . 16
Figure 1.10 – Class diagram: analytics . . . 17
Figure 1.11 – Gantt chart of the project . . . 19

Figure 2.1 – Logical frontend architecture . . . 25
Figure 2.2 – Logical backend architecture . . . 26
Figure 2.3 – Physical architecture . . . 27
Figure 2.4 – CI/CD pipeline . . . 27

Figure 3.1 – Use case diagram, Sprint 1 . . . 30
Figure 3.2 – Class diagram, Sprint 1 . . . 31
Figure 3.3 – Sequence diagram: sign in and silent refresh . . . 32
Figure 3.4 – The public feed, as seen by a visitor . . . 33
Figure 3.5 – Sign-in, with Google sign-in . . . 33
Figure 3.6 – Registering a personal account . . . 34
Figure 3.7 – Registering a magazine account . . . 34
Figure 3.8 – A free article, readable without an account . . . 35
Figure 3.9 – Profile settings . . . 35
Figure 3.10 – Use case diagram, Sprint 2 . . . 37
Figure 3.11 – Class diagram, Sprint 2 . . . 38
Figure 3.12 – Sequence diagram: inline AI action . . . 39
Figure 3.13 – The rich-text editor with the assistant panel . . . 40
Figure 3.14 – The publish dialog: placement, visibility, tags and excerpt . . . 40
Figure 3.15 – The AI assistant panel . . . 41
Figure 3.16 – An answer from the assistant . . . 41
Figure 3.17 – The inline actions on a selection . . . 42
Figure 3.18 – An inline suggestion, ready to replace the selection . . . 42

Figure 4.1 – Use case diagram, Sprint 3 . . . 45
Figure 4.2 – Class diagram, Sprint 3 . . . 46
Figure 4.3 – Sequence diagram: analytics aggregation . . . 47
Figure 4.4 – Sequence diagram: live notification delivery . . . 49
Figure 4.5 – The feed of a signed-in reader, with live card controls . . . 50
Figure 4.6 – Notifications, filtered and grouped by day . . . 50
Figure 4.7 – The writer’s dashboard . . . 51
Figure 4.8 – “My articles”: every article, sortable by performance . . . 51
Figure 4.9 – The writer’s analytics: views over the last 30 days . . . 52
Figure 4.10 – A writer’s public profile . . . 52
Figure 4.11 – A magazine browsing eligible writers . . . 53

Figure 4.12 – Use case diagram, Sprint 4 54
Figure 4.13 – Class diagram, Sprint 4 55
Figure 4.14 – Sequence diagram: the publishing pipeline 56
Figure 4.15 – Sequence diagram: AI chat with retrieval 57
Figure 4.16 – Sequence diagram: hybrid search with reciprocal rank fusion 58
Figure 4.17 – Search results 59
Figure 4.18 – A writer evaluation report with portfolio insights 59

Figure 5.1 – Use case diagram, Sprint 5 62
Figure 5.2 – Class diagram, Sprint 5 64
Figure 5.3 – Sequence diagram: preview, purchase and publication 65
Figure 5.4 – Activity diagram: the marketplace eligibility gate 66
Figure 5.5 – Sequence diagram: reporting and moderation 67
Figure 5.6 – Activity diagram: the article access decision 68
Figure 5.7 – State diagram: the article lifecycle 68
Figure 5.8 – A premium article behind the paywall 69
Figure 5.9 – The plans: free and premium 69
Figure 5.10 – A magazine's subscription and credits 70
Figure 5.11 – The marketplace, with preview and purchase prices 70
Figure 5.12 – A listing before preview 71
Figure 5.13 – Confirming a preview unlock: price and balance after 71
Figure 5.14 – The magazine's library: unpublished and published 72
Figure 5.15 – Publishing a purchased article under the magazine's masthead 72
Figure 5.16 – A licensed article naming both the writer and the magazine 73
Figure 5.17 – A magazine's public profile and its published articles 73
Figure 5.18 – A writer's earnings, itemised by preview and purchase 74
Figure 5.19 – A sold article, as its author sees it 74
Figure 5.20 – Administration: manual eligibility grant 75
Figure 5.21 – Administration: user management 75
Figure 5.22 – Administration: the report queue 76
Figure 5.23 – Test strategy 77

Figure 6.1 – Use case diagram, Sprint 7 80
Figure 6.2 – Class diagram, Sprint 7 81
Figure 6.3 – The Saved shelf 82
Figure 6.4 – Blocked accounts, in settings 82
Figure 6.5 – The following feed 83
Figure 6.6 – A follower list, searched 83
Figure 6.7 – Use case diagram, Sprint 8 84
Figure 6.8 – Sequence diagram: one assistant turn 85
Figure 6.9 – Sequence diagram: writing into the document 86
Figure 6.10 – Sequence diagram: the voice round trip 87
Figure 6.11 – The Reposted tab of a profile 88
Figure 6.12 – A write awaiting Keep or Discard 88
Figure 6.13 – The live step on the collapsed panel 89
Figure 6.14 – Recording a prompt 89
Figure 6.15 – Use case diagram, Sprint 9 90
Figure 6.16 – Class diagram, Sprint 9 91
Figure 6.17 – Sequence diagram: document upload and ingestion 92
Figure 6.18 – The document library, one document refused with its reason 93
Figure 6.19 – Attaching a document from the Sources tab 93
Figure 6.20 – A reply citing its source as [Title, p. N] 94

List of Tables

Table 1.1 – Comparison of existing solutions, from each product's public documentation 3
Table 1.2 – Actors of the system 6
Table 1.3 – Functional requirements by actor and priority 7
Table 1.4 – Non-functional requirements and how each is verified 7
Table 1.5 – Product backlog by epic 17
Table 1.6 – Release plan 18

Table 2.1 – Hardware environment 20
Table 2.2 – Software tools 20
Table 2.3 – Frontend technologies compared with their alternatives 21
Table 2.4 – Backend technologies compared with their alternatives 22
Table 2.5 – AI models, their alternatives and the reasons for the choice 23
Table 2.6 – Main parameters of the system and their justification 24

Table 3.1 – Sprint 1 backlog 29
Table 3.2 – Textual description of "Publish an article" 30
Table 3.3 – Sprint 2 backlog 36

Table 4.1 – Sprint 3 backlog 44
Table 4.2 – Main metrics of the evaluation report 47
Table 4.3 – Sprint 4 backlog 53

Table 5.1 – Sprint 5 backlog 61
Table 5.2 – Marketplace rules and their justification 63
Table 5.3 – Sprint 6 backlog 76
Table 5.4 – Test results 78

Table 6.1 – Sprint 7 backlog 79
Table 6.2 – Sprint 8 backlog 84
Table 6.3 – Sprint 9 backlog 90

Table 7.1 – Retrieval of the writer's own passages, by similarity floor (54 queries) 96
Table 7.2 – Lexical, semantic and hybrid search (54 queries, 56 articles) 96
Table 7.3 – Confusion matrix of the moderation classifier (identical in the three runs) 97
Table 7.4 – Latency of the main API routes under concurrent load 98
Table 7.5 – Lighthouse scores and axe-core violations of the public pages 98
Table 7.6 – Committed and delivered points per sprint 99
Table 7.7 – Retrospective by release 99

Table A.1 – Functional requirements: visitor 106
Table A.2 – Functional requirements: reader 106
Table A.3 – Functional requirements: writer 107
Table A.4 – Functional requirements: magazine 108
Table A.5 – Functional requirements: administrator 108
Table A.6 – Functional requirements: cross-cutting 109

List of acronyms

Acronym	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
CI/CD	Continuous Integration / Continuous Deployment
CLS	Cumulative Layout Shift (Lighthouse metric)
CRUD	Create, Read, Update, Delete
DOCX	Office Open XML word-processing document format
DSDM	Dynamic Systems Development Method
DTO	Data Transfer Object
ESN	Entreprise de Services du Numérique (IT services company)
FR / NFR	Functional / Non-Functional Requirement
GHCR	GitHub Container Registry
HNSW	Hierarchical Navigable Small World (approximate nearest-neighbour index)
HTML	HyperText Markup Language
JSON	JavaScript Object Notation
JWT	JSON Web Token
LCP	Largest Contentful Paint (Lighthouse metric)
LLM	Large Language Model
MoSCoW	Must, Should, Could, Won’t (prioritisation)
MRR	Mean Reciprocal Rank
nDCG	normalised Discounted Cumulative Gain
OAuth	Open Authorization (delegated authorisation protocol)
OpenAPI	OpenAPI Specification (machine-readable description of a REST API)
ORM	Object-Relational Mapping
PDF	Portable Document Format
PFE	Projet de Fin d’Études (end-of-studies project)
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
RRF	Reciprocal Rank Fusion
S3	Simple Storage Service (object storage protocol)
SQL	Structured Query Language
SSE	Server-Sent Events
TLS	Transport Layer Security
TTS	Text-to-Speech
UI	User Interface

Acronym	Meaning
UML	Unified Modeling Language
US	User Story
WCAG	Web Content Accessibility Guidelines

General introduction

Writing has never been easier to publish and never harder to be paid for. Blogging platforms and newsletter services let anyone reach readers in minutes, and large language models now help with drafting, rephrasing and editing. Yet the market that pays writers is narrow: of the roughly fifty million people who create content for an audience, only about four percent earn more than a hundred thousand dollars a year, and most of the income comes from brand deals rather than from the writing itself [1]. Publications that would pay for good work struggle to find it and, once they find it, to judge it on anything firmer than a few samples and intuition.

This end-of-studies project starts from that double gap. On one side, writers need an assistant that helps them write *in their own voice*, not a generic text generator; on the other, magazines need objective signals (who reads a writer, how far, how often they come back) before they commit money. Between the two, a market is missing: a place where a magazine can browse, evaluate and license an existing article directly from its author, with clear terms and a trustworthy record of who paid what.

Inkwell.ai is the platform built to fill that gap. It combines three pillars:

1. **AI-collaborative writing**: an assistant integrated into the editor as chat, inline actions and voice input, grounded in the writer's published corpus and in reference documents the writer uploads, able to write directly into the draft;
2. **decision-support analytics**: every published article generates audience, content and quality signals that feed dashboards for the writer and an evaluation report for magazines, completed by AI-generated portfolio insights;
3. **an article licensing marketplace**: magazines are a distinct account type that subscribes, receives monthly credits, previews an article for a tenth of its price and buys the rest for exclusive republication rights.

The work was carried out over six months, from February to July 2026, following the Scrum framework in nine fixed two-week sprints grouped into four releases. This report follows that structure:

- **Chapter 1** presents the context of the project, studies the existing solutions, justifies the development method, and then analyses and specifies the requirements: actors, functional and non-functional requirements, the global use case and class diagrams, the product backlog and the release plan.
- **Chapter 2** describes the working environment, compares the technological choices with their alternatives, justifies the main parameters of the system, and presents its logical and physical architecture.
- **Chapters 3 to 6** present the four releases sprint by sprint (the sprint backlog, the analysis, the design and the realisation), from the functional foundation to the marketplace, testing and deployment, and finally to the reworked assistant and the document library.
- **Chapter 7** evaluates the result: the retrieval behind the assistant, the hybrid search, the moderation classifier, the performance and accessibility of the platform, the test coverage and the delivery itself.

A general conclusion summarises the results, states the limitations of the work and opens perspectives for the platform. Two annexes give the full functional requirements and the full product backlog.

CHAPTER 1

Project context and requirements analysis

Introduction

This chapter sets out what the project is and what it must do. Its first part places the project in its context: the host organisation, the observations that motivated it, the existing solutions, the problem and the proposed solution, and the development method adopted and how it was adapted. Its second part specifies the requirements: the actors, the functional and non-functional requirements, the global use case and class diagrams, and the organisation of the work as a product backlog and a release plan.

1.1 Project context

1.1.1 Host organisation

This project was carried out at **Intuitiv Group**, a company with a dual culture: at once an IT services company (*Entreprise de Services du Numérique*, ESN) and an interactive agency. It works with large national and international groups while keeping a human-scale structure, and positions itself on reactivity, reliability and quality of service. That dual culture suits Inkwell.ai, which is as much a software engineering effort (architecture, data, security, delivery) as a product and interface design effort.

1.1.2 Context and motivation

The project was proposed as an end-of-studies engineering project with a product-shaped scope: not a prototype of one feature, but a complete two-sided platform with real accounts, real data flows, an AI layer and money-like transactions. Three observations motivated it.

- **Writing assistants are generic.** Mainstream assistants draft, shorten and change the tone of a text from a description of what is wanted [2], [3], [4]; none of those cited learns from what the writer has already published, so they write *for* the writer rather than *like* the writer.
- **Engagement data is collected, but not offered as evidence.** Platforms measure reading in detail: Medium pays its writers according to members' reading time, read ratio and engagement [5]. Yet these signals are shown to the writer or used to rank content, never presented to a buyer judging the writer.
- **Paid writing is a narrow market.** About four percent of creators earn more than a hundred thousand dollars a year, and most creator income comes from brand deals [1]. None of the publishing platforms studied below offers a publication a way to license an article that already exists.

1.1.3 Study of existing solutions

The solutions close to the project fall into five families. **Publishing platforms** such as Medium pay writers from a pool of reader subscriptions [5]. **Newsletter platforms** (Substack, Ghost and beehiiv) let a writer charge readers directly [6], [7], and beehiiv adds an AI writing assistant to its editor [4]. **Freelance marketplaces** such as Contently and Upwork connect clients with writers for *commissioned* work: the article does not yet exist when it is bought, and the platform holds no engagement data on the writer's published pieces [8], [9].

AI writing assistants such as Grammarly and Notion AI improve or generate text inside a document [2], [3]. Finally, **source-grounded assistants** (NotebookLM, ChatGPT with uploaded files, Perplexity Spaces) answer from documents the user uploads and cite the passages they used [10], [11], [12].

Table 1.1 compares these families on the criteria the project targets. Each cell records what the product's own documentation states; "partial" means the capability exists in a restricted form, explained below the table.

Table 1.1 – Comparison of existing solutions, from each product's public documentation

Criterion	Medium	Newsletter platforms[1]	Freelance markets[2]	Writing assistants[3]	Grounded assistants[4]	Inkwell.ai
Publishing and public reading	✓	✓	✗	✗	✗	✓
AI assistance in the editor	✗	partial	✗	✓	✗	✓
Assistant grounded in the writer's published work	✗	✗	✗	✗	partial	✓
Answers citing uploaded documents	✗	✗	✗	✗	✓	✓
Engagement analytics for the writer	✓	✓	✗	✗	✗	✓
Analytics exposed to a buyer as evidence	✗	✗	✗	✗	✗	✓
Licensing of existing articles to publications	✗	✗	✗	✗	✗	✓
Traceable payments	✓	✓	✓	✗	✗	✓

[1] Substack, Ghost, beehiiv. [2] Contently, Upwork. [3] Grammarly, Notion AI. [4] NotebookLM, ChatGPT with files, Perplexity Spaces.

Two cells need a word. beehiiv is the only newsletter platform of the three with an assistant in its editor, hence "partial" for that family. The source-grounded assistants could be grounded in a writer's published work only if the writer uploaded it by hand, and they are separate from any editor or publishing workflow.

The decisive cells are the last three rows. Citing uploaded documents, taken alone, is **not** distinctive: the source-grounded assistants already do it, and Inkwell.ai borrows the idea. What no family offers is the *combination*: an assistant grounded in the writer's own published corpus inside the editor where the writing happens, analytics presented to a buyer as evidence, and a market in which a publication licenses an existing article. Each family covers one or two of these needs; the platforms that pay writers do not give buyers evidence, and the tools that ground answers in sources do not publish or pay.

1.1.4 Problem statement and proposed solution

The study shows a problem on three levels. On the **supply side**, writers lack an assistant that writes in their own voice from what they have published and the material they are working from, and a direct path to be paid for quality. On the **demand side**, magazines lack objective, comparable signals about a writer's audience, consistency and quality. Between them, the **market** is missing: there is no self-service place where a publication can discover, evaluate, preview and license an existing article, with the money recorded in a way both parties can trust.

Inkwell.ai answers the three levels with one platform: an **editor with an AI assistant** that retrieves passages from the writer's own articles and attached documents, decides per request whether to answer or to write into the draft, and cites its sources by page; an **analytics pipeline** that turns reading events into per-article

and per-writer metrics, presented to the writer and, as an evaluation report with AI portfolio insights, to magazines; and a **marketplace** where subscribed magazines preview an article for 10% of its price, buy the remaining 90% for exclusive rights and publish it under their own masthead, every credit movement being written to a ledger inside one database transaction.

1.2 Development methodology

1.2.1 Sequential and agile approaches

A **sequential (waterfall) process** runs each phase once, in order, and its classic description warned that late testing exposes design errors when they cost most [13]. An **agile process** delivers working software in short, reviewed iterations [14], [15]. Figure 1.1 contrasts the two.

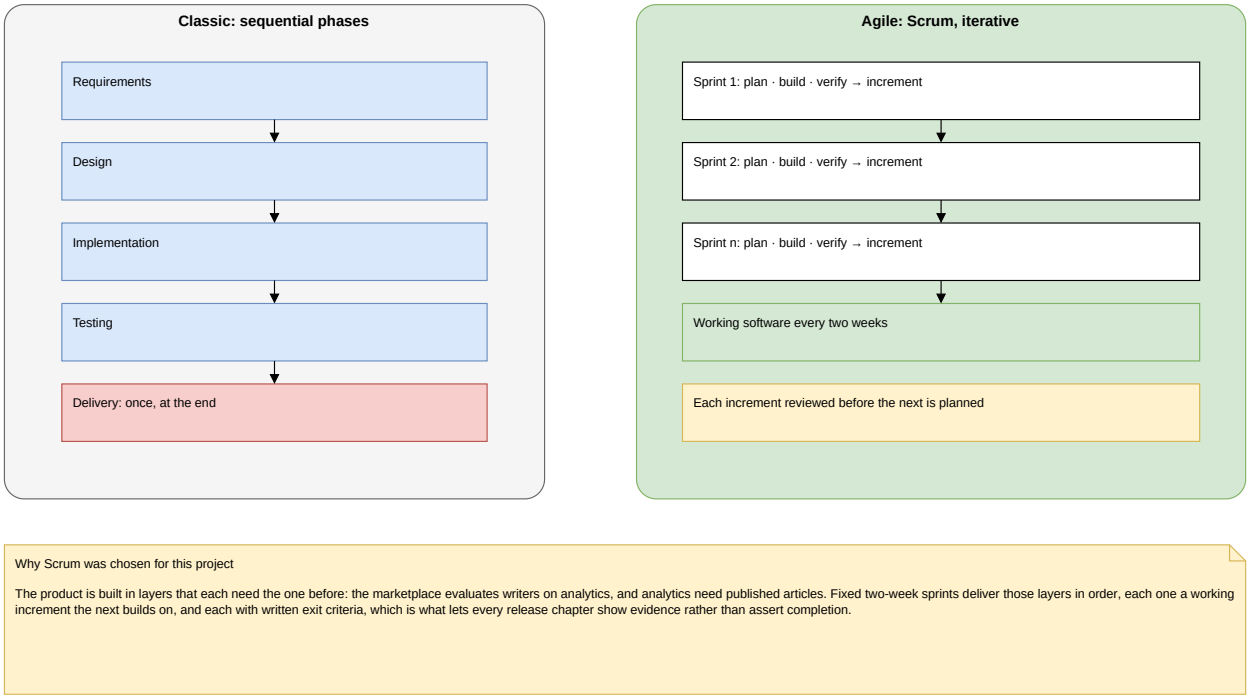


Figure 1.1 – Classic and agile approaches compared

Working software appears only in the last phase of the sequential process, but at the end of every agile iteration. Inkwell.ai is built in layers (accounts, articles, analytics, marketplace), each depending on the one below, so each must work before the next is built on it.

1.2.2 Scrum, and its adaptation to a one-person project

Among agile frameworks, **Scrum** was chosen for its fixed-length sprints and its explicit artefacts and events [16], which give an individual project the regular checkpoints a supervisor can follow. Figure 1.2 shows the framework as it was applied.

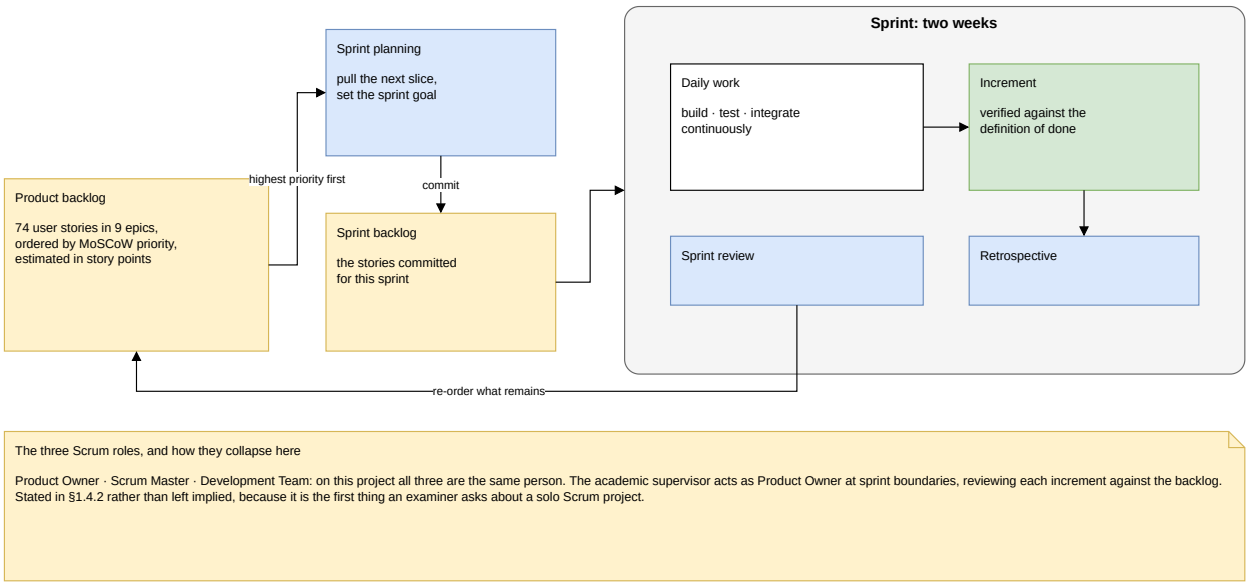


Figure 1.2 – The Scrum framework, as practised in this project

The backlog feeds sprint planning; each two-week sprint ends with an increment checked against the definition of done, a review and a retrospective whose actions return to the next planning. For this project the unit of progress is therefore the increment: every sprint leaves the platform runnable and tested.

Scrum is designed for a team of ten or fewer people holding three accountabilities (Product Owner, Scrum Master and Developers) [16], and this project has one person. The adaptation, and its limits, were the following.

- **Roles.** The author held the Developer and Scrum Master accountabilities. The academic supervisor acted as Product Owner at sprint boundaries, accepting or returning each increment against the backlog. The limit is that one person both estimates and executes the work: there is no team discussion to correct an optimistic estimate, which shows in the uneven sprint loads discussed in Section 7.7.
- **Events.** Sprint planning, the sprint review and the retrospective were kept; the daily scrum, which synchronises a team, has no purpose for one person and was replaced by a written log of what was done and what blocked. Its limit is the loss of a daily outside view.
- **Artefacts.** The product backlog, the sprint backlogs and the increment were kept unchanged, with a written **definition of done**, given with the evaluation in Section 7.7: code reviewed against the specification, the type check and linter passing with no warning, the backend tests passing in the container, and the feature demonstrated in a browser.

With one person, the value of Scrum lies in the discipline it imposes (a fixed cadence, a written goal per sprint, an increment that must work) rather than in coordination.

1.2.3 Prioritisation and estimation

Requirements and user stories are prioritised with **MoSCoW** (Must, Should, Could, Won't), the prioritisation technique of the DSDM agile framework, in which the Must items form the minimum usable subset that the delivery guarantees [17]. Stories are estimated in **story points** on a Fibonacci-like scale (1, 2, 3, 5, 8, 13), relative to the simplest story: the gaps between values grow with size because estimating larger work is less precise, which is the reason for the scale [18]. Points measure relative size, not hours; their sum per sprint gives the velocity reported in Section 7.7.

1.2.4 Modelling language

Analysis and design use **UML 2.5** [19]: use case diagrams for the functional scope of each sprint, class diagrams for the data model, sequence diagrams for the main interactions, and activity and state diagrams where a decision or a lifecycle is the subject. The diagrams are kept as text sources (PlantUML) or generated (draw.io) in the project repository, versioned beside the code they describe.

1.3 Identification of the actors

The platform has five actors. Table 1.2 lists them with the way each is represented in the data model, because the actors are not invented for the analysis: they follow from two properties every account carries: its **account type** (personal or magazine) and its **role** (reader, writer or administrator).

Table 1.2 – Actors of the system

#	Actor	Data-model representation	Definition
A1	Visitor	no account	Unauthenticated. Browses the public feed and reads free articles; cannot interact.
A2	Reader	personal account	Reads and interacts: likes, comments, reposts, saves, follows, blocks, reports. On the premium plan, also reads premium articles and uses the AI assistant.
A3	Writer	personal account, role writer	Everything a reader does, plus drafting, publishing and analysing their own articles. Once eligible, also publishes premium articles and lists on the marketplace.
A4	Magazine	magazine account	Subscribes, then browses writers, reads evaluation reports, previews, buys and publishes licensed articles.
A5	Administrator	personal account, role admin	Works the moderation queue, manages users and grants eligibility manually.

Three further properties change what an actor may do without making it a different actor: a reader's **plan** (free or premium), a writer's **marketplace eligibility**, and a magazine's **subscription** (active or not). They are states of an account: the same person moves between them by upgrading, by being read, or by subscribing. So they are drawn in the use case diagrams as **conditions** on the use cases they unlock, not as separate actors: *Read a premium article* extends *Read an article* when `[plan = premium]`, *List on the marketplace* extends *Publish an article* when `[writer is eligible]`, and every marketplace use case requires an active subscription.

Two generalisations hold: the reader specialises the visitor, and the writer specialises the reader. The other two actors specialise nobody. A **magazine** is a different account type with no role and no plan (a database constraint enforces it), so it inherits nothing from the readers or writers; a person who runs a magazine and also writes uses two accounts. The **administrator** holds a role that cannot be obtained through the interface and is granted in the database; its capabilities are moderation and management, not an extension of reading or writing, so it is not drawn as a specialised reader either.

1.4 Functional requirements

The functional requirements are stated per actor, each with a stable identifier, a MoSCoW priority and the sprint that delivers it. Identifiers are stable rather than sequential: a requirement added later keeps the next free number and is listed beside the feature it belongs to. Table 1.3 summarises them; the full statements are

given in Annex A.

Table 1.3 – Functional requirements by actor and priority

Actor	Requirements	Must	Should	Could	Sprints
Visitor (A1)	11	9	2	0	1, 4, 5, 8
Reader (A2)	21	16	3	2	1, 2, 3, 5, 7, 8
Writer (A3)	31	24	6	1	1, 2, 3, 4, 5, 8, 9
Magazine (A4)	15	14	1	0	1, 3, 4, 5
Administrator (A5)	5	3	2	0	5
Cross-cutting	4	3	1	0	5
Total	**87**	**69**	**15**	**3**	

The platform counts 87 functional requirements, of which 69 are Must: four in five requirements belong to the minimum the delivery guarantees, which is expected for a platform whose three pillars depend on one another: dropping a Must from one pillar would break another. The Should and Could requirements are the comforts that make each feature pleasant rather than possible: search and browsing by tag, live comment previews, voice, sharing. The writers carry the most requirements (31), because the editor, the assistant, the analytics and the earnings all belong to them; the administrator carries the fewest (5), all delivered in one sprint. A requirement listed under two sprints is delivered in two steps, the second sprint completing the rule the first one prepared; Chapters 3 to 6 say which part each sprint delivers.

1.5 Non-functional requirements

Each non-functional requirement is stated as a property together with the mechanism that provides it and the way it is verified: by an automated **test** of the backend suite, by a **measurement** reported in Chapter 7, by **inspection** of the code or the configuration, or not at all when it is not met. Table 1.4 groups them by quality attribute. Identifiers are stable, as for the functional requirements.

Table 1.4 – Non-functional requirements and how each is verified

ID	Requirement	Mechanism	Verified by
	Security		
NFR-01	Passwords stored as bcrypt hashes, cost factor 12	authentication module, following [20]	inspection
NFR-02	Access tokens expire after 15 minutes, refresh tokens after 7 days	signed JWTs [21]	inspection
NFR-03	Every protected route passes one composed guard stack	one `@Auth()` decorator composing six guards	inspection
NFR-04	Authorisation is decided server-side; the client is told the verdict	the article access decision; locked content arrives as `null`	test
NFR-05	Per-address rate limits, tighter on credential routes	60 requests/min, 10/min on sign-in, 5/min on registration	measurement
NFR-06	Financial operations are idempotent under retry	a unique idempotency key on every ledger transaction	test

ID	Requirement	Mechanism	Verified by
NFR-07	Uploads are presigned, time-limited and never routed through the API	10-minute presigned upload URL	inspection
NFR-08	Datastore ports bind to the loopback interface only	Compose port bindings on `127.0.0.1`	inspection
NFR-09	A ban takes effect within 15 minutes at most	refused at sign-in, Google sign-in and refresh; an access token lasts 15 minutes	test
NFR-10	No stored HTML is ever rendered as markup	no raw-HTML rendering in the frontend	inspection
	Performance		
NFR-11	Semantic retrieval stays sub-linear as the corpus grows	HNSW index (m = 16, ef_construction = 64) [22]	measurement
NFR-12	Full-text search is index-backed with weighted ranking	weighted `tsvector` column and GIN index [23]	measurement
NFR-13	Prompt context is bounded	at most 5 passages, a memory block under 200 tokens	inspection
NFR-14	Feed pagination is cursor-based, stable under insertion	none	**not met**: pagination is by offset
NFR-15	Dashboard totals and rates read pre-aggregated rows	four rollup tables refreshed by the worker	measurement
NFR-16	Analytics aggregation is incremental and idempotent	each run reads only the events since the last one	inspection
NFR-17	Expensive AI output is cached and invalidated by what changes it	insights cached 24 h, dropped when the writer publishes	inspection
NFR-18	Streaming responses are never buffered end to end	nginx `proxy_buffering off`, 300 s read timeout	inspection
	Reliability		
NFR-19	Every denormalised balance equals the sum of its ledger rows	three invariants asserted after every money-moving test	test
NFR-20	Concurrent purchases cannot double-spend	`SELECT … FOR UPDATE` on the balance-owning row	test
NFR-21	Schema changes are versioned and applied before the application starts	a one-shot migration service	inspection
NFR-22	The vector extension exists before any migration that needs it	the migration runner creates it first	inspection
NFR-23	Failed background jobs retry with exponential back-off	none	**not met**: jobs run once
NFR-24	An AI provider outage degrades AI only	a model chain falling back from Groq to Gemini	test
NFR-25	Every service reports liveness and readiness	`GET /health` and `GET /ready`	partly met: readiness checks PostgreSQL only
	Scalability		

ID	Requirement	Mechanism	Verified by
NFR-26	The API holds no session state	identity carried by JWT claims	inspection
NFR-27	Long-running work never runs in the request path	six BullMQ queues consumed by a separate worker	inspection
NFR-28	API and worker scale horizontally behind the proxy	stateless containers, one Compose service each	not measured
	Maintainability		
NFR-29	Both code bases compile in strict mode with zero lint warnings	`tsc --noEmit`, `eslint --max-warnings=0`	continuous integration
NFR-30	The API contract is generated from OpenAPI	client types generated from the backend's OpenAPI document	inspection
NFR-31	Business rules are tested, not asserted	796 backend tests	test
NFR-32	The build fails when schema and migrations disagree	a schema-drift check	continuous integration
NFR-33	Errors are captured in every runtime without personal data	Sentry in API, worker, Next.js server and browser	inspection
	Usability		
NFR-34	Two breakpoints are designed and implemented	375 px and 1280–1440 px	measurement (accessibility at both)
NFR-35	One design system, applied consistently	39 design tokens and 87 components in Figma	inspection
NFR-36	A complete dark mode	theme switch across every screen	inspection
NFR-37	Lighthouse score above 90 and no critical accessibility violation on article pages	production build audited	measurement: partly met

Of the 37 requirements, seven are verified by an automated test and six by a measurement in Chapter 7; two are enforced by the continuous-integration pipeline. The others rest on inspection: they are properties of configuration (port bindings, nginx buffering, Compose dependencies) rather than of behaviour, and a test would only restate the configuration. Five are not fully met and are stated as such: NFR-14 (offset pagination), NFR-23 (no queue is configured with retries, so a failed job is not attempted again), NFR-25 (readiness ignores Redis), NFR-28 (horizontal scaling is designed for but was never exercised) and NFR-37 (met on desktop and for critical violations, not on mobile; see Section 7.5). They return as limitations in the general conclusion.

1.6 Global use case diagrams

The global use cases are drawn in three diagrams, one per group of actors, so that each use case is a single action and every label can be read at page width. The use cases are those of the functional requirements; the per-sprint diagrams of Chapters 3 to 6 then refine each sprint's share. In the three diagrams, every use case of a signed-in actor includes *Sign in*, which is stated once in a note rather than drawn twenty times.

Visitor and reader. Figure 1.3 gives the use cases of the two actors who read.

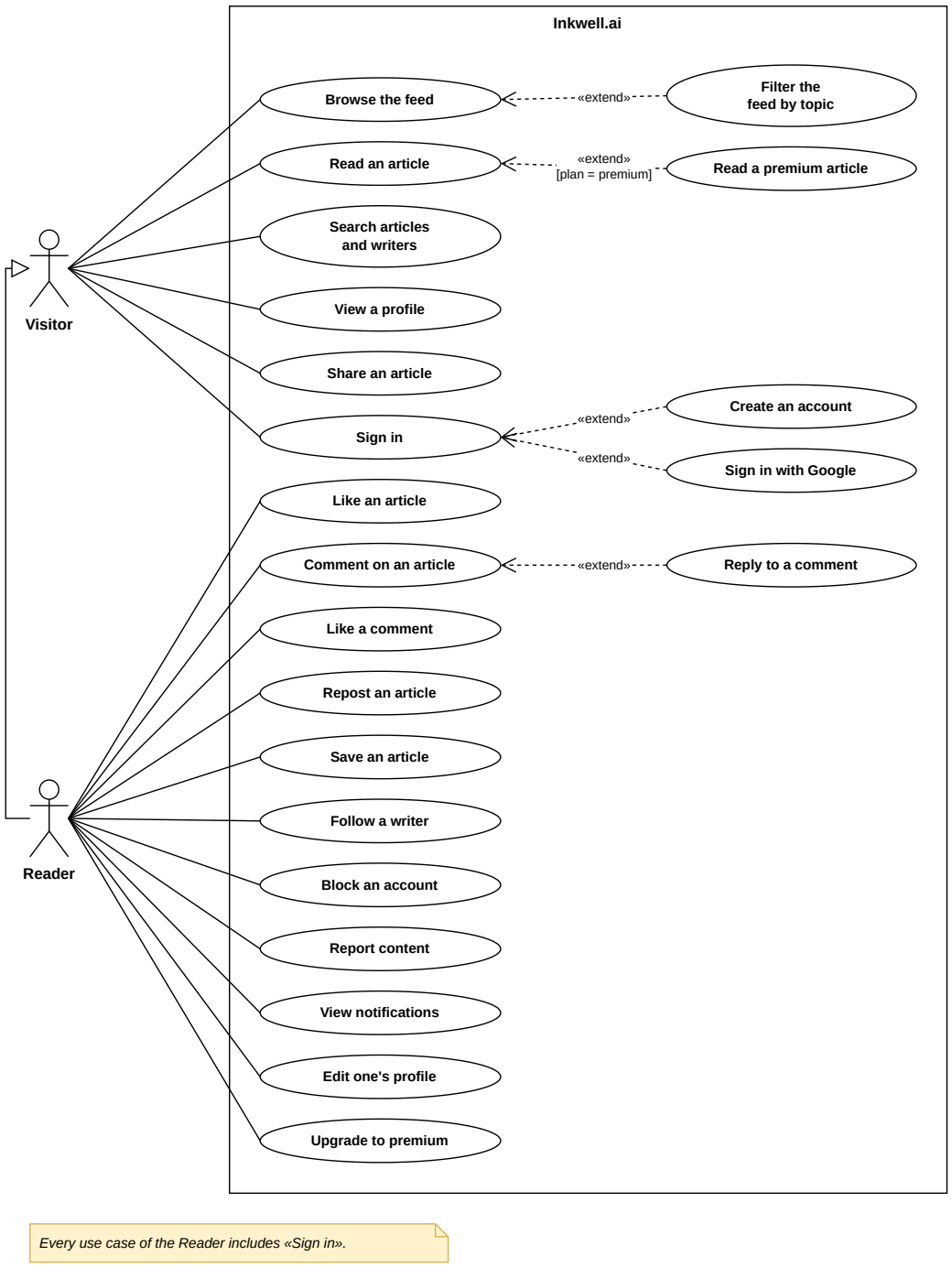


Figure 1.3 – Global use case diagram: visitor and reader

The visitor browses, reads free articles, searches, views profiles and shares; *Create an account* and *Sign in with Google* extend *Sign in*. The reader inherits all of this and adds the social use cases, notifications, the profile and the upgrade. Premium is a condition, not an actor: *Read a premium article* extends *Read an article* when `[plan = premium]`.

Writer. Figure 1.4 gives the use cases of the writer.

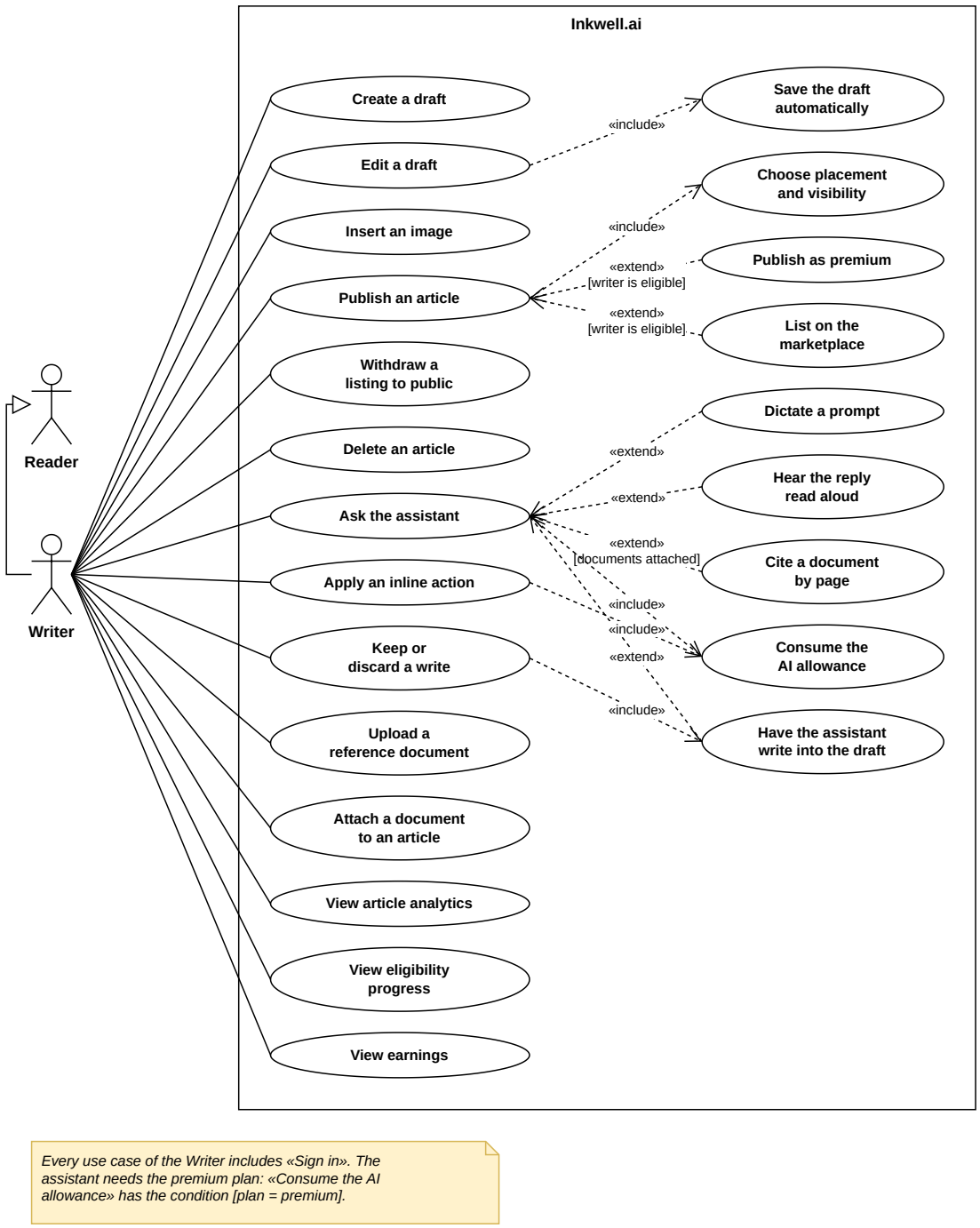


Figure 1.4 – Global use case diagram: writer

The writer inherits the reader's use cases and adds writing, the assistant and their own figures. *Publish as premium* and *List on the marketplace* extend *Publish an article* when `[writer is eligible]`, and the assistant includes the premium AI allowance. Most of the complexity sits around publishing and the assistant.

Magazine and administrator. Figure 1.5 gives the use cases of the two actors that specialise nobody.



Figure 1.5 – Global use case diagram: magazine and administrator

The magazine manages its subscription and credits; with an active subscription it evaluates writers, previews, buys and publishes articles, and the four marketplace use cases include *Check the active subscription*. The administrator works the report queue, manages users and grants eligibility. No generalisation links the two: neither is a kind of reader or writer.

1.7 Global class diagram

The data model counts thirty persisted tables. Drawn in a single diagram they form an unreadable strip, because `User` is related to most of the others, so the global class diagram is split by domain into five figures, each small enough to be read at page width. Attributes are shown with domain types (`Json`, `Vector`, `Timestamp`);

audit columns are omitted; the enumerations each figure uses are drawn underneath it. Each class shows the main **operations** it supports; they are implemented by the service of the same domain (for instance `Article.publish()` by the articles service), because the classes are persisted records and the behaviour lives in the service layer. `User` and `Article` are defined in the first figure and appear in the others as references, with only the attributes and operations that figure needs.

Accounts and articles. Figure 1.6 shows the `User` and its optional `MagazineProfile`, the `Article` with its placement, visibility and price, and its tags.

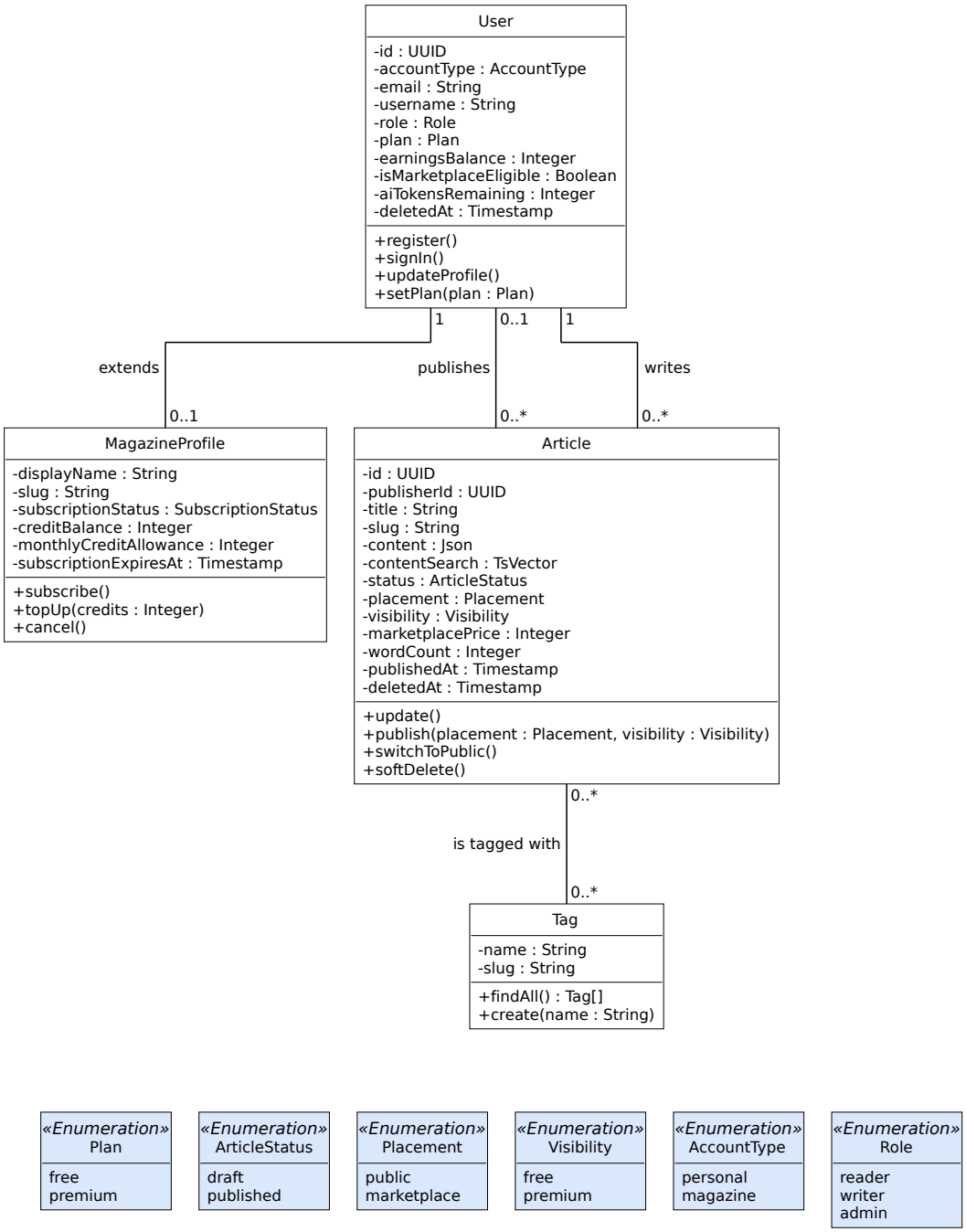


Figure 1.6 – Class diagram: accounts and articles

One `User` class serves every account; type, role and plan are attributes, and a constraint keeps role and plan empty for a magazine, which is the data-model reason it inherits nothing from the reader.

`Article.publisherId`, set only on a licensed article, is the single attribute through which the marketplace changes the core.

Social layer and moderation. Figure 1.7 shows comments, notifications, reports and the social associations.

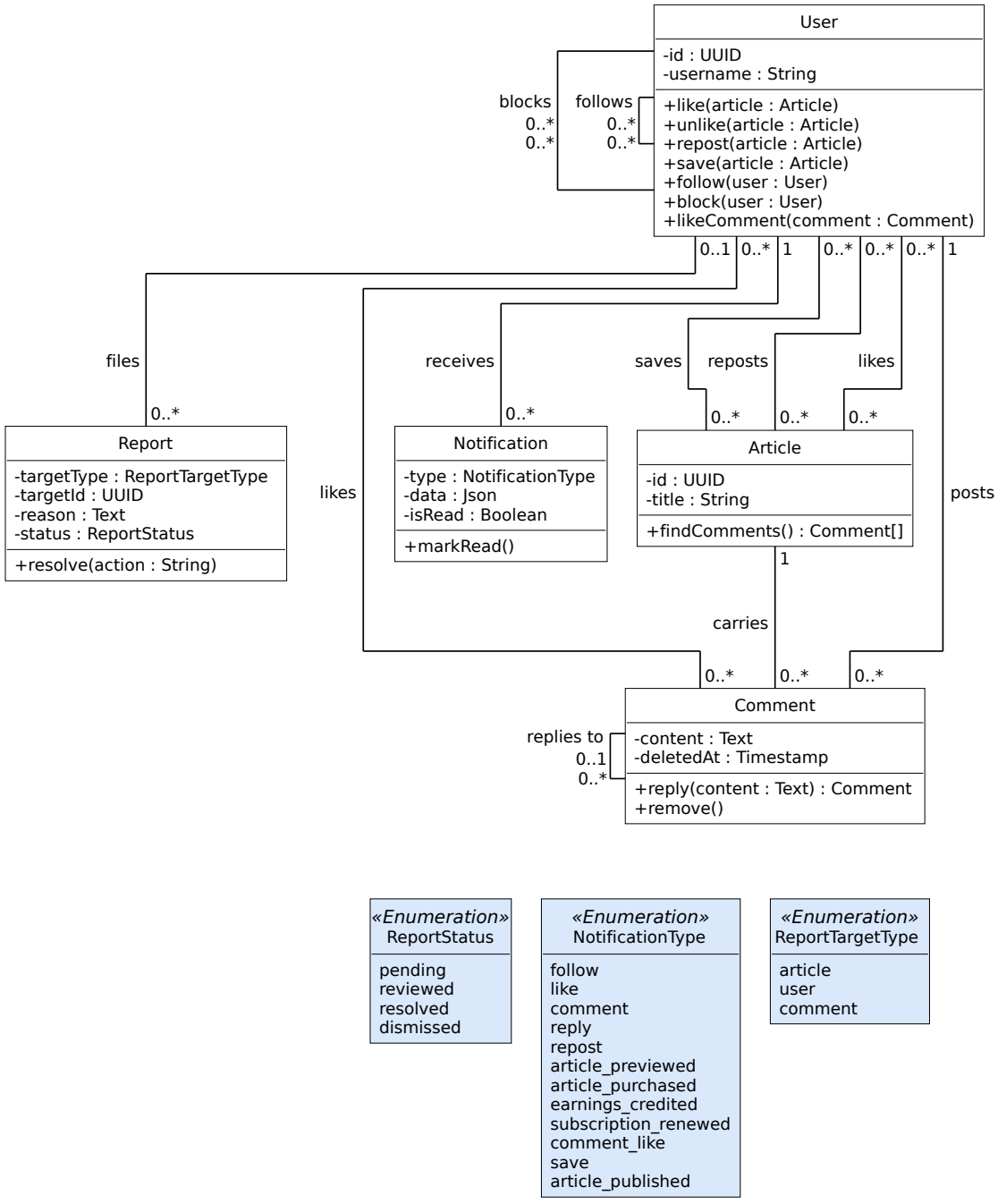


Figure 1.7 – Class diagram: social layer and moderation

Likes, comment likes, reposts, saves, follows and blocks carry only a pair and a date, so they are many-to-many associations stored as join tables with a unique pair; `User` holds the operations that create them. Interactions are therefore cheap to count and cannot be duplicated.

AI and documents. Figure 1.8 shows the article chunks embedded for retrieval, the writer memory, the cached portfolio insights, the AI interaction log, and the private document library with its chunks.

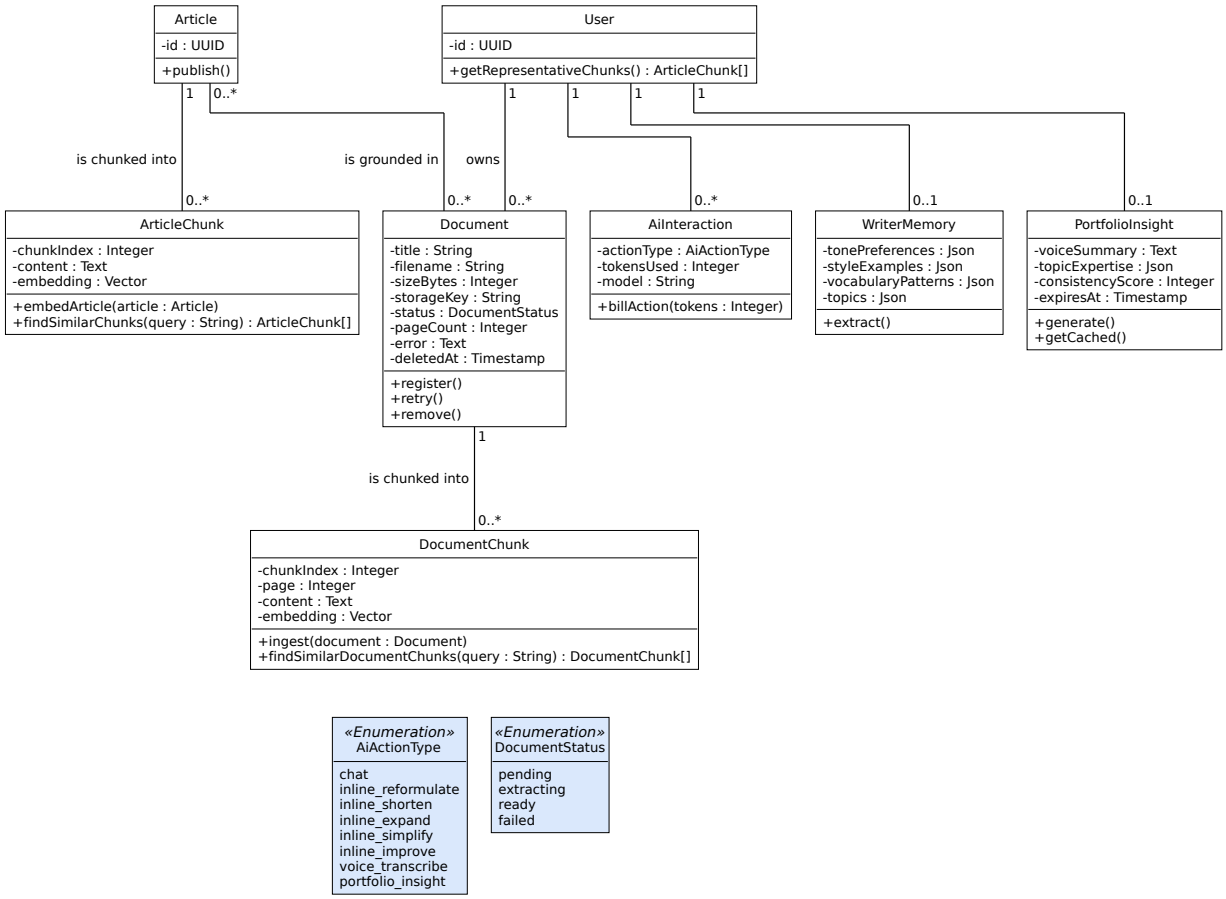


Figure 1.8 – Class diagram: AI and documents

Everything here can be rebuilt from articles and documents: chunks are re-embedded on every publication and the insights are a cache. A `DocumentChunk` keeps its page, which a citation points to, and a document's chunks serve only the articles it is attached to, never memory, insights or search (FR-87).

Marketplace. Figure 1.9 shows purchases, the ledger transactions, magazine subscriptions and the eligibility audit log.

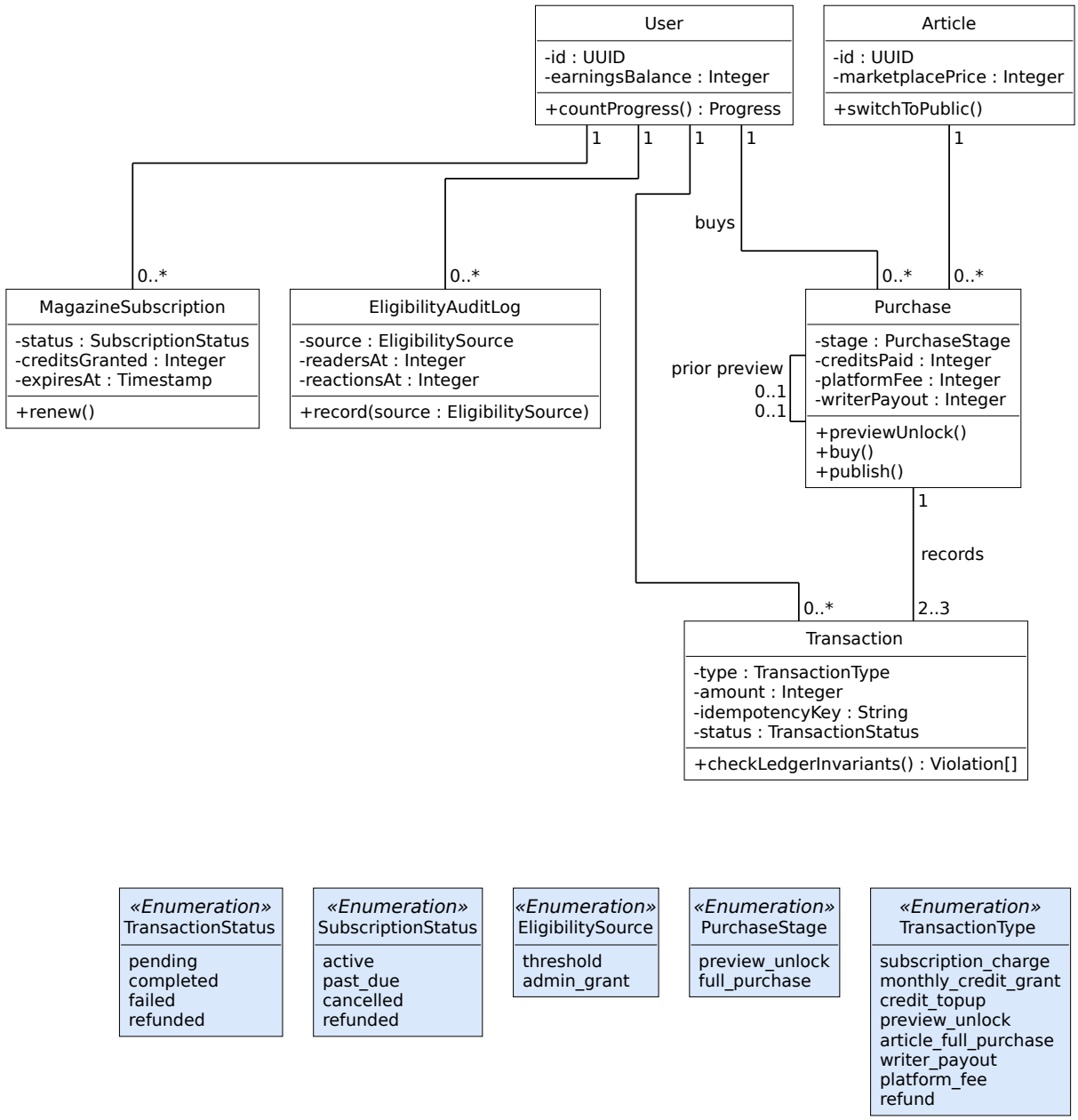


Figure 1.9 – Class diagram: marketplace

Money is never a single number: a `Purchase` records two or three `Transaction` rows (the magazine's debit, the writer's payout and the platform fee), and every balance can be recomputed from them, which is what the ledger invariants of NFR-19 check. Amounts are always positive; their direction comes from the transaction type, enforced by a constraint.

Analytics. Figure 1.10 shows the raw analytics events and the four metric rollups.

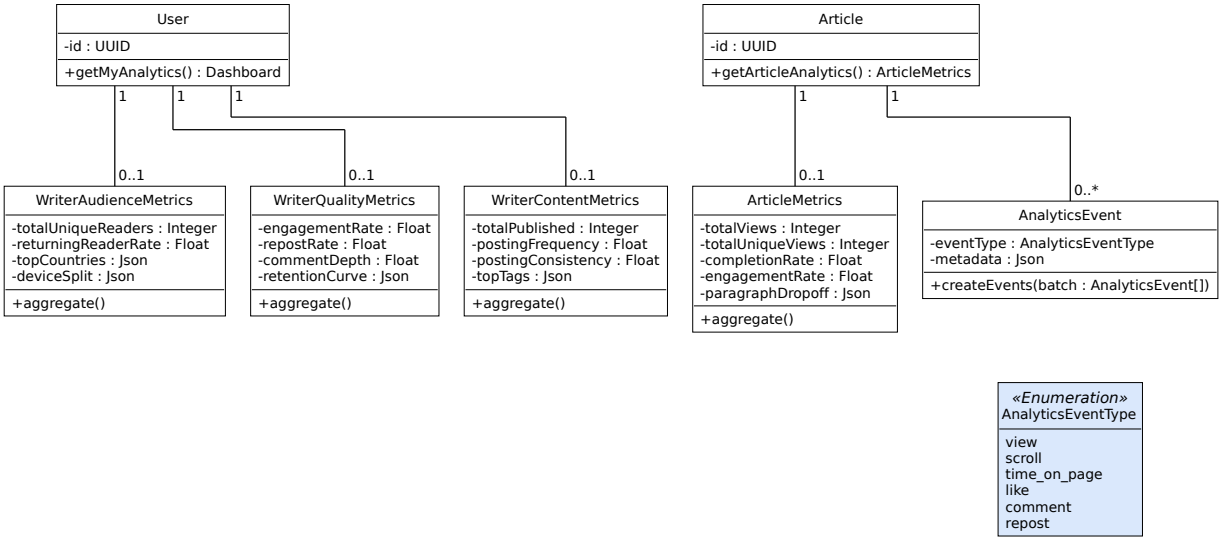


Figure 1.10 – Class diagram: analytics

Raw `AnalyticsEvent` rows are written once and never read by a dashboard, which reads only the rollups, so its cost does not grow with the events (NFR-15). Fields the browser cannot measure (country, device, paragraph drop-off) are stored empty, not zero, so "not measured" stays distinct from "none".

1.8 Product backlog

The product backlog expresses the requirements as user stories of the form *"As an ⟨actor⟩, I want ⟨goal⟩ so that ⟨benefit⟩"*, grouped into nine epics, prioritised with MoSCoW and estimated in story points relative to the simplest story (US-01). The backlog counts **74 stories for 350 points**. Table 1.5 summarises it by epic; the full backlog is given in Annex B.

Table 1.5 – Product backlog by epic

Epic	Stories	Points	Share of points
E1: Accounts and authentication	5	17	5 %
E2: Writing and publishing	8	30	9 %
E3: AI assistance	13	73	21 %
E4: Discovery and search	4	18	5 %
E5: Social interactions and notifications	15	52	15 %
E6: Analytics	4	32	9 %
E7: Marketplace and subscriptions	12	63	18 %
E8: Moderation and administration	6	24	7 %
E9: Quality and operations (technical stories)	7	41	12 %
Total	**74**	**350**	100 %

The distribution of points shows where the effort lies. The AI assistance epic is the heaviest (21% of the points) although it is not the largest in stories: its stories (retrieval grounded in the writer's corpus, writing into the document, voice, the document library) are few but each is large, because each joins a model call to the editor and to the data model. The marketplace comes next (18%), for the same reason applied to money: a small number of stories, each guarded by transactions, locks and ledger checks. The social epic has the

most stories but small ones, and the technical epic E9 (12%) is the price of a platform that runs, migrates and is tested with one command. In short, the effort follows the product's ambition: two of its three pillars, AI and the marketplace, take almost two fifths of the points.

1.9 Release plan and schedule

The stories are delivered in four releases of two or three sprints each, every sprint lasting two weeks. Table 1.6 gives the plan with the points committed per sprint. The foundation week and the three-sprint design track are not sprints, because they deliver no user-facing increment; they are presented with the working environment in Chapter 2.

Table 1.6 – Release plan

Release	Sprint	Dates	Goal	Points
R1: Functional foundation	Sprint 1	09/02 – 20/02	Schema, authentication, article core, ledger invariant harness	45
	Sprint 2	06/04 – 17/04	Editor, AI chat and inline actions, analytics event capture	45
R2: Social, analytics and AI	Sprint 3	20/04 – 01/05	Social interactions, notifications, aggregation, dashboards	44
	Sprint 4	04/05 – 15/05	Retrieval-augmented generation, portfolio insights, hybrid search, writer memory	52
R3: Marketplace, testing and deployment	Sprint 5	18/05 – 29/05	Marketplace, subscriptions, premium, moderation, administration	85
	Sprint 6	01/06 – 12/06	Tests, demonstration corpus, deployment, CI/CD	20
R4: Social depth, assistant and documents	Sprint 7	15/06 – 26/06	Comment likes, follower lists, block, save, share	18
	Sprint 8	29/06 – 10/07	Live card controls, reposts, byline preview, assistant writing into the document, voice	38
	Sprint 9	13/07 – 24/07	Document library, ingestion, attachment, citations	16

The order of the releases follows the layering argued in Section 1.2: each release needs the data the previous one produces. The loads are uneven. Sprint 5 commits 85 points against an average near 40, because the marketplace stories (subscription, credits, preview, purchase, publication, earnings) share one transaction model and were planned together rather than split across two sprints. Sprints 6, 7 and 9 are light, for two different reasons: Sprint 6's real work, the test suite that proves the business rules, is not expressed as user stories, and Release 4 runs alongside the writing of this report. How this plan compares with what was delivered is measured in Section 7.7.

Figure 1.11 places the work on the project window, from 2 February to 31 July 2026: the foundation week and Sprint 1, the design track, Sprints 2 to 9 in four releases, and the writing of this report alongside the last sprints.

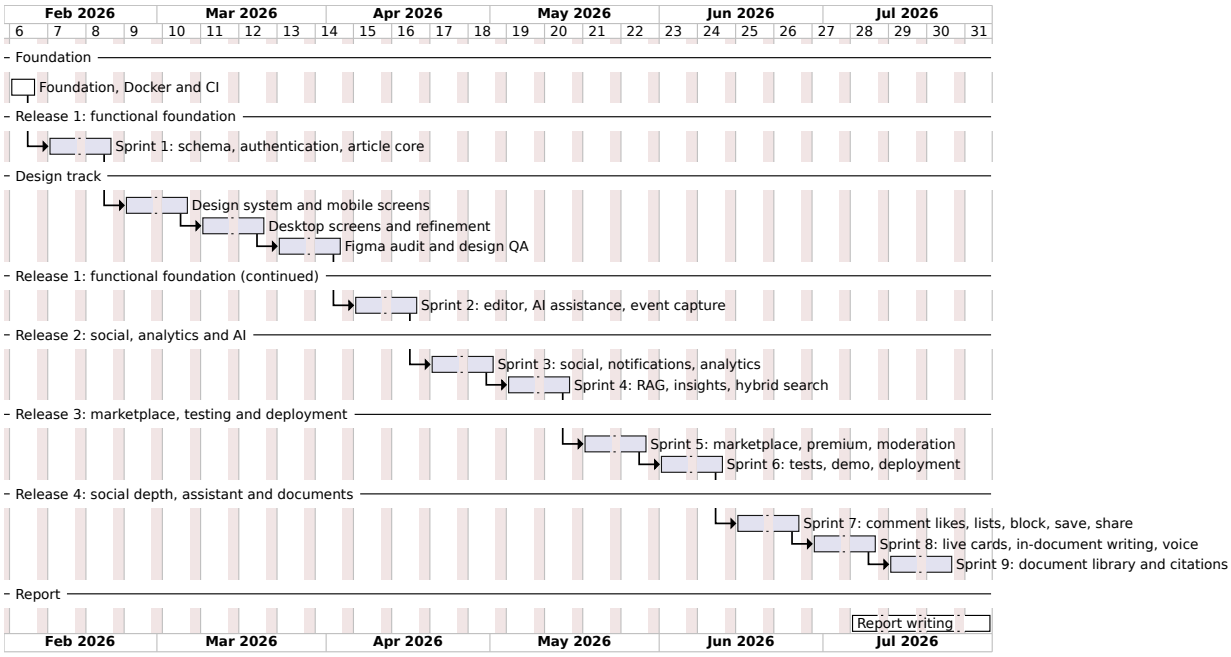


Figure 1.11 – Gantt chart of the project

The chart shows one break in the rhythm: the six-week design track between Sprints 1 and 2. It was deliberate, since later features were built from finished screens rather than designed while coded, and it is why the first release spans two months.

Conclusion

This chapter situated the project, showed that no existing family of solutions combines AI writing grounded in the writer's own work, buyer-facing analytics and a licensing market, and justified the use of Scrum and its adaptation to a one-person project. It then identified the five actors, specified 87 functional and 37 non-functional requirements with the way each is verified, presented the global use case and class diagrams, and organised the work into a backlog of 74 user stories delivered by nine sprints in four releases. The next chapter describes the environment in which the work was done, the technological choices and the architecture of the system.

CHAPTER 2

Working environment and architecture

Introduction

Before the release chapters, this chapter describes the environment the platform was built in: the hardware and software used, the work that preceded the feature sprints, the technological choices compared with their alternatives, the main parameters of the system and their justification, and the architecture of the system: its logical decomposition on the frontend and the backend, its physical deployment and its delivery pipeline.

2.1 Working environment

2.1.1 Hardware environment

The project was developed and every measurement of Chapter 7 was taken on a single workstation, described in Table 2.1. A mid-range laptop is a deliberate baseline: the whole stack of eight containers runs on it, so the performance figures reported later are those of commodity hardware, not of a server.

Table 2.1 – Hardware environment

Component	**Specification**
Machine	Lenovo ThinkPad
Processor	Intel Core i7-1355U
Memory	24 GB RAM
Graphics	NVIDIA GeForce MX550
Storage	512 GB NVMe SSD
Operating system	Ubuntu Linux

2.1.2 Software environment

Table 2.2 lists the tools used outside the application code itself. They share a principle: every artefact (code, diagrams, this report) is kept as text under version control, and everything that runs, runs in a container.

Table 2.2 – Software tools

Tool	**Use**
JetBrains IDE	code editor
Git and GitHub	version control, pull requests, continuous integration (GitHub Actions) and container registry (GHCR)
Docker and Docker Compose	containers for every service, in development and in production [24]
Figma	design system and screen design
PlantUML and draw.io	UML and architecture diagrams, kept as versioned sources
Swagger UI	exploring the REST API through the OpenAPI document the backend generates
Playwright	end-to-end tests and the capture of this report’s screenshots [25]

Tool	Use
Pandoc and LuaLaTeX	assembling and typesetting this report from its Markdown sources

2.1.3 Foundation and design track

Two blocks of work preceded the feature sprints and are presented here rather than as sprints, because neither delivers a user-facing increment.

- **Foundation (one week).** The three repositories, their Docker images, the Compose stack, the reverse proxy and the continuous-integration pipeline were set up first, so that every later sprint started from a stack that builds, tests and runs with one command.
- **Design track (three sprints).** The user interface was designed in Figma before the editor and the social features were built: a design system of **39 tokens and 87 components**, then the full screen set at two widths (**375 px for mobile and 1280–1440 px for desktop**, 58 screens), then an audit of the screens against the specification. The release chapters implement these designs.

2.1.4 Repository organisation

The source code lives in four Git repositories. `docker.inkwell.ai` is the superproject: it holds the Compose files, the nginx configuration, the deployment workflow and the developer Makefile, and it carries the three others as **Git submodules**: `backend.inkwell.ai`, `frontend.inkwell.ai`, and `spec.inkwell.ai`, which holds the specifications, the diagrams and this report. A commit of the superproject therefore identifies one exact, deployable revision of the whole system.

2.2 Technological choices

Each choice below is compared with the alternatives that were considered and the criteria that decided between them, with a source for every claim that is not a property of the project itself.

2.2.1 Frontend

Table 2.3 gives the frontend stack. Its central decision is the web framework: the platform's public pages (the feed, articles, profiles) must be rendered on the server so that search engines and link previews see their content, while the editor and the assistant are interactive applications.

Table 2.3 – Frontend technologies compared with their alternatives

Choice	Alternatives considered	Deciding criteria
Next.js 16 (App Router) with **React 19**	Remix, Nuxt (Vue), a client-only React application	server rendering per route for public pages and a request proxy for protected routes, in the most used React framework [26], [27]; a client-only application would serve empty HTML to search engines
TypeScript (strict)	JavaScript	one typed language from the database schema to the browser; the API client types are generated from the backend (NFR-30)

Choice	Alternatives considered	Deciding criteria
Tailwind CSS 4 with accessible headless primitives	CSS modules, a component library such as MUI	the Figma tokens map directly to utility classes; headless primitives give accessible dialogs and menus without imposing a visual style
TipTap 3	Lexical, Slate, Quill	built on ProseMirror, stores the document as JSON, and accepts custom extensions, which the protected range the assistant writes into needs [28]
TanStack Query 5 and **Zustand 5**	Redux Toolkit	server state (caching, invalidation, optimistic updates) and client state handled separately, with far less code than a single global store
Vercel AI SDK (React bindings)	hand-written stream parsing	consumes the assistant's typed stream of text, status and write parts [29]

2.2.2 Backend

Table 2.4 gives the backend stack. Its two structuring decisions are the framework, which must host both an HTTP API and a background worker, and the data store, which must hold relational data, full-text search and vector search.

Table 2.4 – Backend technologies compared with their alternatives

Choice	Alternatives considered	Deciding criteria
NestJS 11	Express or Fastify alone	modules, dependency injection, guards and OpenAPI generation built in [30]; the same modules serve the API and the worker. Fastify is faster in raw benchmarks [31], but the API's latency is dominated by the database and the model calls (Section 7.5), not by the HTTP layer
PostgreSQL 16 + pgvector	PostgreSQL with a separate vector database (Qdrant, Pinecone)	relational data, full-text search and vector search in one database [23], [32]: an article and its embeddings change in one transaction and are never out of sync, and there is one service less to run. HNSW, the index pgvector offers, is among the best recall-per-latency trade-offs in public benchmarks [33]
Drizzle ORM	Prisma, TypeORM	SQL-shaped, type-safe queries with explicit migrations [34]; the purchase transaction needs `SELECT … FOR UPDATE` and the search needs `ts_rank` and vector operators, which Drizzle writes directly
Redis 7 + BullMQ	RabbitMQ, PostgreSQL-based queues	repeatable jobs, priorities and a separate worker process on one Redis instance [35]; Redis also carries the notification fan-out
MinIO	storing files in PostgreSQL, a cloud bucket	the S3 API, so the same presigned-upload code runs against a cloud bucket in production; files never pass through the API
Passport (JWT, Google OAuth 2.0) and **bcrypt**	an external identity provider	standard strategies [21], [36] with the session data kept in the platform's own database

2.2.3 Artificial-intelligence services

Table 2.5 lists the models and why each was chosen. Two constraints shaped all of them: the project runs on providers' free tiers, and a model that is slow to *start* answering makes the assistant feel broken, whatever its quality.

Table 2.5 – AI models, their alternatives and the reasons for the choice

Use	Model	Alternatives considered	Deciding criteria
Chat, writing, inline actions, insights, writer memory	Groq `gpt-oss-120b`, then `gpt-oss-20b`, then Google `gemini-3.5-flash`	OpenAI API, a single provider	Groq serves open-weight models on dedicated inference hardware with a generous free tier [37]; a second provider makes an outage degrade the assistant only (NFR-24)
Embeddings	Google `gemini-embedding-001`, 1,536 dimensions	OpenAI embeddings, a local model	separate task types for queries and documents [38]; its free tier covered the corpus
Speech-to-text	Groq `whisper-large-v3-turbo`	browser speech recognition	same provider as the chat; multilingual; works in every browser
Text-to-speech	Google `gemini-2.5-flash-preview-tts`	browser speech synthesis	a natural voice, identical in every browser
Moderation at publish	Groq `gpt-oss-20b`	a dedicated safety model (`gpt-oss-safeguard-20b`)	measured: precision 0.93 and recall 1.00 on a labelled set, while the dedicated model exceeded the time budget (Section 7.4)

The chain of chat models is what makes a provider outage degrade the assistant only: the first chunk of a response is pulled before anything is sent to the browser, so a refusal from one model can still be answered by the next. The moderation classifier is the one choice settled by measurement rather than by documentation; Chapter 7 reports the measurement.

2.2.4 Infrastructure and quality

Docker Compose runs eight services: nginx, web, api, worker, a one-shot migration service, PostgreSQL, Redis and MinIO. **nginx** terminates TLS and routes traffic. **GitHub Actions** runs the quality gates and builds the images; **GHCR** stores them. **Jest** tests the backend against a real PostgreSQL database, **Playwright** drives a real browser end to end, and **Sentry** captures errors in the four runtimes. Kubernetes was considered and rejected: a single host with Compose is enough for the expected load, and the containers are already stateless (NFR-26) should the platform ever need an orchestrator.

2.3 Main parameters and their justification

The behaviour of the platform depends on a few numbers. Table 2.6 gives each, with its value and the reason for it: a source, a measurement, or a product decision stated as such.

Table 2.6 – Main parameters of the system and their justification

Parameter	Value	Justification
Passages retrieved per question	5	enough context for an answer while keeping the prompt near 2,000 tokens (NFR-13)
Similarity floor for retrieval and search	0.60	measured: relevant passages score 0.66–0.84 and off-topic ones 0.52–0.66 (Section 7.2)
Chunk length	120 – 1,200 characters, prefixed by the nearest heading	a paragraph is the unit of meaning; shorter fragments embed poorly, longer ones mix topics
Fusion constant of RRF	k = 60	the value recommended by the method's authors [39]; confirmed in Section 7.3
HNSW index	m = 16, ef_construction = 64	pgvector's defaults, a balanced recall–build-time setting [22], [32]
Embedding width	1,536	supported output width of the model [38]; fixed by the schema
Premium AI allowance	20,000 tokens a day	product decision: about twenty long replies or sixty inline edits, a working day
Voice billing	200 tokens per started minute, at most 5 minutes	product decision, so that dictation consumes the same allowance as typing
Preview price	10% of the article's price	product decision, credited towards the purchase
Platform fee	20% of each stage, rounded down	product decision; computed in integer basis points so that no amount is ever a float
Monthly credits	500 per subscription	product decision; unspent credits roll over
Eligibility thresholds	5,000 unique readers and 1,000 reactions	product decision: high enough that eligibility signals a real audience
Token lifetimes	15 minutes (access), 7 days (refresh)	a short access token bounds how long a ban or a stolen token stays valid (NFR-09)
Rate limits	60/min, 10/min on sign-in, 5/min on registration	enough for a reader, too few for password guessing
Password hashing	bcrypt, cost 12	about 250 ms per hash: slow enough to make an offline attack expensive [20]
Presigned upload lifetime	10 minutes	long enough for a slow mobile upload, short enough that a leaked URL is stale
Document limits	10 MB, 200 pages, 20 per writer	bound the cost of ingestion and the size of the library

The table shows three kinds of justification. The retrieval parameters are set by measurement or by the literature, and Chapter 7 checks them. The security parameters follow published guidance. The economic parameters (prices, fee, credits, thresholds) are product decisions: they have no "correct" value, and they are named constants in one place in the code precisely so that they can be changed as a decision rather than discovered in a calculation.

2.4 Logical architecture

2.4.1 Frontend

Figure 2.1 shows the logical architecture of the frontend, organised in layers from the routes down to the network.

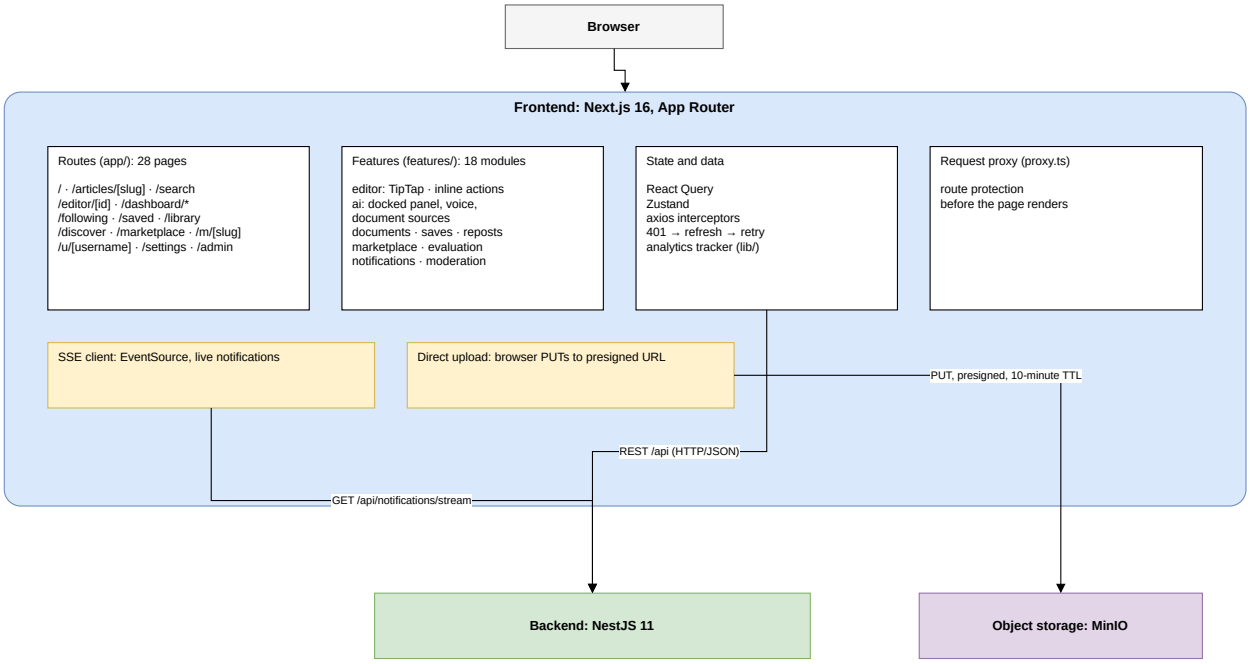


Figure 2.1 – Logical frontend architecture

The **routes** (28 pages) compose **feature modules** (18) over a **state and data** layer (query cache, stores, token-refresh interceptors, analytics tracker), and a **request proxy** guards private routes. Only live notifications and direct uploads bypass the API, so token refresh and error handling are written once.

2.4.2 Backend

Figure 2.2 shows the backend, which runs as two processes from the same image.

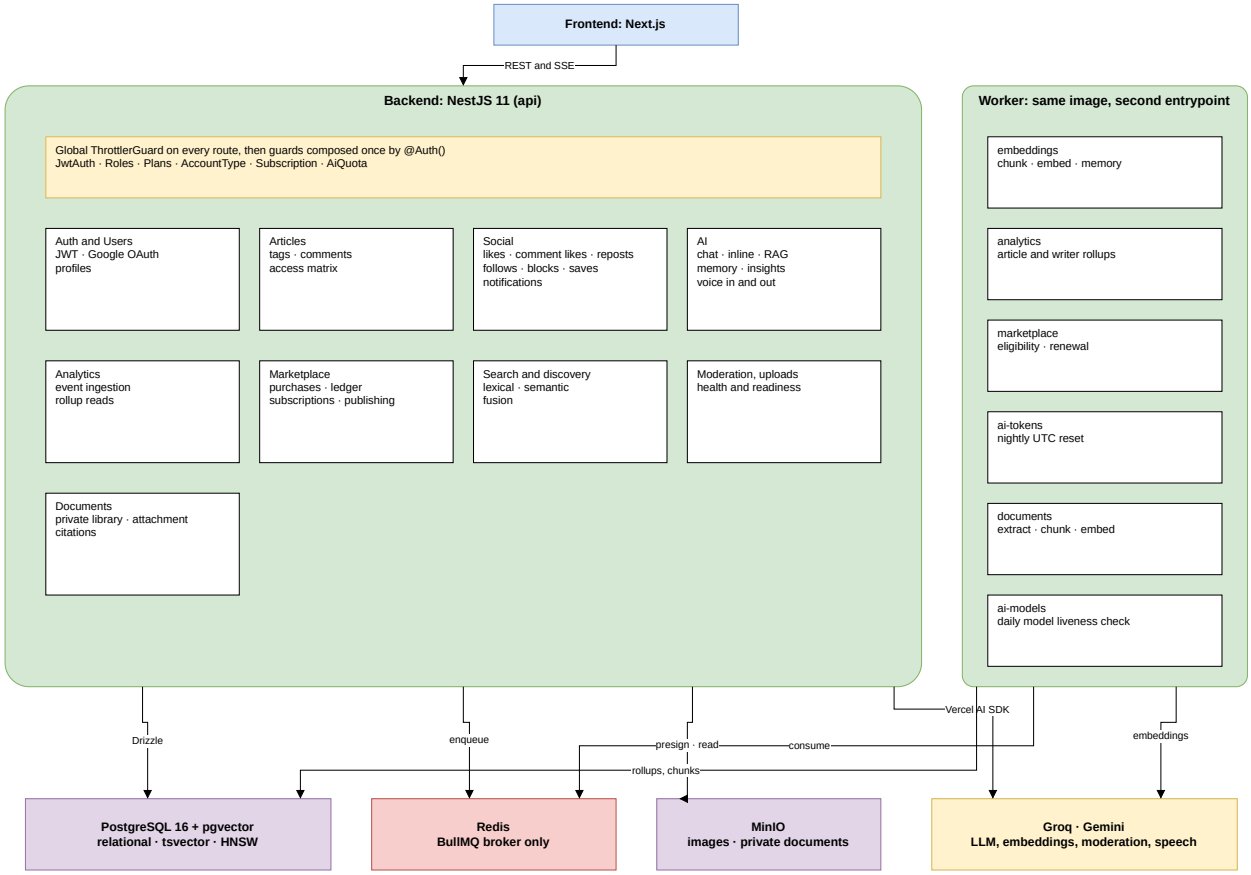


Figure 2.2 – Logical backend architecture

The **API** passes every request through a rate limiter, then the guards composed by `@Auth()`, into feature modules by domain. The **worker** runs from the same image and consumes six queues: embeddings, analytics, marketplace, AI tokens, documents and model checks. Everything slow or scheduled lives in the worker, so no API response waits for background work (NFR-27).

2.5 Physical architecture

Figure 2.3 shows the deployment of the eight services on one host.

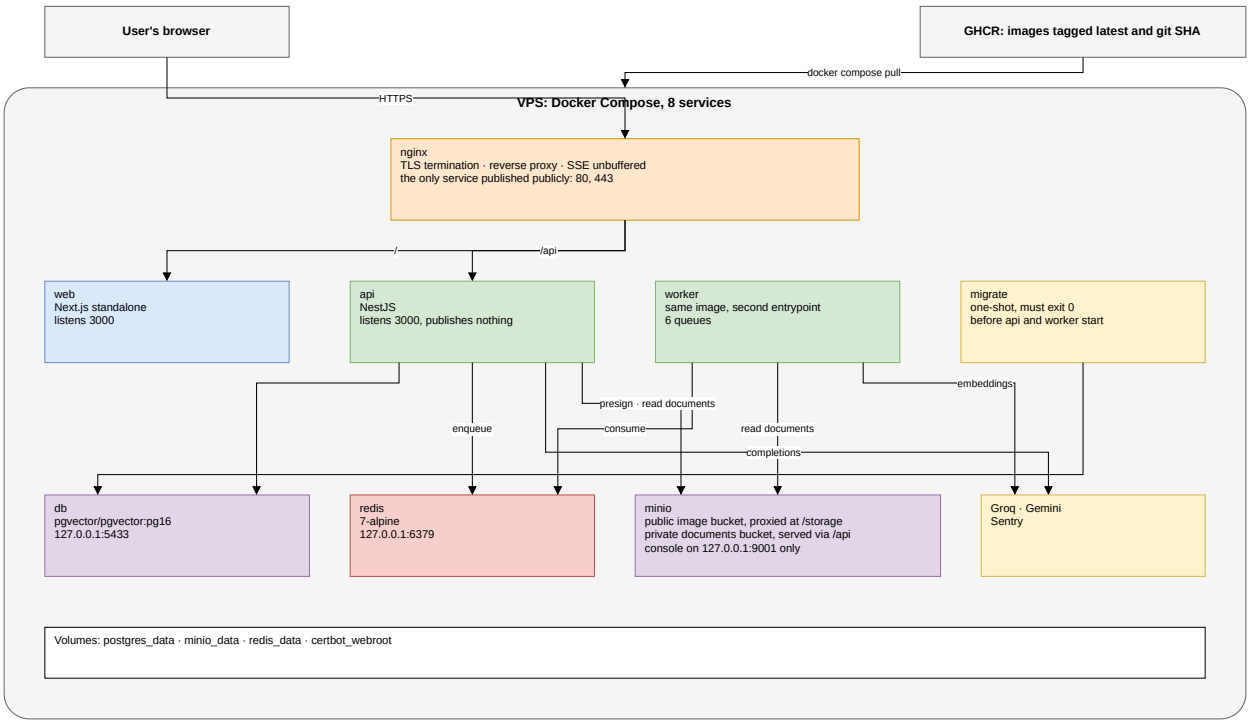


Figure 2.3 – Physical architecture

nginx is the only service published (ports 80 and 443): it terminates TLS and routes to Next.js, the API and the public image bucket. Private documents go through the API, which checks ownership, and the datastores stay inside the Compose network: one attack surface and one place to configure TLS and timeouts.

2.6 Continuous integration and delivery

Figure 2.4 shows the pipeline from a push to a running service.

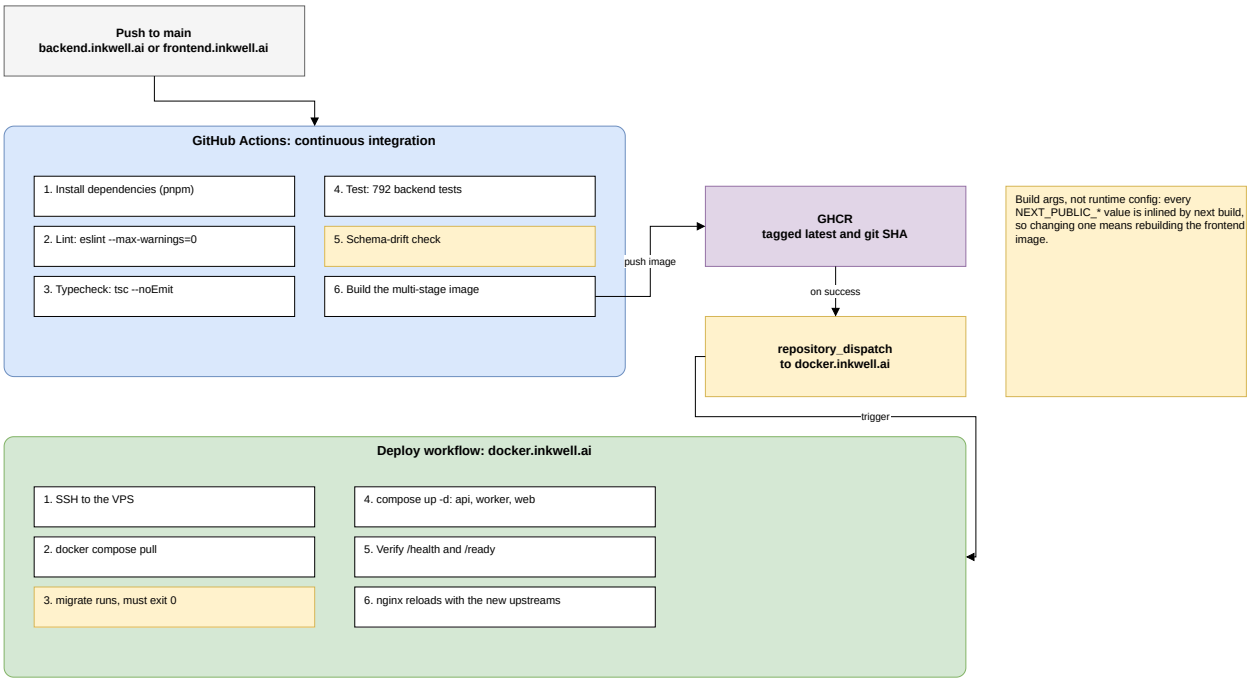


Figure 2.4 – CI/CD pipeline

A push to `main` runs the quality gates, then builds an image tagged `latest` and with the commit hash; the deployment workflow pulls it, migrates, restarts the services and checks their health. No image skips the gates, and every deployed version can be traced to its commit.

Conclusion

This chapter presented the working environment, compared the technological choices with their alternatives, justified the main parameters of the system, and described its logical and physical architecture and its delivery pipeline. The following chapters present the four releases, starting with the functional foundation.

CHAPTER 3

Release 1: functional foundation

Introduction

The first release lays the foundation every later feature depends on. Sprint 1 delivers the data model, authentication and the core of article management; Sprint 2 delivers the rich-text editor, the first version of the AI assistant and the capture of reading events that the analytics of Release 2 will aggregate. Each sprint is presented in the same four steps: its backlog, the analysis, the design and the realisation.

3.1 Sprint 1: Schema, authentication and article core

3.1.1 Sprint backlog

The goal of Sprint 1 is that a visitor can register and sign in, by password or with Google; that a writer can create, edit, publish and delete articles, choosing their placement and visibility; that a visitor can browse the public feed and read free articles; and that the ledger invariant harness exists before any feature moves money. Table 3.1 lists the stories committed.

Table 3.1 – Sprint 1 backlog

ID	User story (goal)	Priority	Points
US-01	Register with an email and a password	Must	3
US-02	Sign in with Google	Should	3
US-03	Register as a magazine with my branding	Must	5
US-04	Stay signed in across a long writing session	Must	3
US-05	Edit my profile and avatar	Should	3
US-10	Tag an article	Should	2
US-11	Choose at publish time whether an article is public or for sale	Must	5
US-49	Prove that every balance equals its ledger sum, after every money-moving operation	Must	8
US-50	Bring the whole stack up with one command, reproducibly	Must	8
US-51	Fail the build on a lint error, a type error, a failing test or schema drift	Must	5
	10 stories		**45**

The sprint commits 45 points, close to the average of the project. Almost a fifth of them (US-49, US-50, US-51) produce nothing a user sees (the invariant harness, the one-command stack and the quality gates) and were placed first on purpose: every later sprint is built and verified with them.

3.1.2 Analysis

Use case diagram. Figure 3.1 refines the global diagram for the scope of Sprint 1.

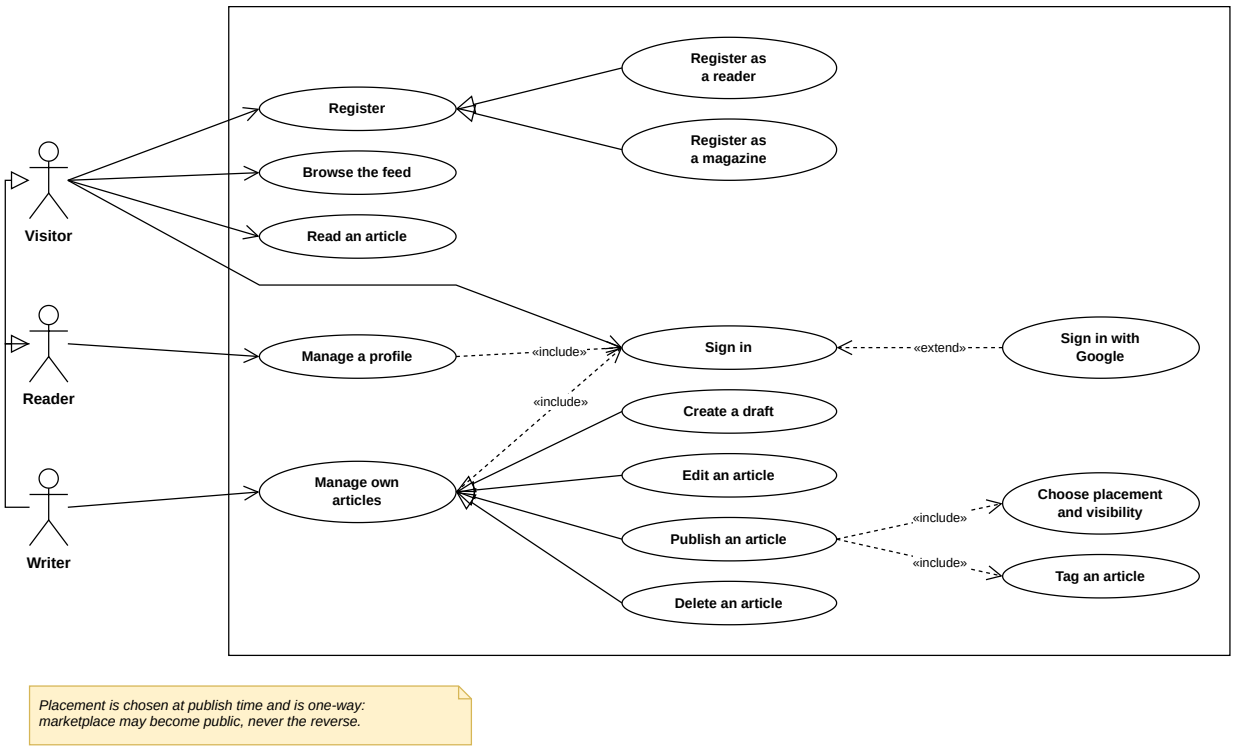


Figure 3.1 – Use case diagram, Sprint 1

Registering specialises into a personal or a magazine account; managing one's articles into drafting, editing, publishing and deleting, with publishing including a placement and a visibility. From the first sprint, account type and placement, the two properties the marketplace rests on, are already in place.

Textual description. Table 3.2 details the central use case of the sprint.

Table 3.2 – Textual description of "Publish an article"

Item	Description
Use case	Publish an article
Actor	Writer (A3)
Pre-condition	The writer is signed in and owns a draft
Post-condition	The article is published with its placement, visibility, slug and reading time
Main scenario	1. The writer opens the publish dialog. 2. The writer chooses a placement (public or marketplace) and, for a public article, a visibility (free or premium), and sets the cover, tags and excerpt. 3. The system checks the choices. 4. The system publishes the article and computes its word count and reading time. 5. The article appears in the feed (public) or is kept for the marketplace (marketplace).
Alternative scenario	3a. A marketplace placement without a price: the system refuses and keeps the draft.

The description shows the scope of Sprint 1 for publishing: the placement and the visibility are stored and validated, but nobody yet acts on them. From Sprint 5, the premium visibility and the marketplace placement are reserved to eligible writers (FR-35, FR-36), which adds a refusal at step 3; FR-23 and FR-24 are therefore delivered in two steps, and the backlog records both sprints.

3.1.3 Design

Class diagram. Figure 3.2 shows the classes of Sprint 1.

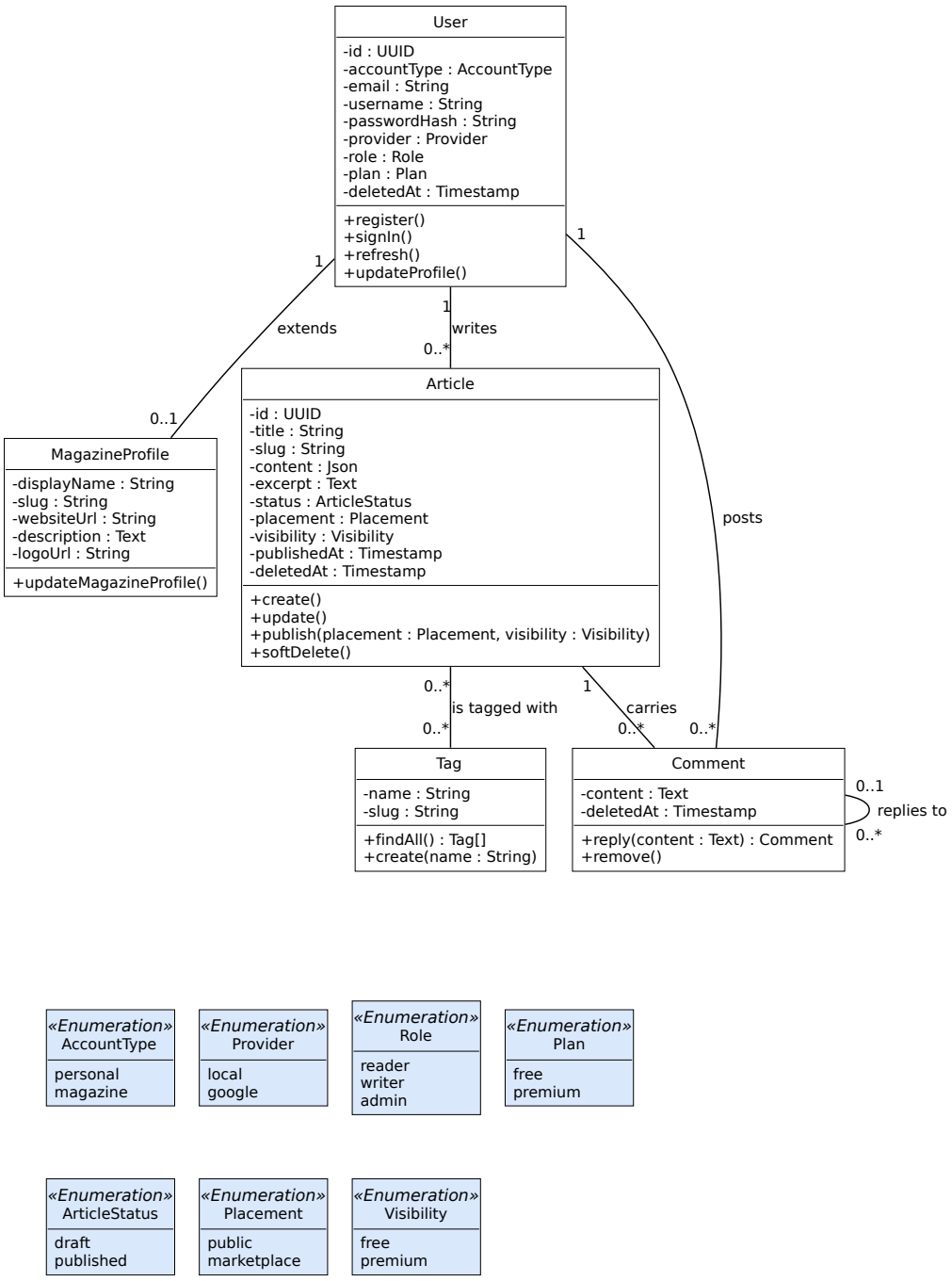


Figure 3.2 – Class diagram, Sprint 1

The `User` carries its account type, provider, role and plan; the `MagazineProfile` adds a magazine's name, slug and branding; the `Article` carries its status, placement, visibility and search vector. One `User` class serves every kind of account, the differences held in attributes, which is what lets one authentication path serve every actor.

Sequence diagram: sign in. Figure 3.3 shows signing in and the silent refresh that keeps a session alive.

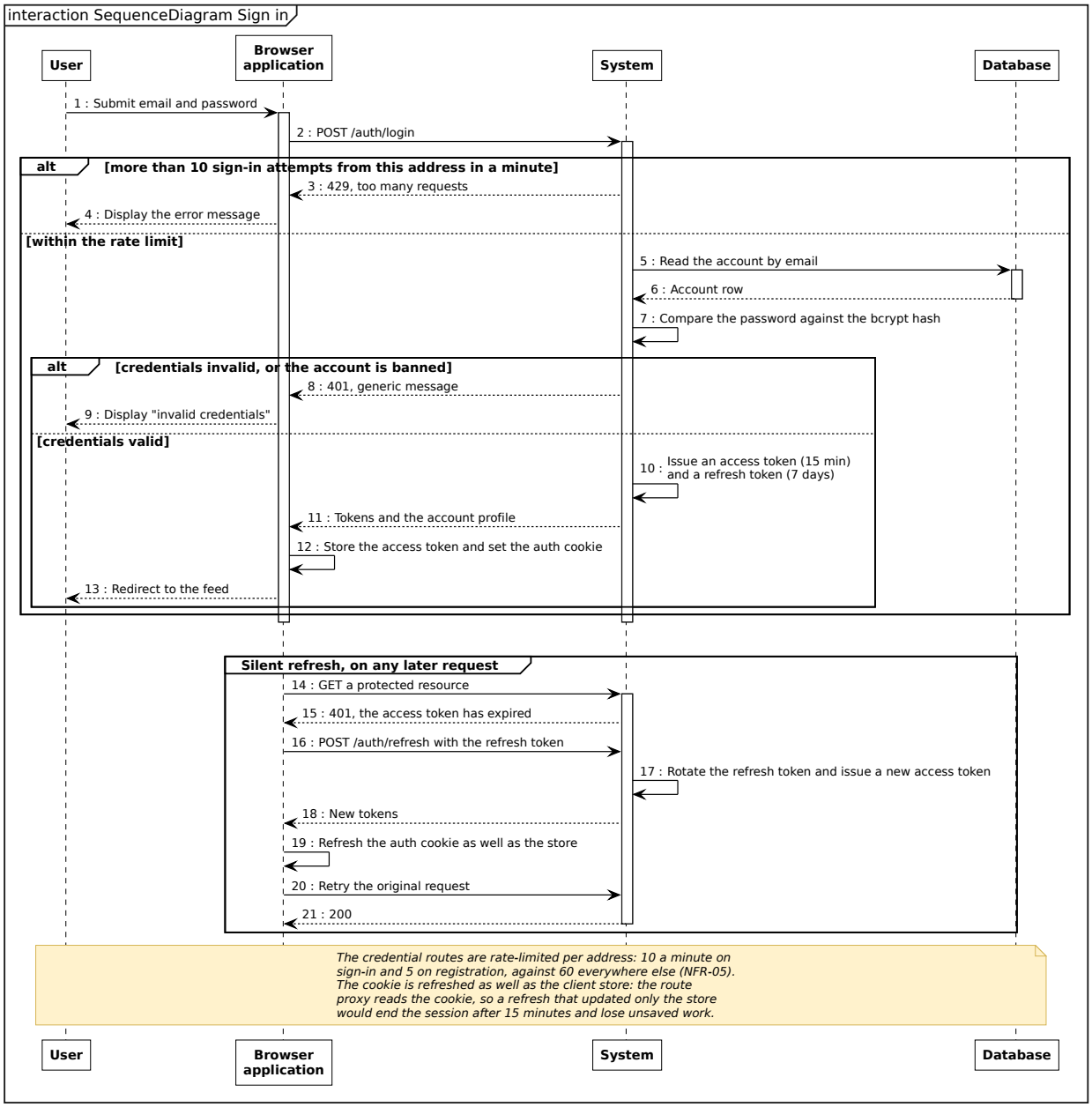


Figure 3.3 – Sequence diagram: sign in and silent refresh

Sign-in is rate-limited (429 after ten attempts a minute) and returns a 15-minute access token and a 7-day refresh token; an expired access token is refreshed and the request retried transparently. A long writing session is never interrupted (US-04), yet a revoked token lasts fifteen minutes at most (NFR-09).

The ledger invariant harness. Money appears in Sprint 5, but its rule is set here: every balance must equal the sum of its ledger rows. Three invariants check it after every test that moves credits, and the harness is itself tested by sabotage: corrupting one column must fail the right invariant. No purchase code was ever written without this check.

3.1.4 Realisation

The realisation of Sprint 1 is what a visitor sees first. Figure 3.4 shows the public feed reached without an account.

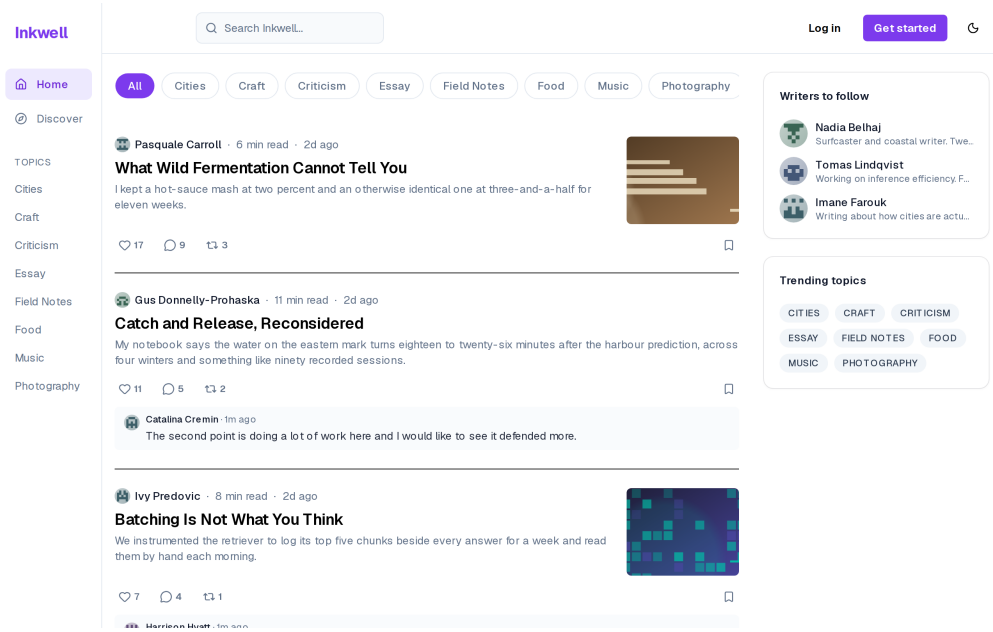


Figure 3.4 – The public feed, as seen by a visitor

The feed lists published public articles with their author, reading time, excerpt, cover and reaction counts, filtered by topic along the top (FR-01). The header offers only *Log in* and *Get started*: a visitor can read and browse but every interaction leads to an account, which is the boundary between the visitor and the other actors.

Figure 3.5 shows the sign-in page.

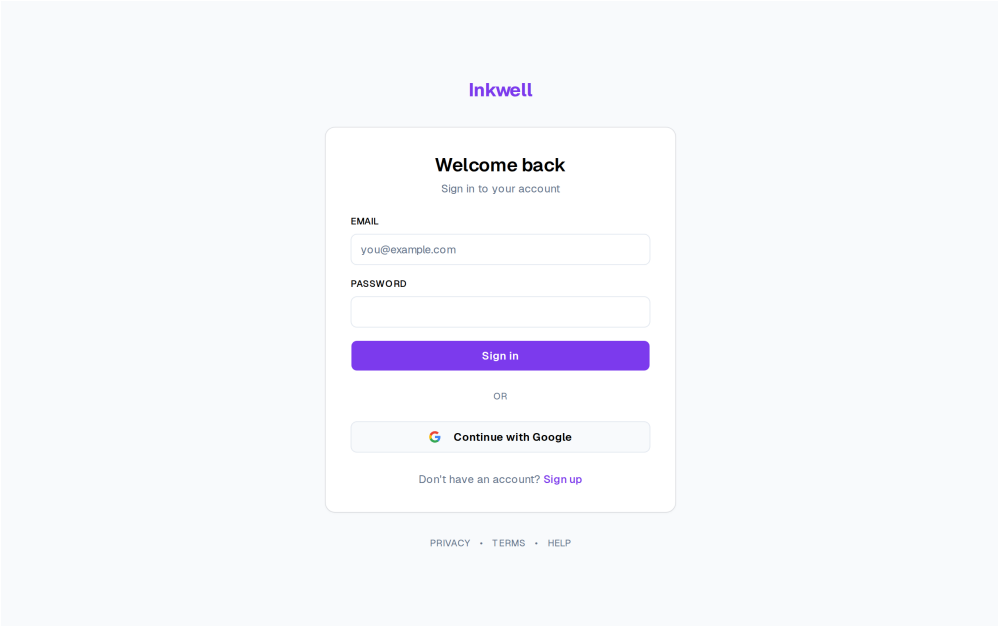


Figure 3.5 – Sign-in, with Google sign-in

It offers the two authentication methods of FR-05 side by side (e-mail and password, and *Continue with Google*) and the link to registration that Figure 1.3 draws as *Create an account* extending *Sign in*.

Registration differs by account type. Figure 3.6 shows a personal account and Figure 3.7 a magazine account.

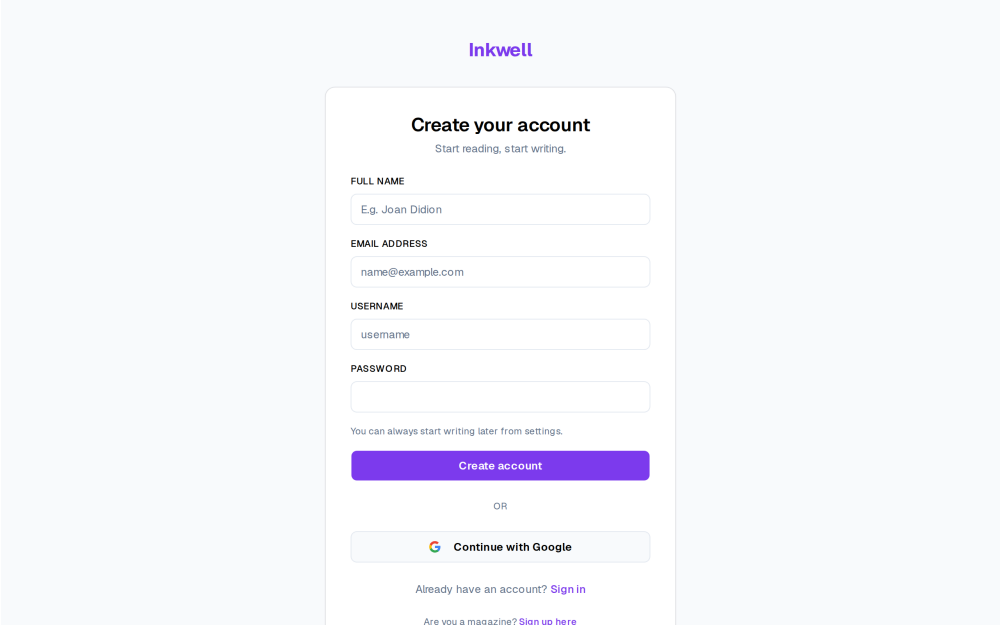


Figure 3.6 – Registering a personal account

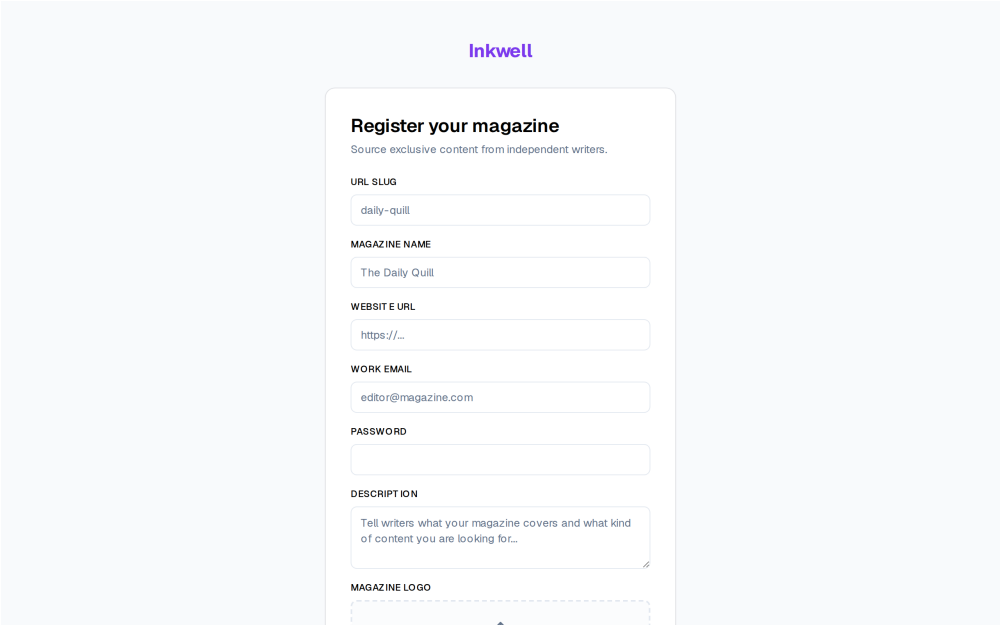


Figure 3.7 – Registering a magazine account

The magazine form asks for what writers will see of the buyer (slug, name, website, description and logo) and no personal fields. FR-04 and FR-39 give two kinds of account from the first screen, which is why a magazine never inherits a reader's capabilities.

Figure 3.8 shows a free article opened by a visitor.

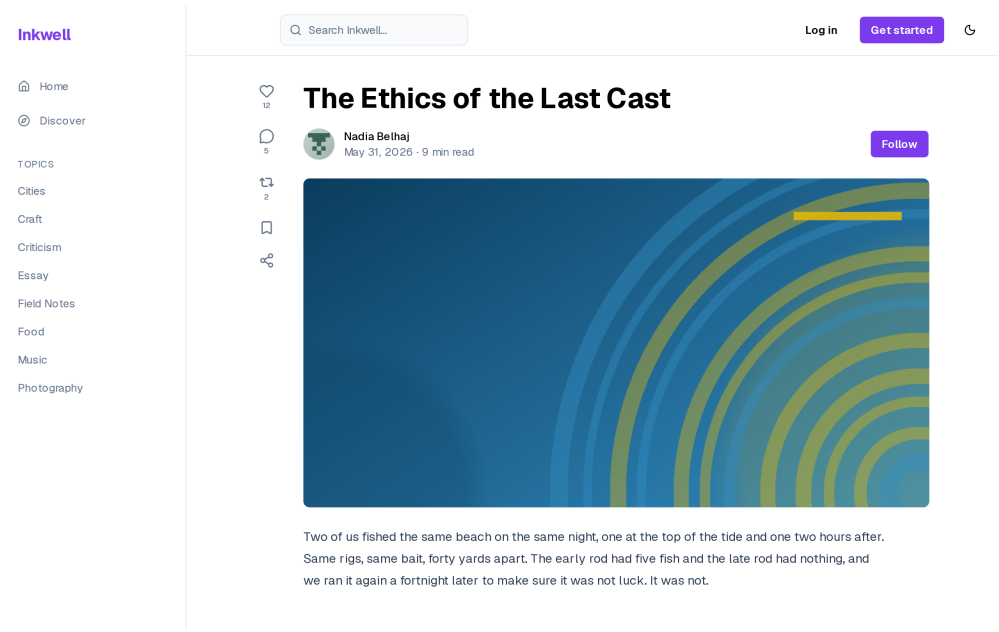


Figure 3.8 – A free article, readable without an account

The full body is readable without an account (FR-02). This was a deliberate decision: gating free articles would contradict the public feed and hide the content from search engines, so the only articles a visitor cannot read are premium and marketplace ones, for which Release 3 adds the paywall.

Figure 3.9 closes the sprint with the profile settings.

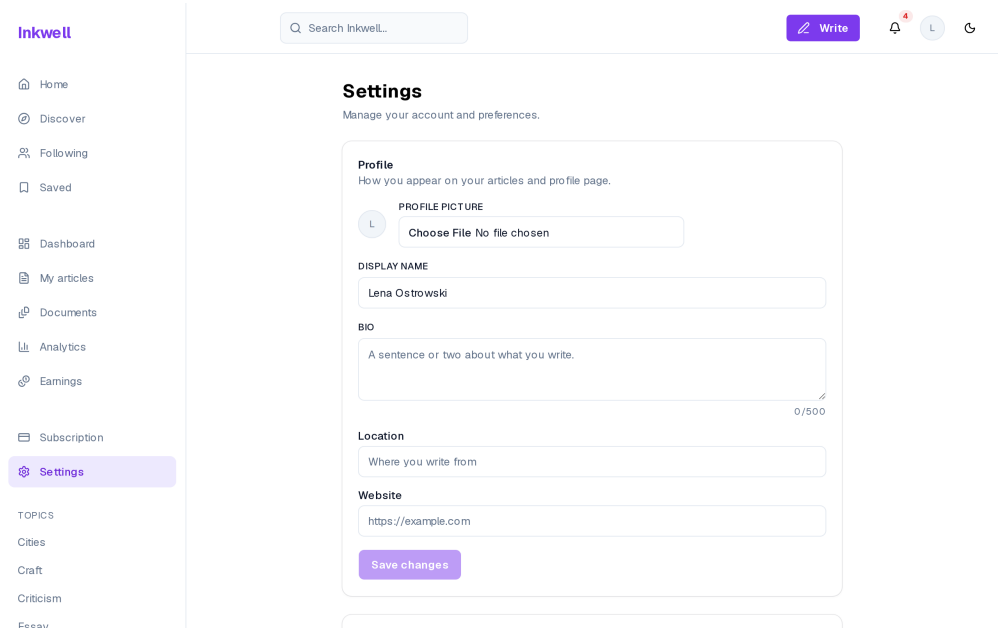


Figure 3.9 – Profile settings

A signed-in user edits the display name, bio, location, website and profile picture (FR-07); the picture is uploaded straight to object storage through a presigned URL (NFR-07). *Save changes* stays disabled until something changes, so a click never sends an empty update.

3.2 Sprint 2: Editor, AI assistance and event capture

3.2.1 Sprint backlog

The goal of Sprint 2 is that a writer drafts in a rich-text editor with automatic saving and images; that a premium writer converses with an AI assistant and applies inline AI actions to a selection, within a daily token allowance; and that every article page records reading events. Table 3.3 lists the stories.

Table 3.3 – Sprint 2 backlog

ID	User story (goal)	Priority	Points
US-06	Write in a rich editor	Must	8
US-07	Have my draft saved as I type	Must	3
US-08	Paste or drag an image into the article	Must	5
US-09	Set a cover image	Should	3
US-13	See word count and reading time	Could	1
US-14	Ask an assistant that has read my published work	Must	13
US-15	Reformulate, shorten, expand, simplify or improve a selection	Must	8
US-16	Insert an answer into my document	Must	2
US-18	See my remaining allowance	Must	2
	9 stories		**45**

The sprint commits 45 points, and its single largest story, US-14 (13 points), is split across two sprints: Sprint 2 delivers the conversation with the assistant, Sprint 4 the retrieval that grounds it in the writer’s corpus. The editor stories (US-06 to US-09) come straight from the screens of the design track, which is why they could be estimated with confidence.

3.2.2 Analysis

Figure 3.10 gives the use cases of Sprint 2.

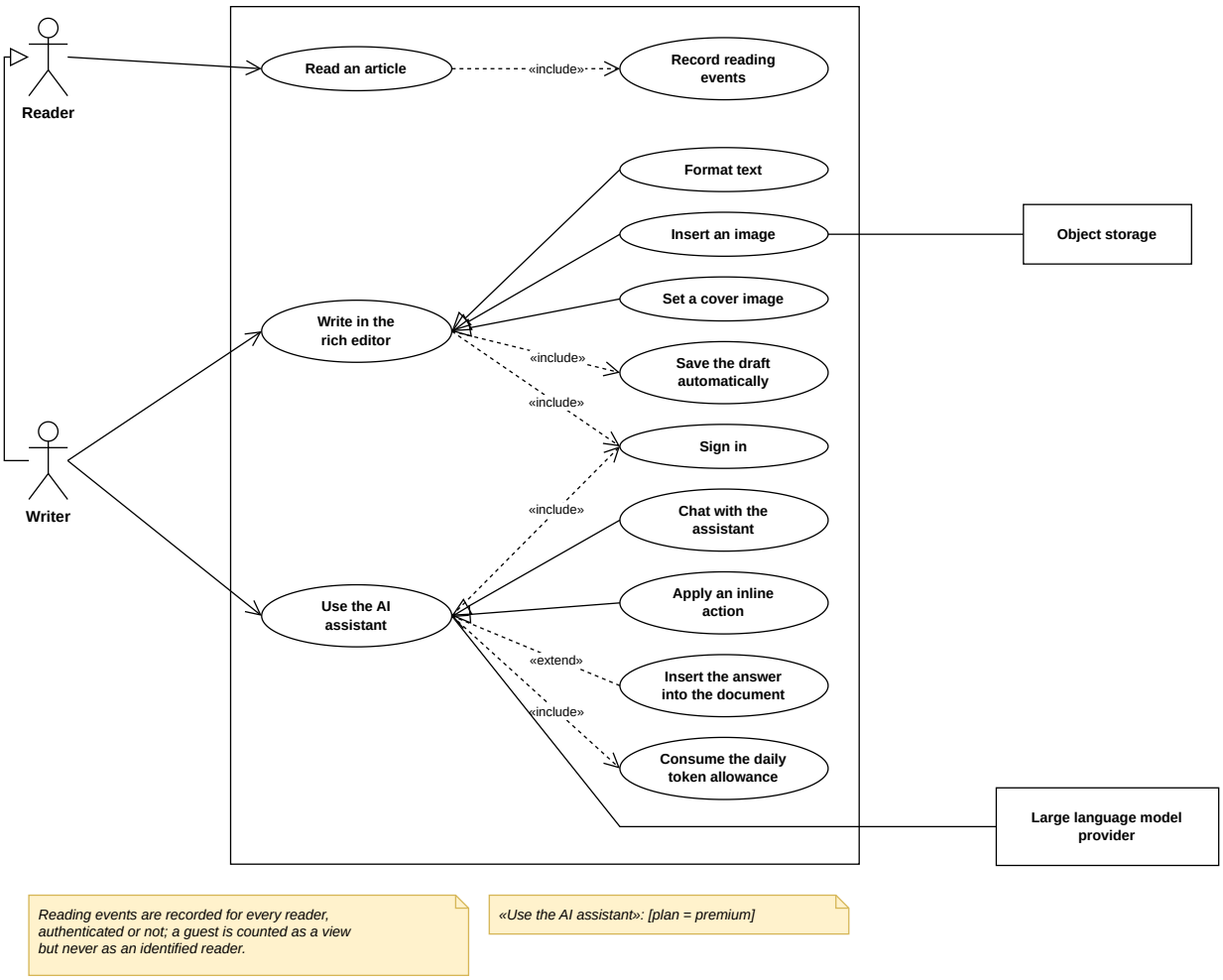


Figure 3.10 – Use case diagram, Sprint 2

Writing specialises into formatting, inserting an image and setting a cover, and includes the automatic save; using the assistant specialises into chatting and inline actions and includes the daily allowance, so no path to a model goes uncounted. Reading an article includes recording reading events. Upgrading from the interface arrives in Sprint 5; here premium accounts come from the seed.

3.2.3 Design

Class diagram. Figure 3.11 adds the classes of Sprint 2.

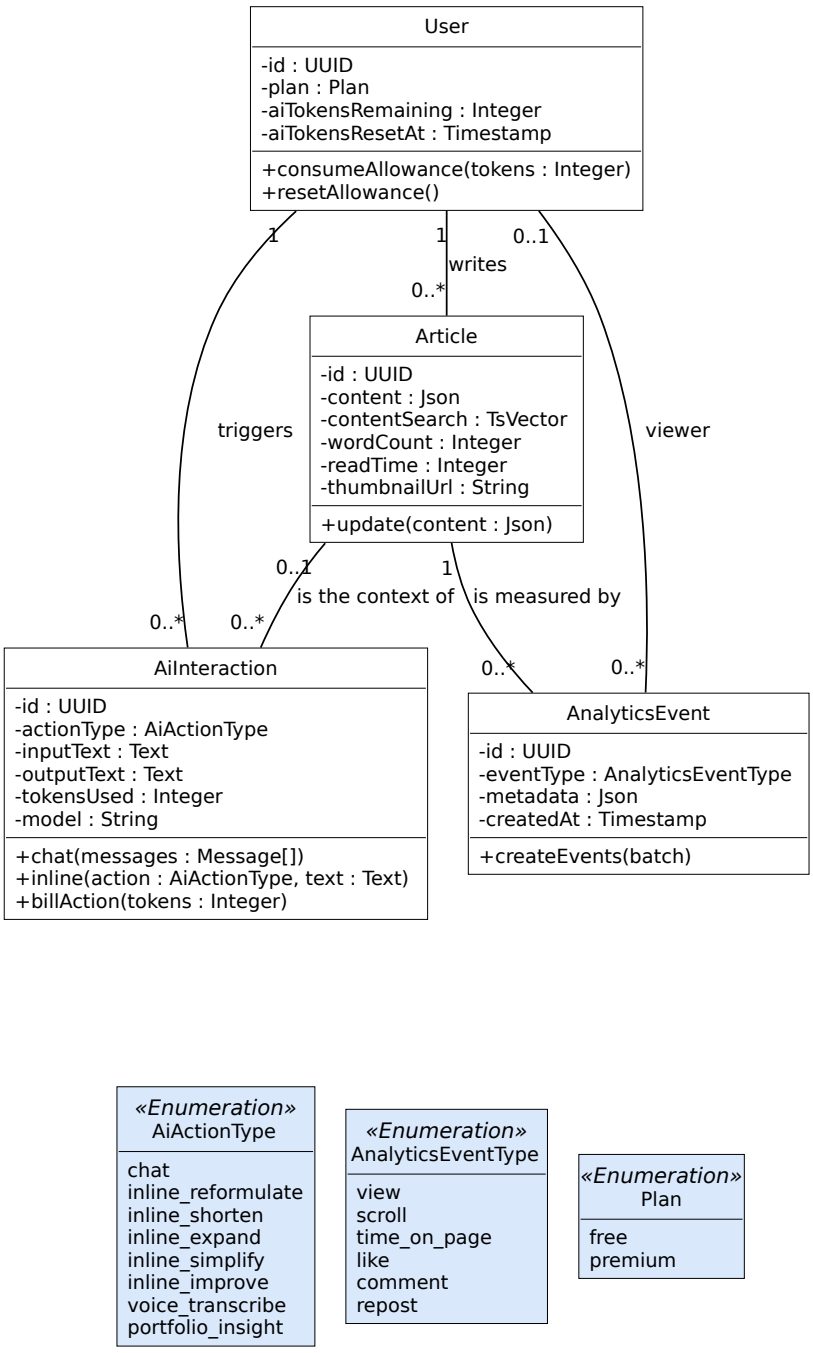


Figure 3.11 – Class diagram, Sprint 2

Each AI call is logged as an `AiInteraction` with its action, tokens and model, and reading is recorded as `AnalyticsEvent` rows; the user carries the remaining daily allowance. Cost and reading are both append-only, so the allowance can be audited and every metric of Release 2 is computed from events recorded from this sprint on.

Sequence diagram: inline AI action. Figure 3.12 shows an inline action on a selection.

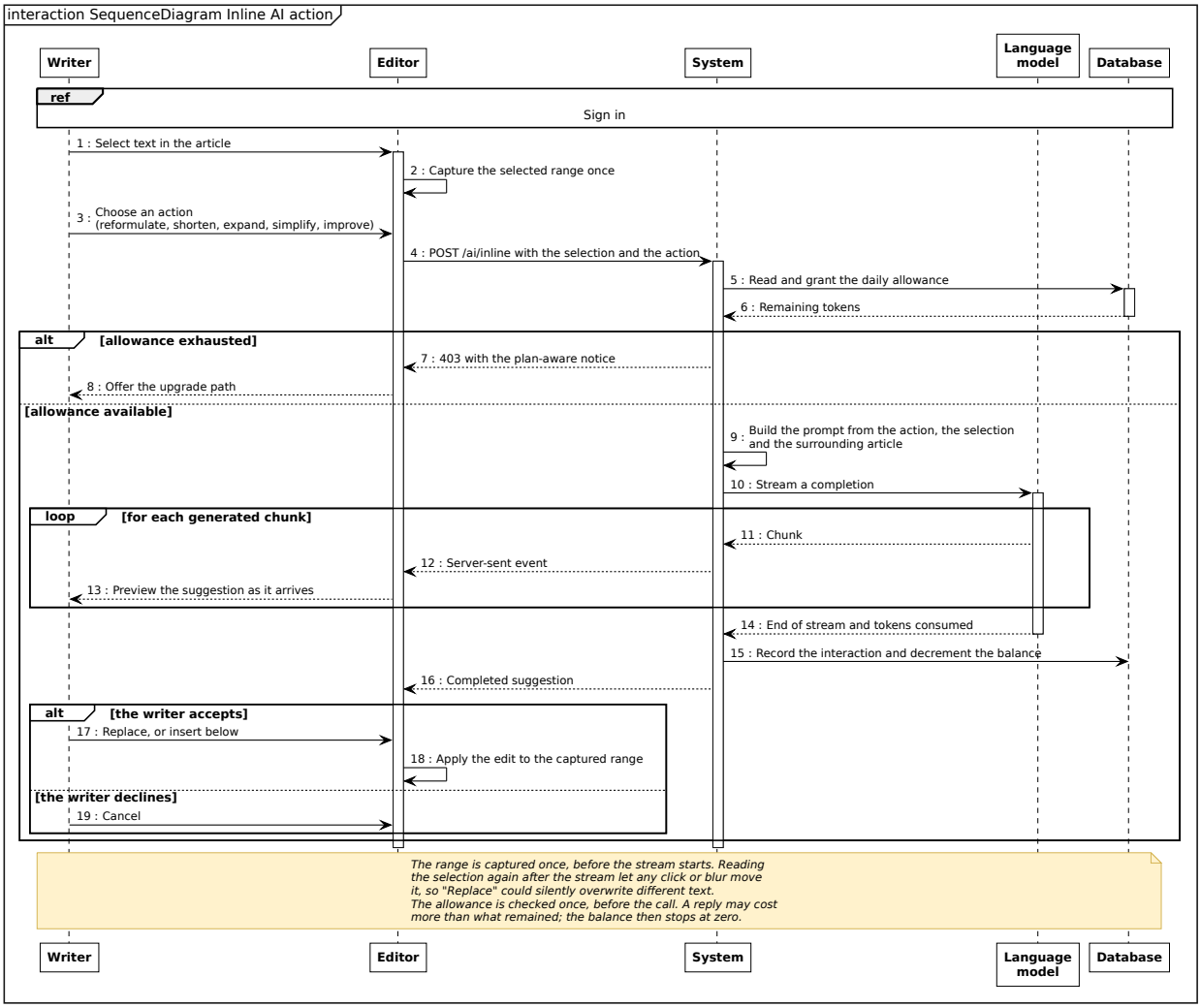


Figure 3.12 – Sequence diagram: inline AI action

The selection is fixed before streaming, so a click elsewhere cannot move it. The allowance is checked once (403 when exhausted), then the suggestion is streamed and its cost recorded. A writer is therefore never cut off mid-reply, at the price of a small overrun on the day's last call.

Event capture. The article page sends batched reading events (scroll depth and engaged time) to `POST /analytics/events`. The server, not the browser, attaches the reader's identity from the access token, so no one can forge another reader's events. Capture starts here so that real data accumulates before the dashboards of Release 2.

3.2.4 Realisation

Figure 3.13 shows the editor, where a writer spends most of their time.

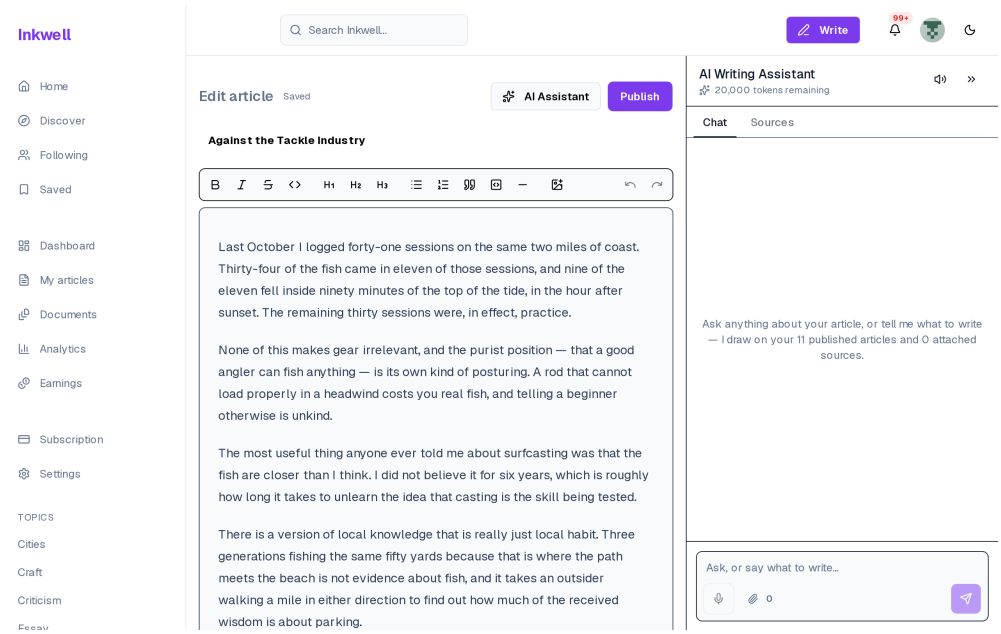


Figure 3.13 – The rich-text editor with the assistant panel

The draft is edited as rich text (emphasis, headings, lists, quotes, code, images; FR-18, FR-20) and saved automatically, as the *Saved* label shows (FR-19). The assistant sits beside the text, not on another page: help arrives where the writing happens.

Figure 3.14 shows the publish dialog designed in Sprint 1 and used here.

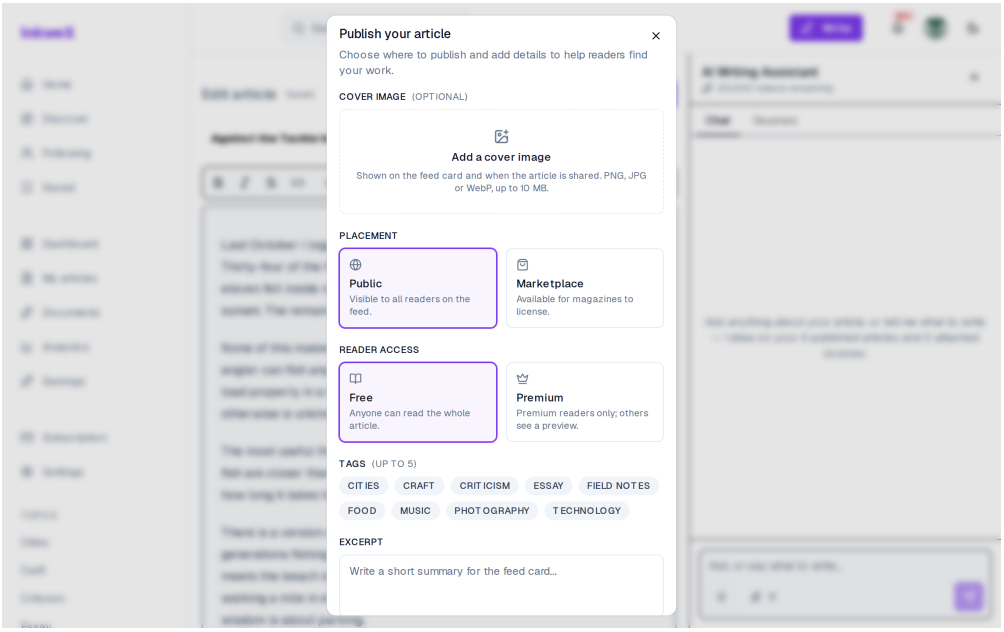


Figure 3.14 – The publish dialog: placement, visibility, tags and excerpt

Every choice of Table 3.2 is on one screen (placement, visibility, cover, tags and excerpt; FR-21, FR-23, FR-24), so publishing is a single confirmed act and not a sequence of settings pages.

The assistant is shown in two states. Figure 3.15 shows the panel before a question and Figure 3.16 an answer.

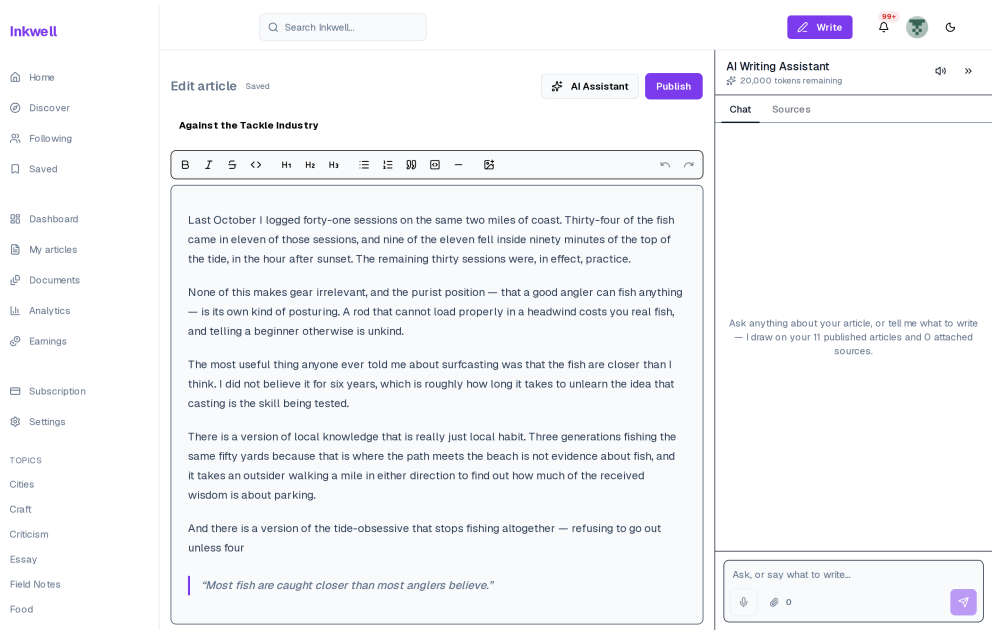


Figure 3.15 – The AI assistant panel

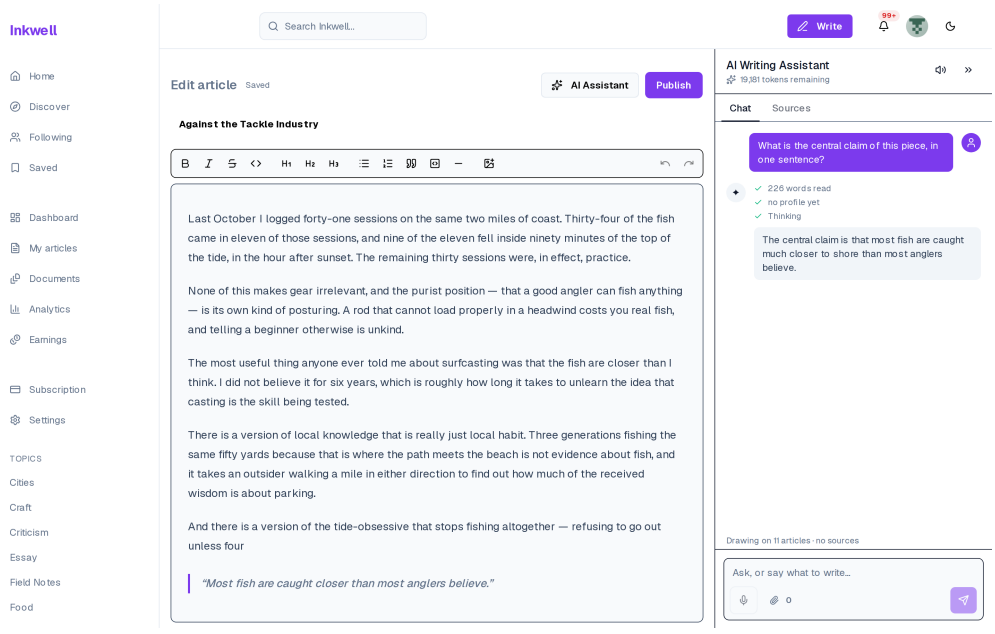


Figure 3.16 – An answer from the assistant

The panel shows the remaining allowance (20,000 tokens at the start of the day) before any question (FR-31), so a question's cost is never a surprise. The answer is streamed and can be inserted at the cursor (FR-27, FR-30), a manual step Sprint 8 replaces.

The inline actions are the second way to use the assistant. Figure 3.17 shows the menu on a selection and Figure 3.18 its result.

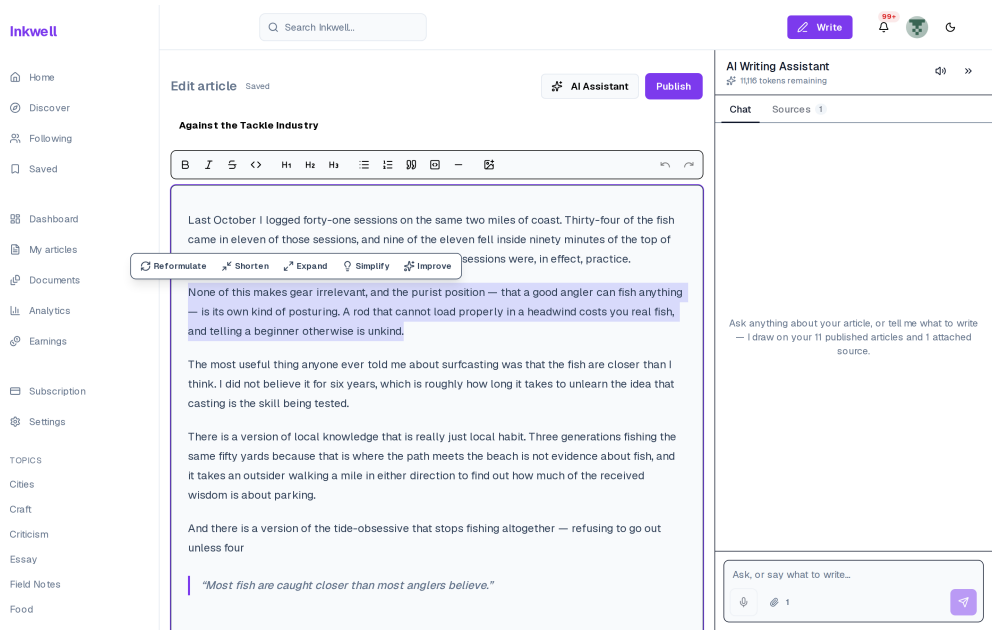


Figure 3.17 – The inline actions on a selection

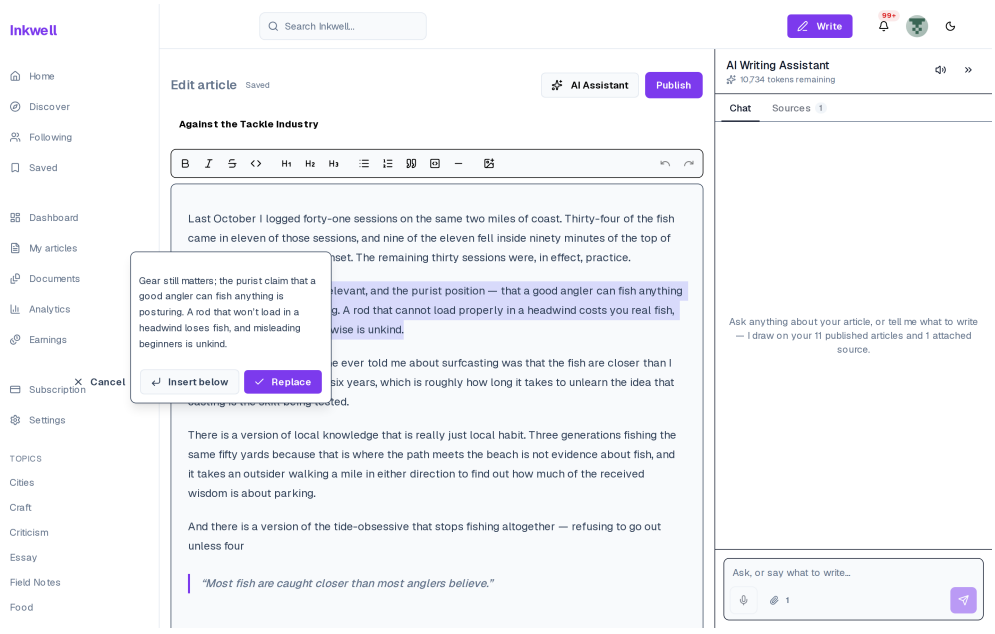


Figure 3.18 – An inline suggestion, ready to replace the selection

Selecting text opens the five actions of FR-28: reformulate, shorten, expand, simplify, improve. The suggestion appears in a popover over the highlighted original rather than replacing it, with three choices: *Replace*, *Insert below* or *Cancel*. The allowance in the panel has dropped accordingly. The assistant proposes and the writer decides, which is the rule the whole assistant follows.

Conclusion

Release 1 delivered the foundation: accounts of two types with rate-limited authentication and a silent refresh, article management with placement and visibility, a rich-text editor, the first AI assistant with inline actions under a daily allowance, the capture of reading events, and the ledger harness that every later money-moving

feature is tested against. Release 2 builds the social layer and the analytics on top of it.

CHAPTER 4

Release 2: social interactions, analytics and AI

Introduction

The second release turns a publishing tool into a platform. Sprint 3 adds the social layer (likes, threaded comments, reposts, follows, reports) with live notifications, and aggregates the reading events captured since Release 1 into the metrics behind the writer's dashboards and the magazine's evaluation report. Sprint 4 adds the platform's retrieval layer: published articles are embedded, the assistant answers from the writer's own passages and voice profile, magazines can request AI portfolio insights, and search combines keywords with meaning.

4.1 Sprint 3: Social interactions, notifications and analytics

4.1.1 Sprint backlog

The goal of Sprint 3 is that readers interact with articles and writers and the author is told live; that writers see how their articles are read; and that magazines browse writers and read an evaluation report built on audience, content and quality metrics. Table 4.1 lists the stories.

Table 4.1 – Sprint 3 backlog

ID	User story (goal)	Priority	Points
US-24	Like an article	Must	2
US-25	Comment and reply in a thread	Must	5
US-26	Follow a writer	Should	3
US-27	Repost an article	Could	2
US-28	Be told the moment someone reacts to my work, without reloading	Must	5
US-29	Review notifications I missed	Must	3
US-30	See how many people read an article and how far they got	Must	8
US-31	Rank my articles by performance	Should	3
US-32	Evaluate a writer on audience, content and quality	Must	13
	9 stories		**44**

The sprint commits 44 points. Its weight is concentrated in two analytics stories, the writer's per-article analytics (US-30, 8 points) and the evaluation report (US-32, 13 points): the social stories are small and similar to one another, while the evaluation needs the aggregation pipeline that every metric depends on.

4.1.2 Analysis

Figure 4.1 gives the use cases of Sprint 3.

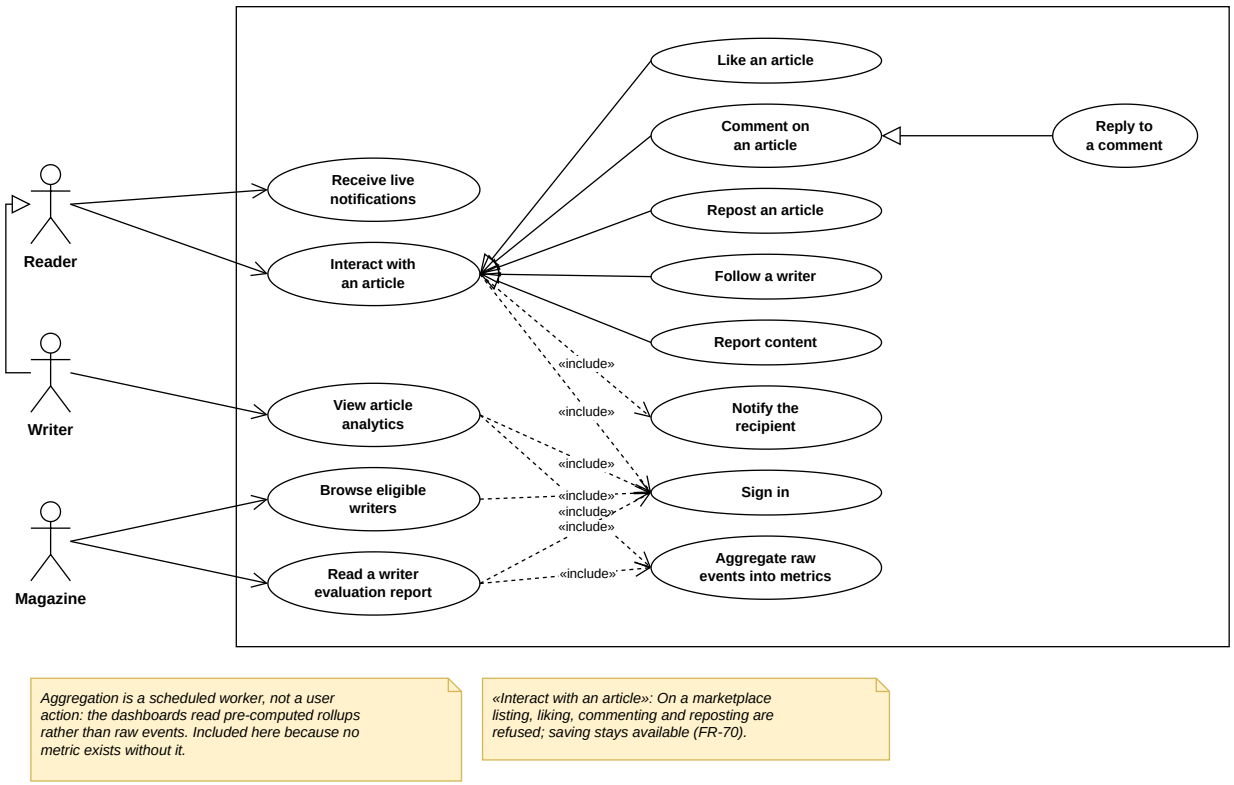


Figure 4.1 – Use case diagram, Sprint 3

Interacting specialises into liking, commenting, replying, reposting, following and reporting, and includes notifying the recipient; on a marketplace listing only saving is allowed (FR-70). Article analytics and the writer evaluation both include the aggregation of events into metrics. The diagram joins the sprint's two halves: interactions produce events, and the aggregation turns them into numbers.

4.1.3 Design

Class diagram. Figure 4.2 adds the classes of Sprint 3.

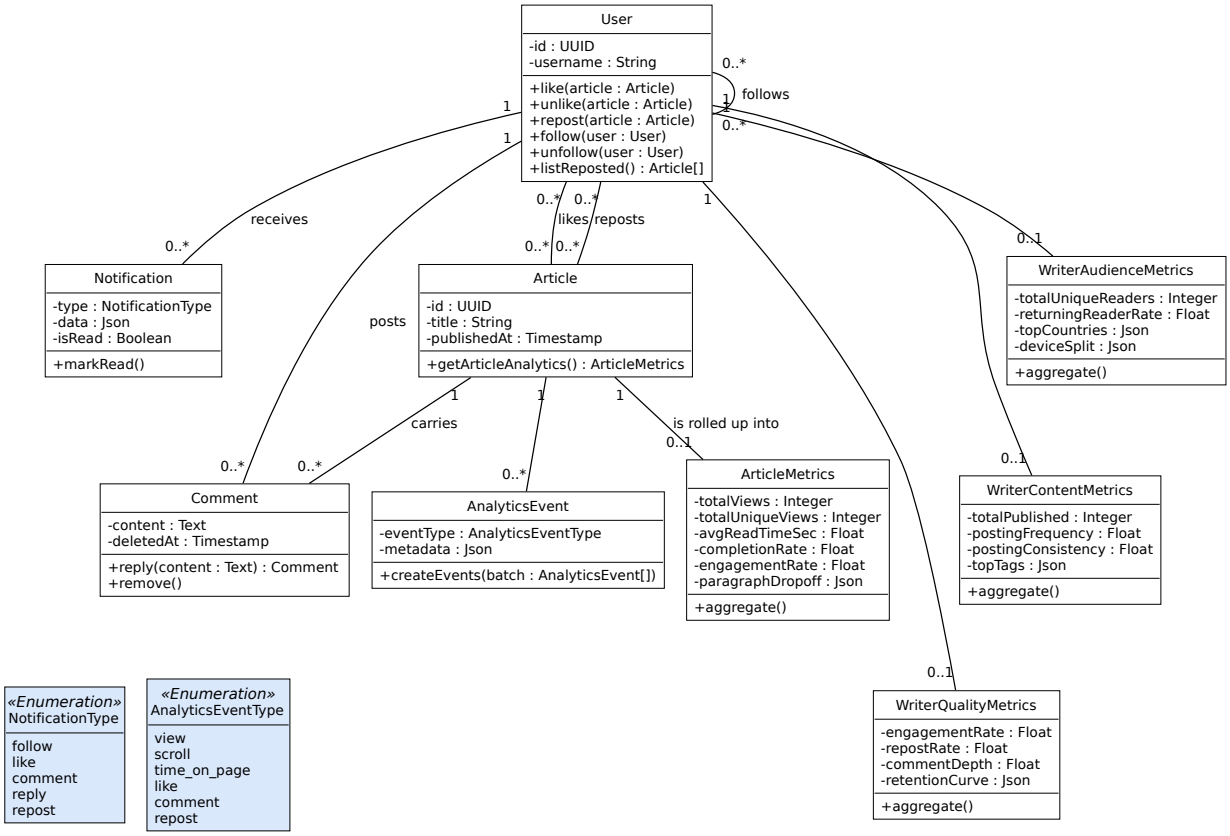


Figure 4.2 – Class diagram, Sprint 3

Likes, reposts and follows are many-to-many associations; comments and notifications are classes, because they carry content and state. The four rollups (`ArticleMetrics` and the three writer metrics) hold one row per article or writer, rewritten by the aggregation, so reading a dashboard costs the same whatever the number of events behind it.

Sequence diagram: from reading events to metrics. Figure 4.3 follows a reading event to the dashboard.

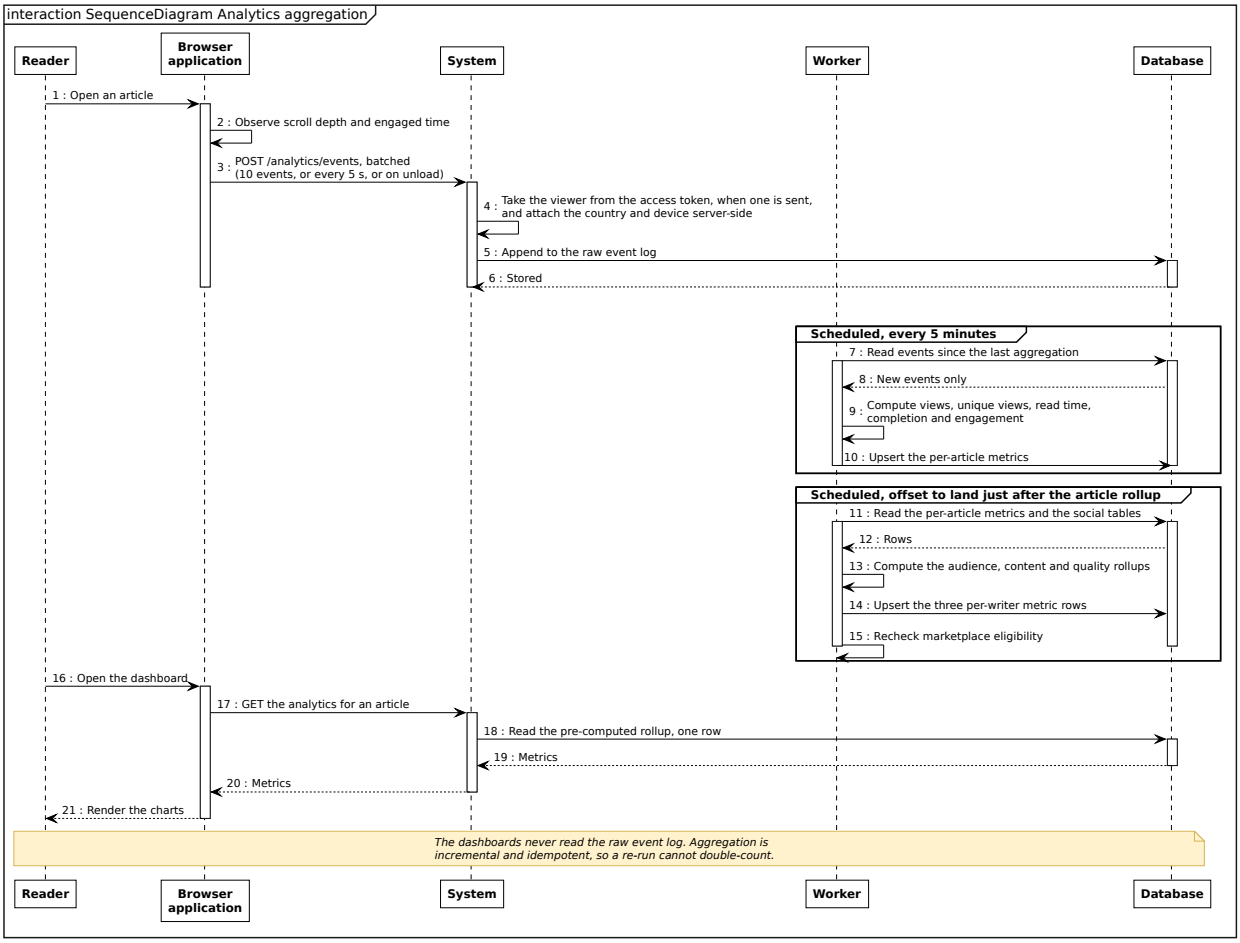


Figure 4.3 – Sequence diagram: analytics aggregation

The API takes the reader's identity from the access token, never from the request. Every five minutes the worker processes only the new events into the per-article and per-writer rollups and rechecks eligibility. Dashboards read only the rollups and stay fast (NFR-15, NFR-16), at the price of a five-minute delay.

The metrics the aggregation produces are those of the evaluation report, defined in Table 4.2. Each definition is the one the aggregation computes.

Table 4.2 – Main metrics of the evaluation report

Panel	Metric	Definition	Why this definition
Audience	Unique readers	distinct identified readers over the last 30 days	reach now, not since the writer joined
Audience	Returning-reader rate	share of those readers seen on more than one day	a reader who refreshes twice is not loyal; one who comes back another day is
Content	Posting frequency	articles per week	regularity is what a magazine commissioning a series needs
Content	Consistency	1 − coefficient of variation of the gaps between articles, clamped to [0, 1]	1 is perfectly regular; near 0 is bursts and silences
Content	Topics	distribution of the writer's articles over tags	the range a magazine can draw on

Panel	Metric	Definition	Why this definition
Quality	Engagement rate	(likes + comments) per view	interaction per reader; it exceeds 100% when articles collect more reactions than views
Quality	Completion rate	share of views that scroll to 90% of the article	90% rather than 100%, because footers and comments sit below the text
Quality	Repost rate, comment depth	reposts per view; replies per top-level comment	whether readers pass the work on and discuss it
Quality	Retention curve	share of views reaching 25%, 50%, 75% and 90% of the page	where readers stop, at the granularity page scroll supports

The table shows that every metric is a ratio or a distribution rather than a raw count, so that a writer with a small but attentive audience can compare with a writer with a large and indifferent one. It also shows the limit of the capture: the retention curve has four points because the browser records the deepest scroll, not the paragraph being read.

Sequence diagram: live notifications. Figure 4.4 shows how a notification reaches its recipient.

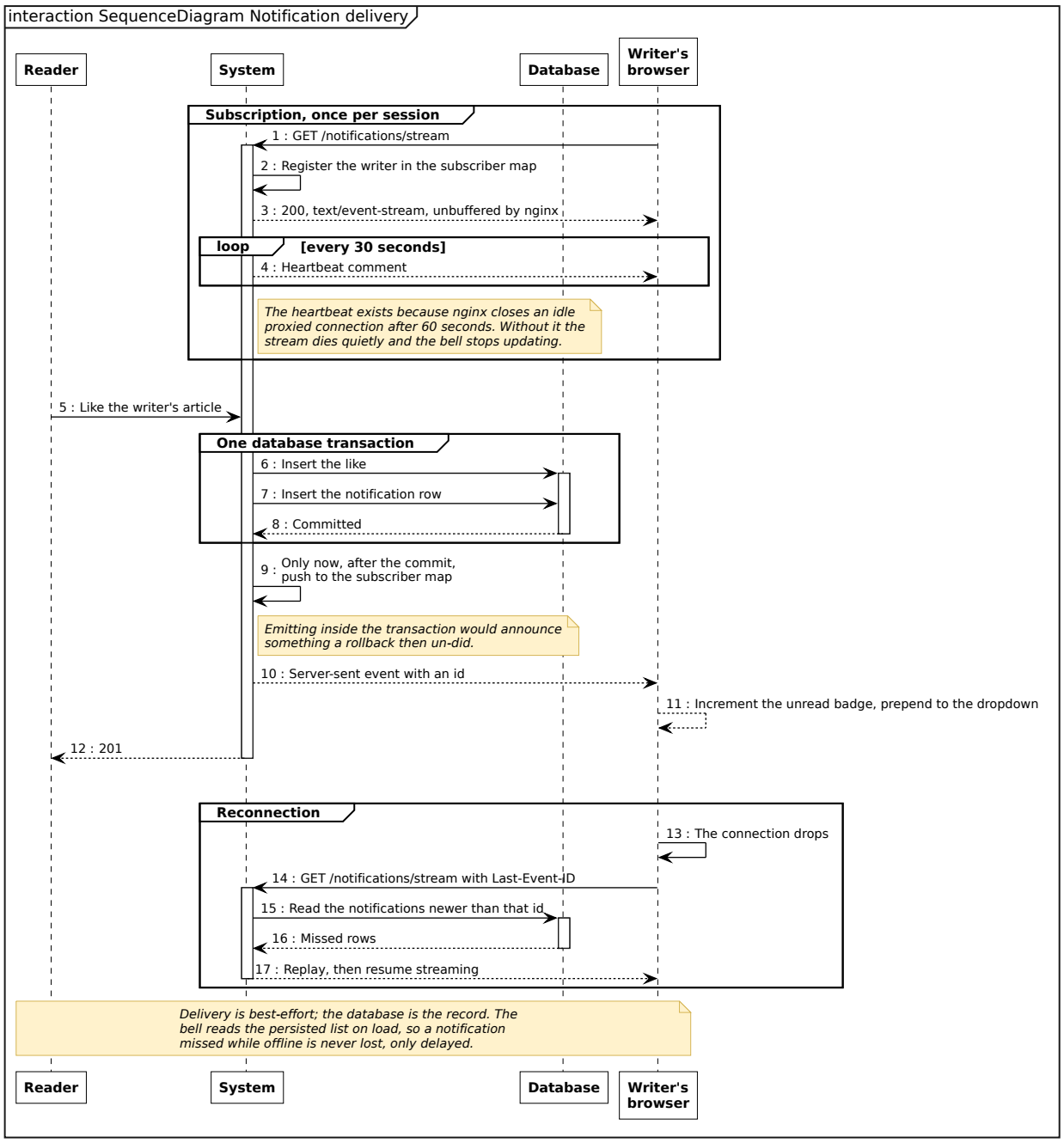


Figure 4.4 – Sequence diagram: live notification delivery

The like and its notification are committed first, then pushed to the recipient's open stream; after a dropped connection the server replays what was missed. A notification is never lost and never announces something that did not happen: the database is the record, the push a faster way to read it.

4.1.4 Realisation

Figure 4.5 shows the feed of a signed-in reader.

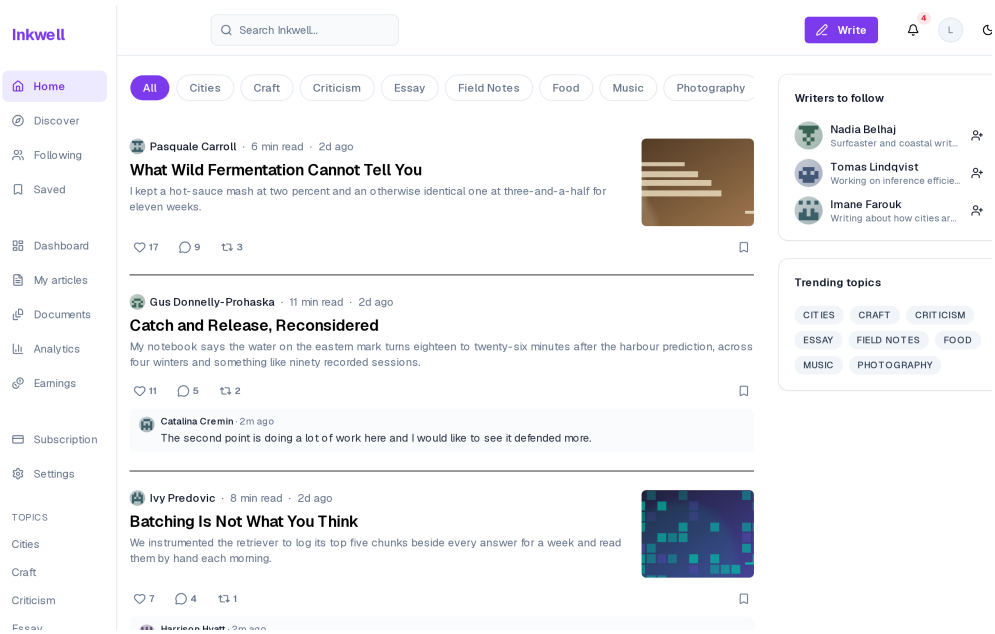


Figure 4.5 – The feed of a signed-in reader, with live card controls

Compared with the visitor's feed of Figure 3.4, the header now offers *Write*, the notification bell and the account; each card carries like, comment, repost and save controls with their counts, and the side panel suggests writers to follow (FR-08, FR-09, FR-11, FR-12). The latest comment under a card is the live preview added in Sprint 8.

Figure 4.6 shows the notifications of a writer.

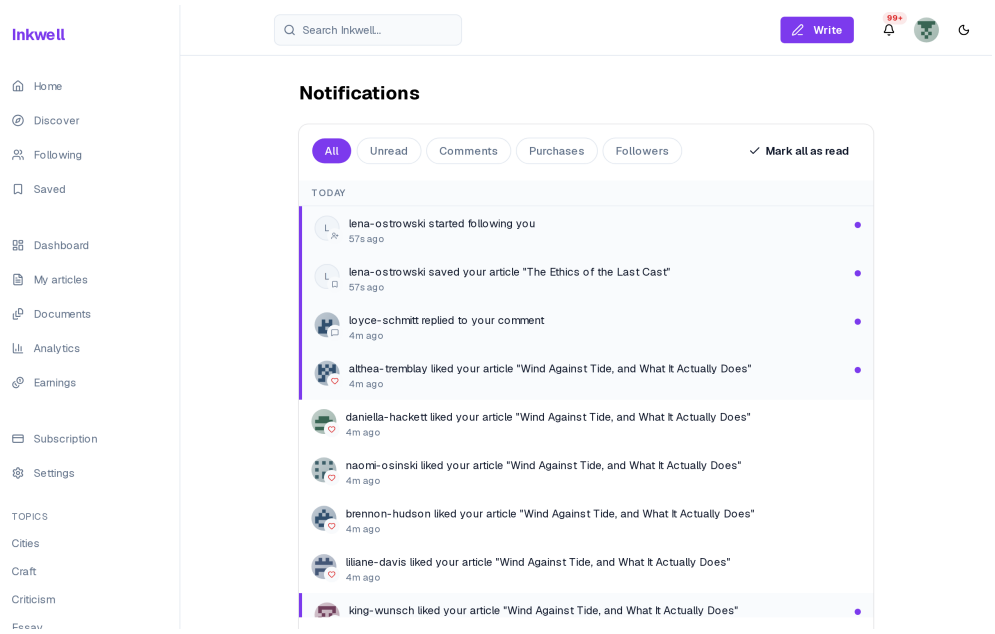


Figure 4.6 – Notifications, filtered and grouped by day

Notifications are grouped by day, filtered by kind (unread, comments, purchases, followers), and unread ones are marked by a bar and a dot until *Mark all as read* (FR-13, FR-14). The list mixes social events with marketplace ones, so a writer follows both the conversation around their work and its sales in one place.

The writer's view of their own work is shown in three screens. Figure 4.7 is the dashboard, Figure 4.8 the list of articles and Figure 4.9 the analytics.

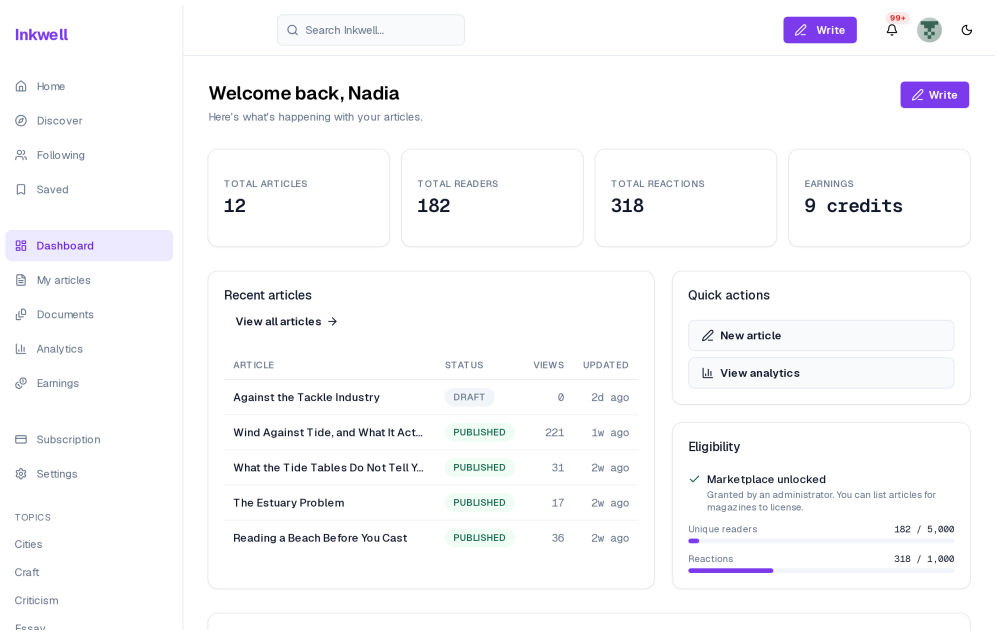


Figure 4.7 – The writer's dashboard

The dashboard summarises articles, readers, reactions and earnings, lists the recent articles with their status and views, and shows the eligibility panel: 182 of 5,000 readers and 318 of 1,000 reactions, with the marketplace already unlocked by an administrator's grant (FR-34, FR-55). It answers the writer's first question (how am I doing?) on one screen.

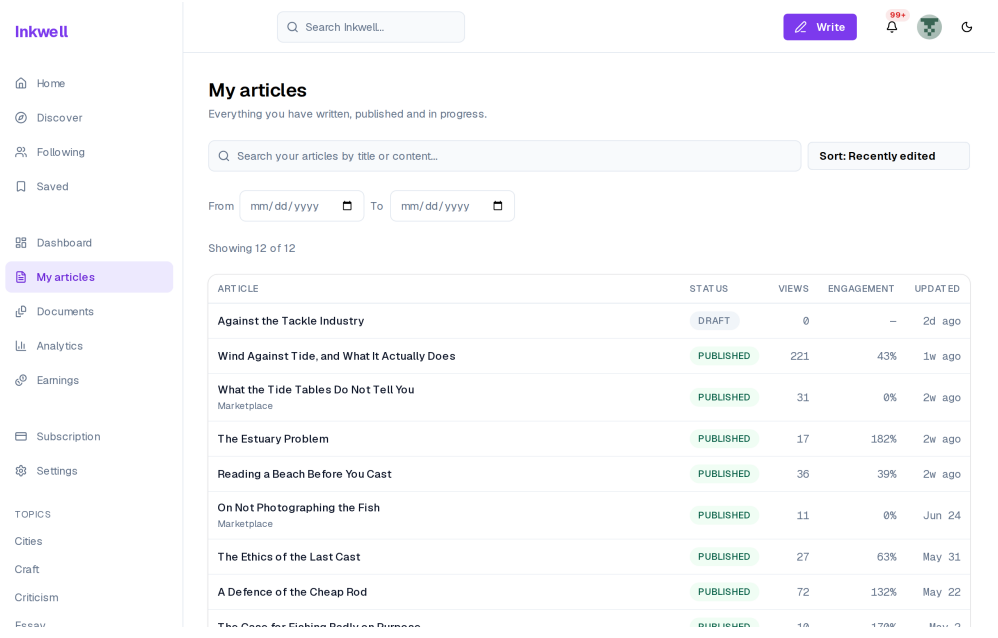


Figure 4.8 – "My articles": every article, sortable by performance

"My articles" lists every article, drafts and listings included, searchable and sorted on the server across all of them (FR-33). Engagement is the per-view rate of Table 4.2, so it exceeds 100% where seeded reactions outnumber recorded views.

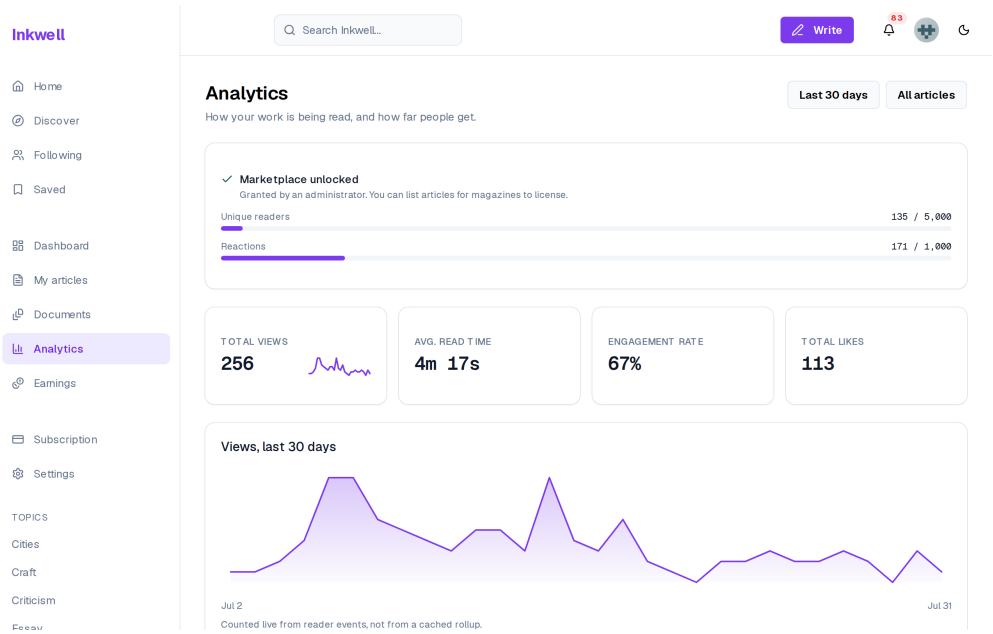


Figure 4.9 – The writer's analytics: views over the last 30 days

The analytics page gives views, read time, engagement and likes, and the daily views of the last 30 days (FR-32). As the note says, this series is counted from the events: the one panel NFR-15 allows to read them directly, bounded to thirty days.

Figure 4.10 shows a writer's public profile.

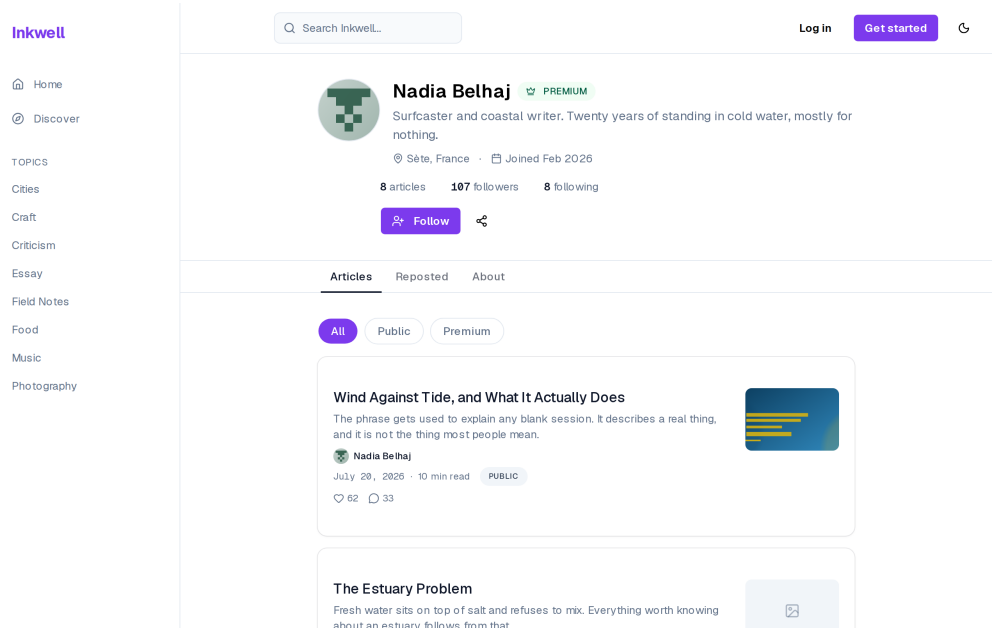


Figure 4.10 – A writer's public profile

The profile shows the writer's bio, location, join date, article and follower counts and published articles to any visitor, filterable by public and premium, with a *Follow* control and a share button (FR-12). It is the public face of the portfolio that a magazine evaluates in more depth.

Figure 4.11 shows the magazine's side of the same data.

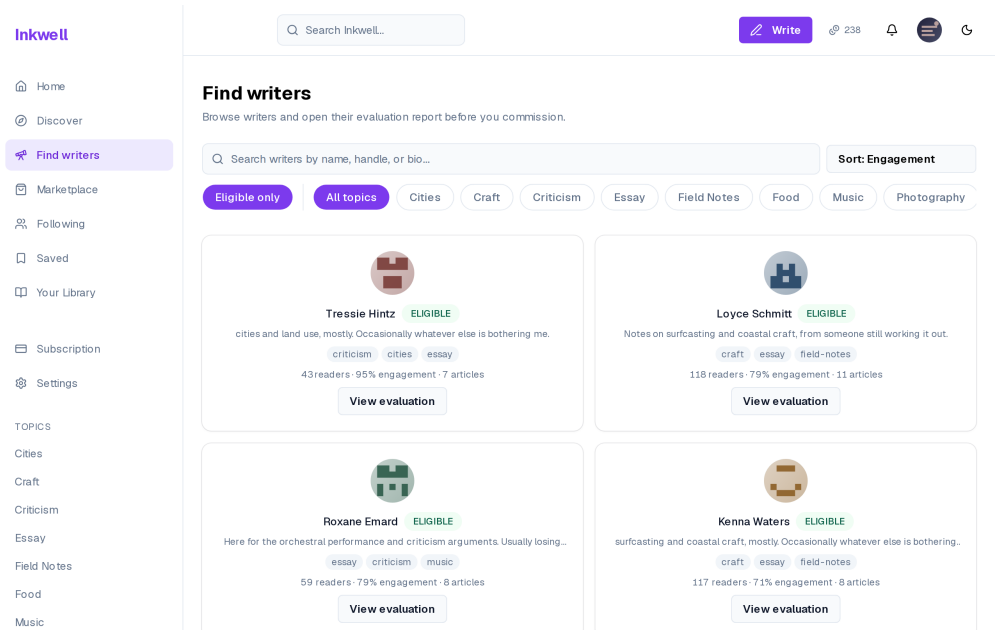


Figure 4.11 – A magazine browsing eligible writers

A magazine filters writers by eligibility and topic, searches and sorts them (FR-44); each card gives readers, engagement and articles, and *View evaluation* opens the report. Any magazine reaches this page in Sprint 3; Sprint 5 puts it behind the subscription wall, hence two sprints for FR-44.

4.2 Sprint 4: Retrieval, portfolio insights and hybrid search

4.2.1 Sprint backlog

The goal of Sprint 4 is that published articles become a retrievable corpus; that the assistant answers from the writer's own passages and shows them; that each writer gets an automatically extracted voice profile; that magazines request AI portfolio insights; and that search finds articles by meaning as well as by keyword. Table 4.3 lists the stories.

Table 4.3 – Sprint 4 backlog

ID	User story (goal)	Priority	Points
US-14	Ask an assistant that has read my published work	Must	13
US-17	See which of my passages the assistant used	Should	5
US-19	Have my voice profile extracted from my corpus	Should	8
US-20	Search by keyword	Must	5
US-21	Find articles that match my meaning even when they share none of my words	Must	8
US-22	Browse by tag	Should	2
US-23	Find writers, not only articles	Should	3
US-33	Read an AI assessment of a writer's voice and range	Must	8
	8 stories		**52**

The sprint commits 52 points, the second heaviest load of the project. Its stories share one new component, the embedding of published articles, and could only be estimated once it existed, which is why the sprint begins with it.

4.2.2 Analysis

Figure 4.12 gives the use cases of Sprint 4.

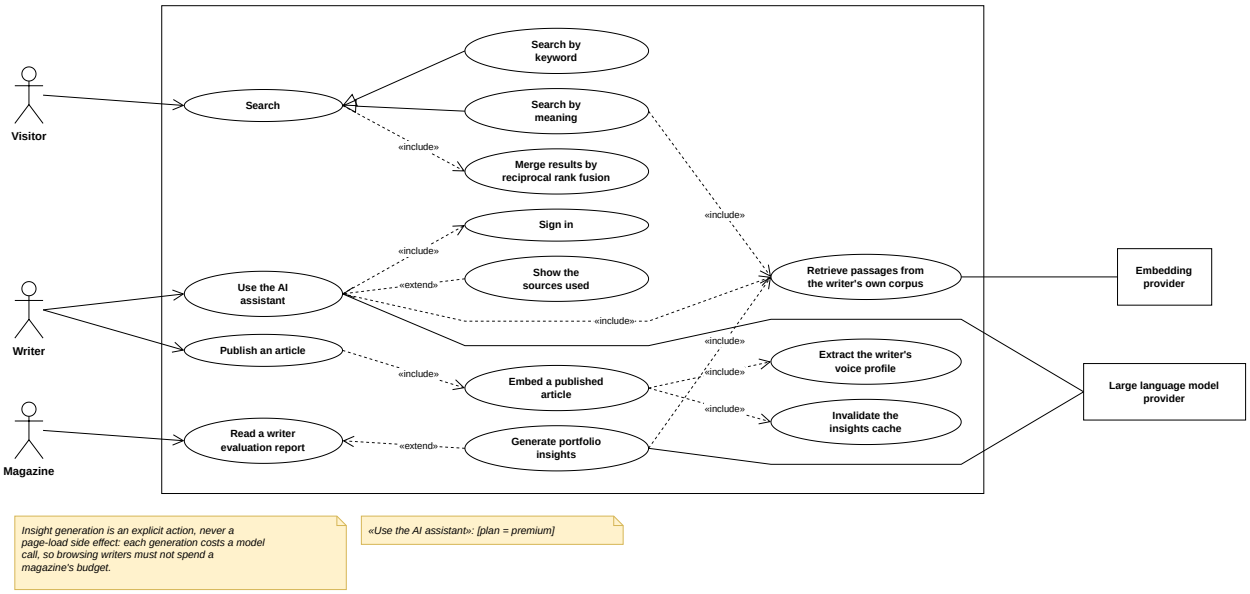


Figure 4.12 – Use case diagram, Sprint 4

Searching specialises into keyword and meaning and includes merging the two results; the assistant includes retrieving the writer's passages and showing them; publishing includes embedding the article, extracting the voice profile and clearing the insights cache; the evaluation is extended by portfolio insights. Publishing has become the trigger of the whole AI layer.

4.2.3 Design

Class diagram. Figure 4.13 adds the retrieval corpus.

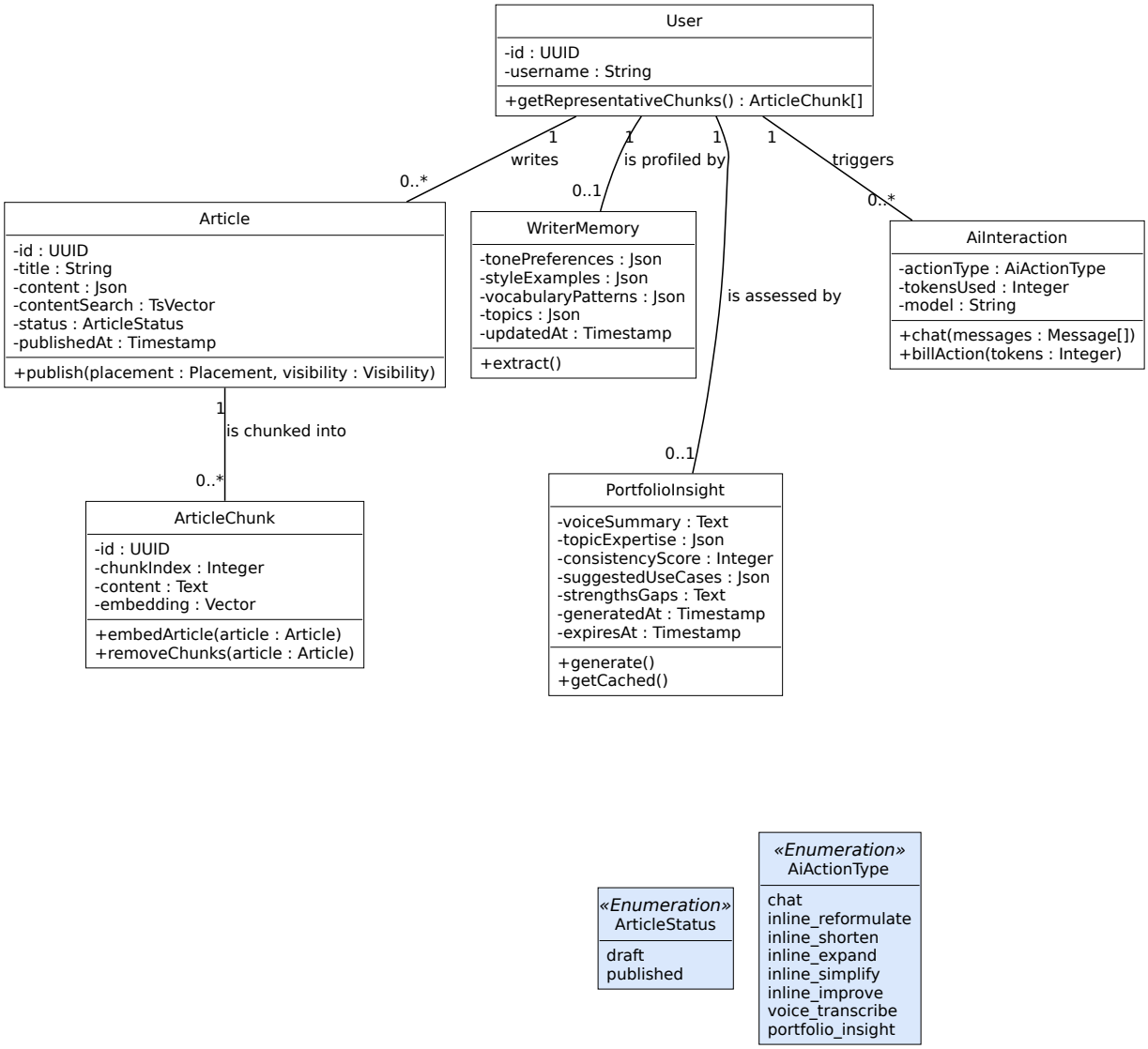


Figure 4.13 – Class diagram, Sprint 4

An `ArticleChunk` is a passage of a published article with its 1,536-dimension embedding; the `WriterMemory` holds the voice profile; the `PortfolioInsight` caches the AI assessment; `AiInteraction` logs every call. The AI layer only adds data derived from articles: every chunk, profile and insight can be rebuilt from them, which the re-embedding on each publication does.

Sequence diagram: what publishing sets in motion. Figure 4.14 shows the background pipeline a publication starts.

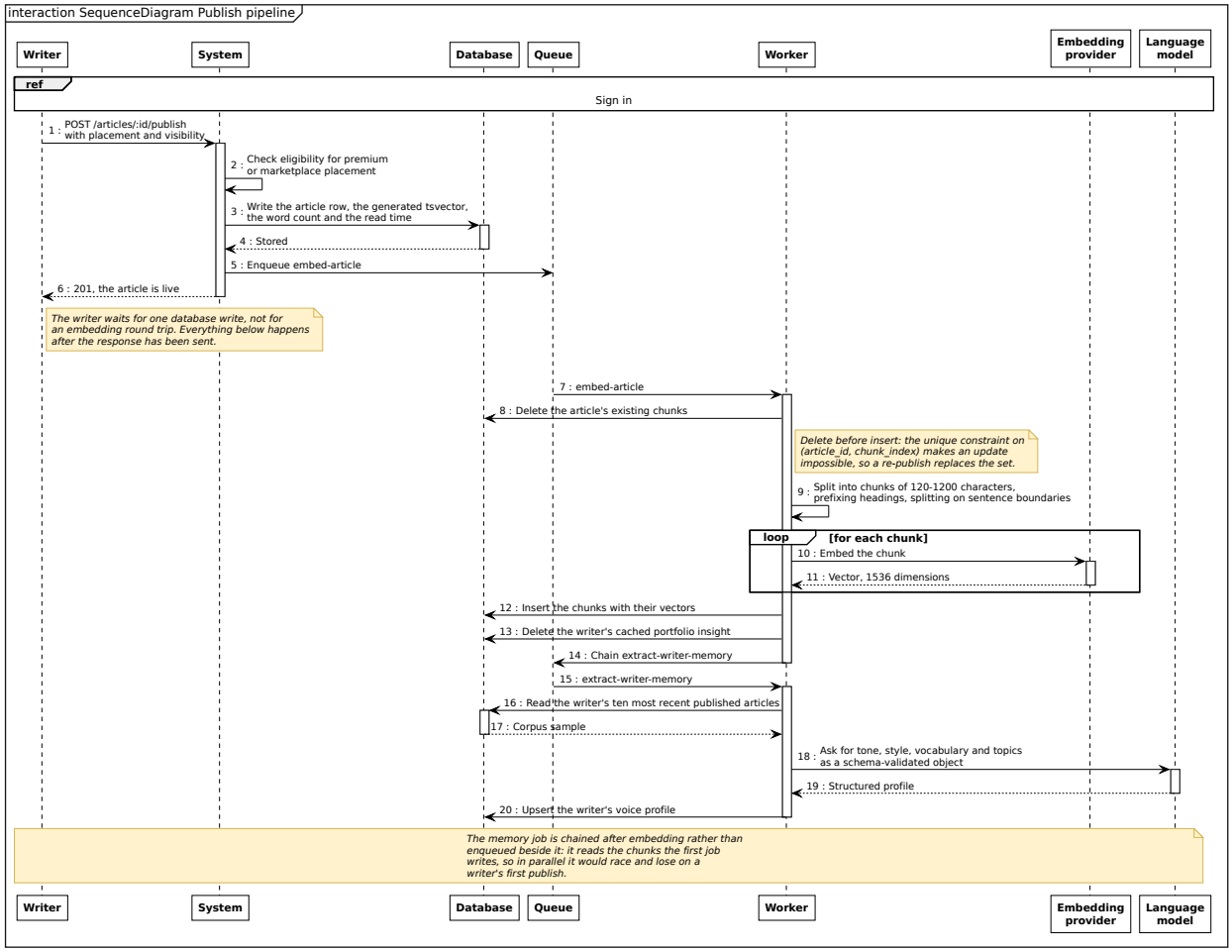


Figure 4.14 – Sequence diagram: the publishing pipeline

The API saves the article and queues an embedding job. The worker replaces the article's chunks (passages of 120 to 1,200 characters, each prefixed with its nearest heading), embeds them, clears the cached insights, then extracts the voice profile. Publishing returns immediately: the writer never waits for an embedding, and a provider failure delays retrieval without blocking publication.

Sequence diagram: a chat turn grounded in the writer's corpus. Figure 4.15 shows retrieval-augmented generation, the technique of conditioning a language model on passages retrieved for the question [40], [41], as delivered in this sprint.

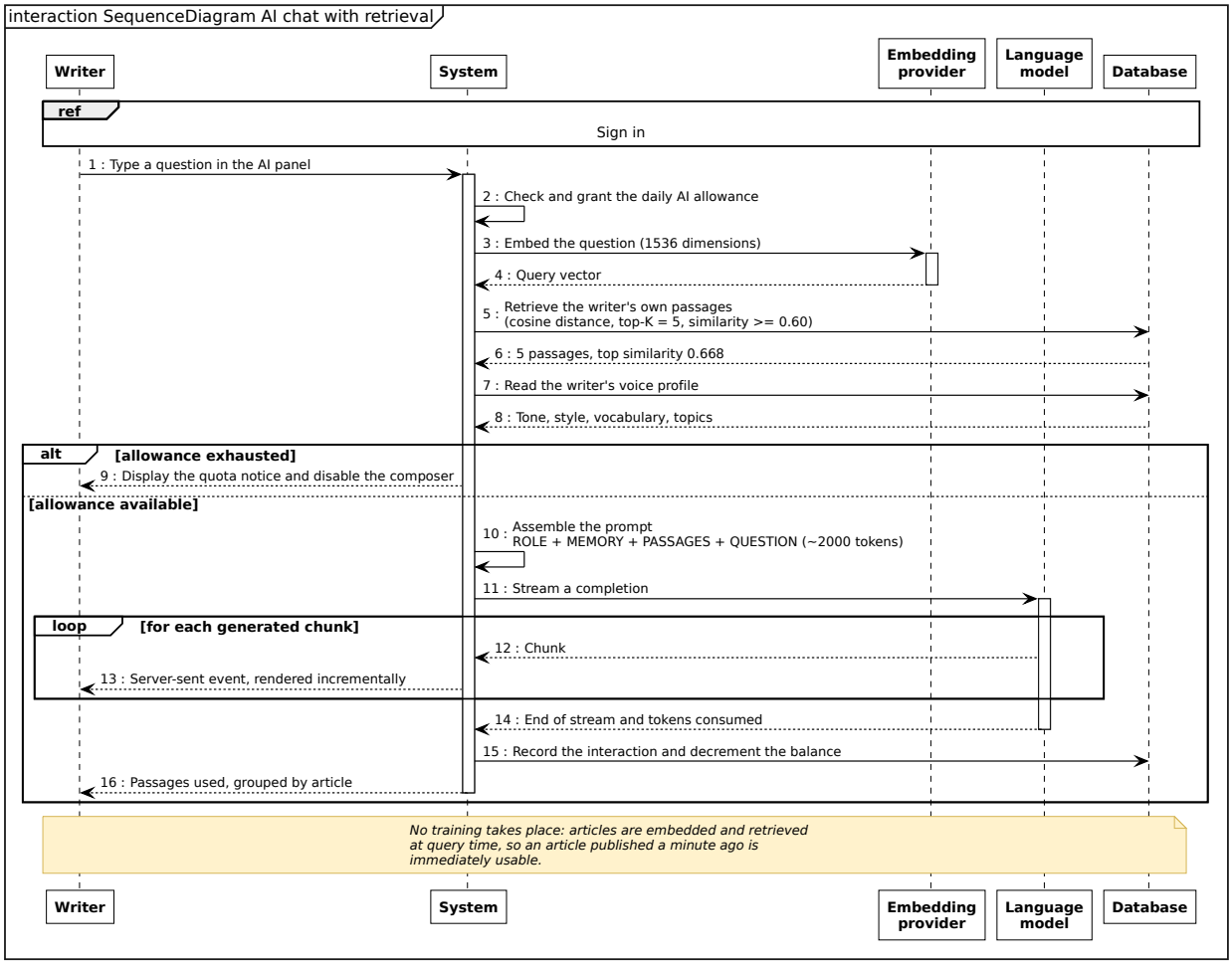


Figure 4.15 – Sequence diagram: AI chat with retrieval

The question is embedded and the five closest passages of the writer's own work are retrieved if they score above 0.60; the prompt adds the voice profile, and the answer is streamed with its sources. The floor lets retrieval return *nothing* rather than five irrelevant passages, and the profile keeps the style even then. Section 7.2 measures both.

Sequence diagram: hybrid search. Figure 4.16 shows search.

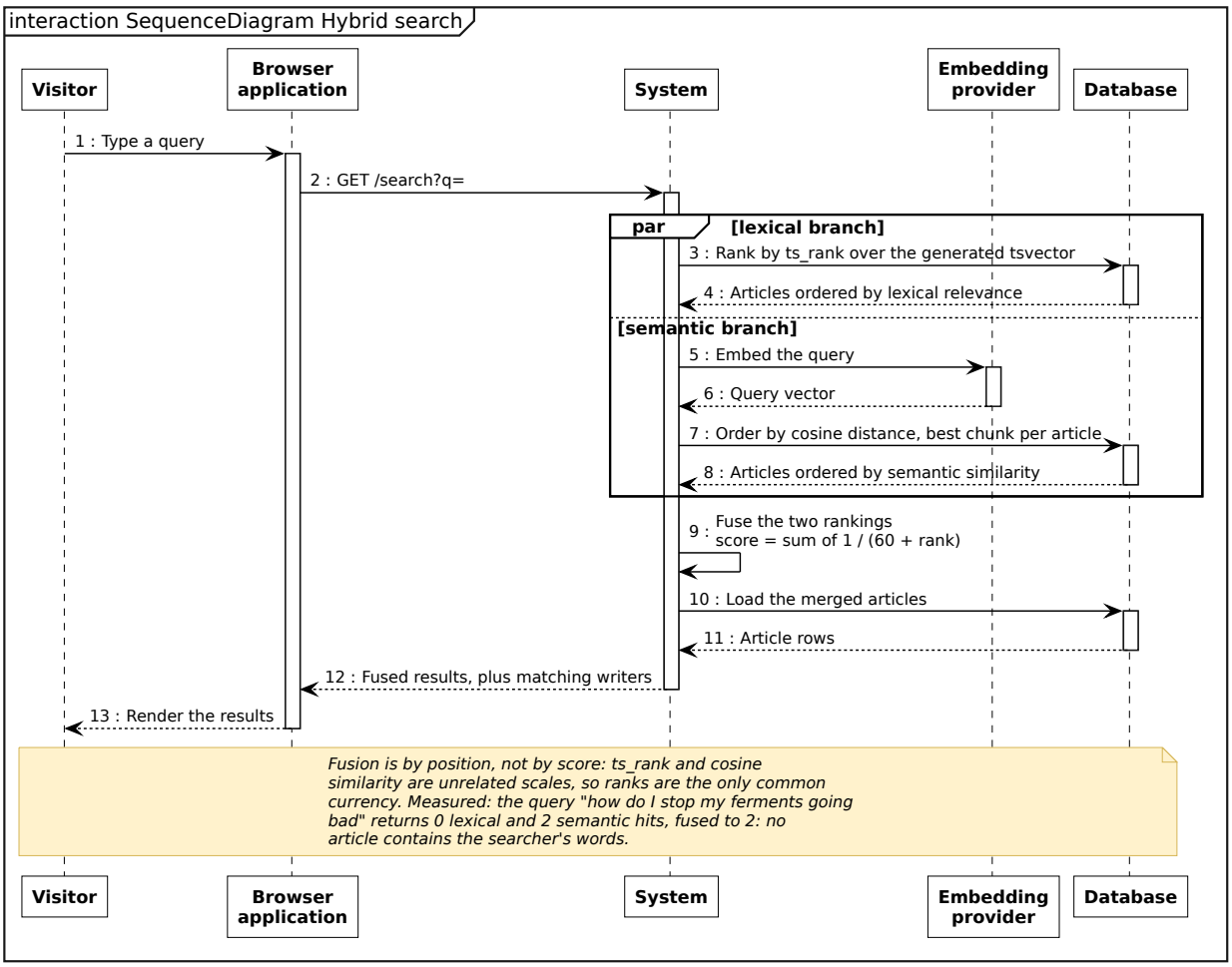


Figure 4.16 – Sequence diagram: hybrid search with reciprocal rank fusion

The query is ranked by words (`ts_rank` [23]) and by meaning (cosine similarity), and the two rankings are fused by position with reciprocal rank fusion, $\Sigma\ 1/(60 + \text{rank})$ [39], so an article can be found without sharing the searcher's words. The rankings are independent: if the embedding provider fails, search falls back to keywords. Section 7.3 compares the three.

4.2.4 Realisation

Figure 4.17 shows search results for a visitor.

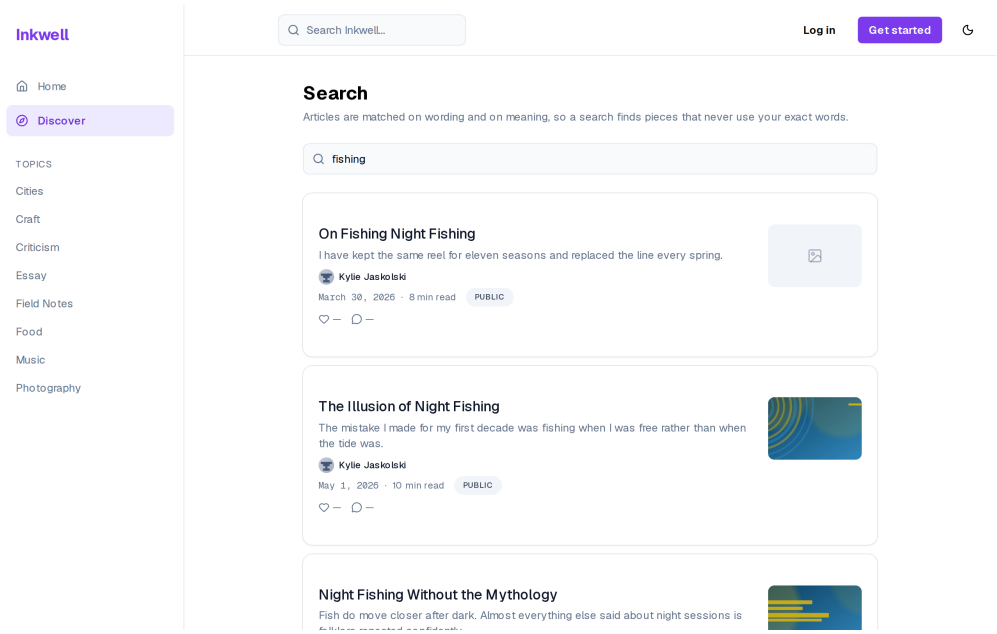


Figure 4.17 – Search results

The page says how it works: articles are matched on wording and on meaning (FR-06). For “fishing”, the first results are articles whose titles contain the word: they rank high in both the lexical and the semantic list, and reciprocal rank fusion rewards agreement between the two. Search is available without an account, like the feed.

Figure 4.18 shows the evaluation report that a magazine reads before buying.

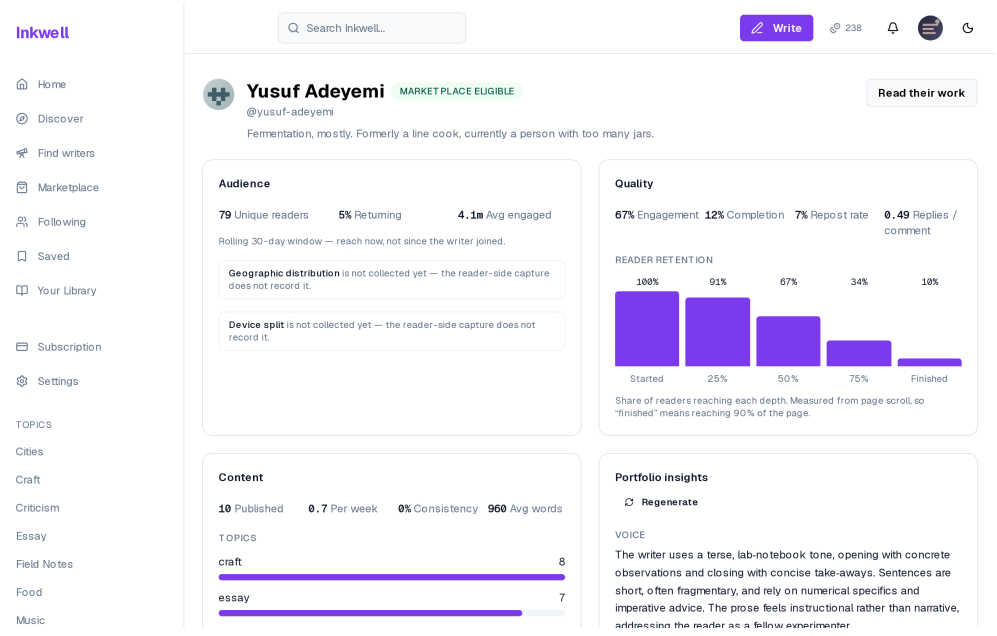


Figure 4.18 – A writer evaluation report with portfolio insights

The report gives the three panels of Table 4.2 and the portfolio insights, with *Regenerate* as the explicit action of FR-46 (FR-45). Fields the capture cannot measure, such as geography and device, are stated as not collected rather than shown as zero. This screen is why the analytics exist: it turns reading into evidence a buyer can weigh.

Conclusion

Release 2 made Inkwell.ai social and measurable: interactions with live notifications, reading events aggregated into per-article and per-writer metrics, an evaluation report for magazines, and a retrieval layer that grounds the assistant in the writer's own work, produces portfolio insights and powers hybrid search. Release 3 builds the marketplace those signals were designed for.

CHAPTER 5

Release 3: marketplace, testing and deployment

Introduction

The third release delivers the reason the analytics exist: the marketplace. Sprint 5 adds the premium plan, magazine subscriptions and credits, the eligibility gate, the two-stage purchase with exclusive rights, the magazine's own act of publishing, and the moderation console. Sprint 6 is a technical sprint: it delivers no new screen, but the test strategy that proves the business rules of Sprint 5, a demonstration corpus, and a deployable production stack. Both sprints follow the same four steps as the others.

5.1 Sprint 5: Marketplace, subscriptions, premium and moderation

5.1.1 Sprint backlog

The goal of Sprint 5 is that readers upgrade to premium; that magazines subscribe and receive monthly credits; that eligible writers list articles; that magazines preview, buy and publish them under their own masthead, every credit movement recorded in a ledger; that writers see their earnings; and that administrators moderate. Table 5.1 lists the stories.

Table 5.1 – Sprint 5 backlog

ID	User story (goal)	Priority	Points
US-12	Restrict an article to premium readers	Should	3
US-34	Upgrade to premium	Must	5
US-35	Subscribe as a magazine	Must	8
US-36	Receive a monthly credit budget	Must	5
US-37	Top up credits	Should	3
US-38	Pay 10% to read an article in full	Must	8
US-39	Pay the remainder to acquire republish rights	Must	8
US-40	Keep purchased articles in a library	Must	3
US-41	List an article for sale at my own price	Must	5
US-42	See my earnings itemised by preview and purchase	Must	5
US-43	See how far I am from eligibility	Should	3
US-44	Report content	Must	3
US-45	Work a queue of reports	Must	5
US-46	Remove an article or ban an author	Must	3
US-47	Grant eligibility manually	Must	3
US-48	Search and filter users	Should	5
US-63	Publish an article I have bought when I am ready, rather than the purchase publishing it for me	Must	5
US-64	Have the articles I publish appear on my public profile under my masthead	Must	5

ID	User story (goal)	Priority	Points
	18 stories		**85**

The sprint commits 85 points, about twice the average of the other sprints. The load was accepted rather than split because the marketplace stories form one transaction model: a preview, a purchase, a publication and an earnings line are four views of the same ledger rows, and delivering half of them would have left money that could be spent but not accounted for. The ledger harness of Sprint 1 and the screens of the design track had removed most of the uncertainty in advance. How far this estimate held is discussed with the velocity in Section 7.7.

5.1.2 Analysis

Use case diagram. Figure 5.1 gives the use cases of Sprint 5.

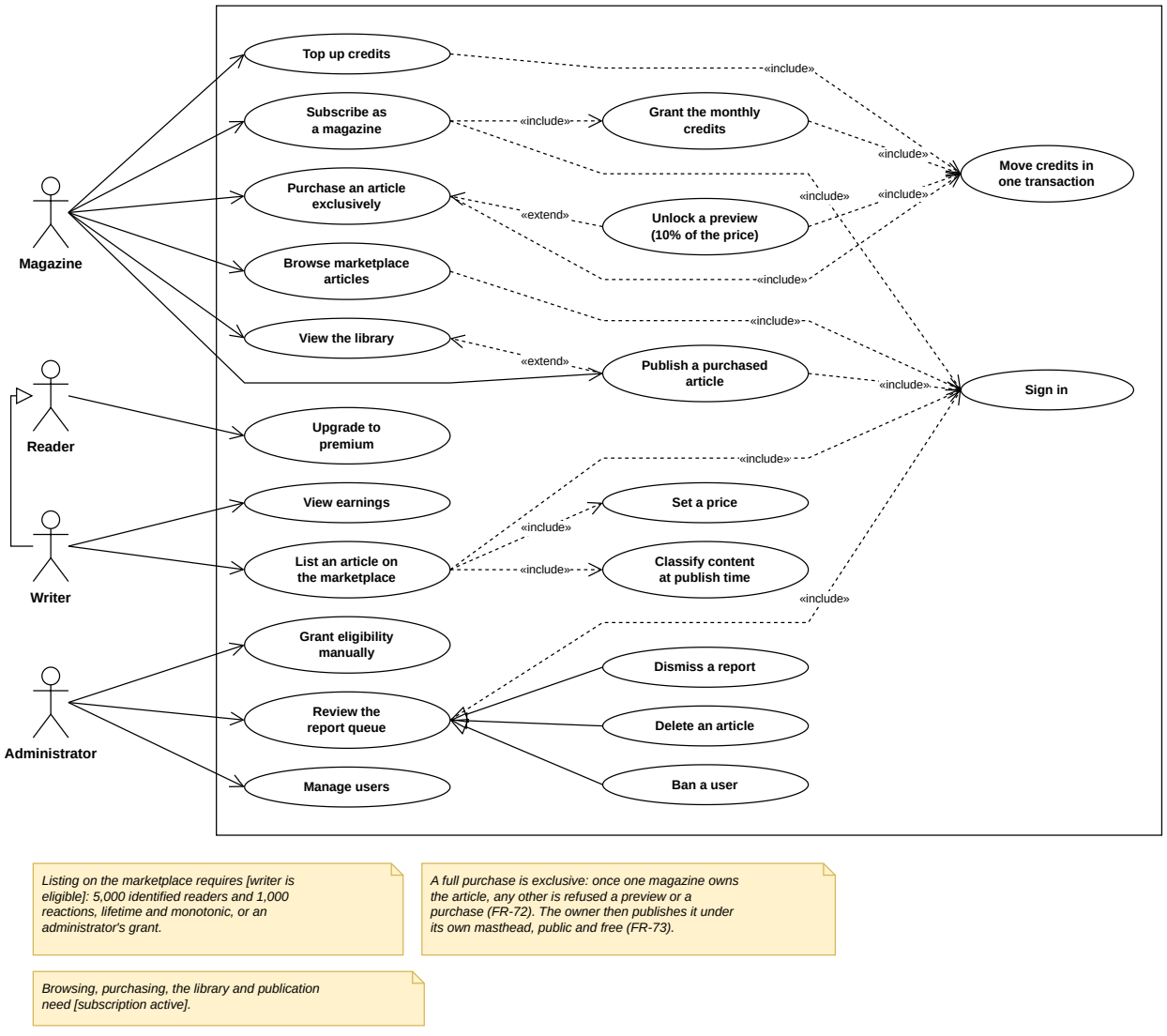


Figure 5.1 – Use case diagram, Sprint 5

Without an active subscription a magazine can only subscribe; with one, it browses, previews and buys articles exclusively, tops up credits and publishes from its library, every credit movement in one transaction. A writer lists an article when `[writer is eligible]`; the administrator moderates and grants eligibility. The subscription is the wall in front of the marketplace.

Business rules. The marketplace is governed by the rules of Table 5.2. They are part of the analysis because every screen and every test of the sprint applies them.

Table 5.2 – Marketplace rules and their justification

Rule	Value	Justification
Eligibility to list or publish premium	5,000 identified unique readers **and** 1,000 reactions (likes and comments) across the writer's public articles, lifetime; or a manual grant by an administrator; never revoked automatically	both an audience and a response to it, so neither a viral article nor a small circle of friends qualifies alone; never revoked, so a writer's listings do not disappear in a quiet month
Magazine subscription	monthly; 500 credits a month; unspent credits roll over	a predictable budget (US-36); roll-over, so a magazine is not pushed to spend
Preview unlock	10% of the article's price, credited towards the purchase	cheap enough to read before deciding, and never lost if the magazine buys
Full purchase	the remaining 90%; exclusive	exclusivity is what a publication pays for; the preview is deducted so the total is the price
Platform fee	20% of each stage, rounded down; the writer receives the rest	rounded down so that the writer, not the platform, keeps the indivisible credit
Publication	a separate act of the owning magazine: the article becomes public and free, its price is cleared and the magazine is recorded as publisher	buying is not publishing; the magazine decides when its masthead appears (US-63)

The rules have one property in common: each is an integer computation with a single rounding, so the sum of what the writer, the platform and the magazine see always reconciles to the credit. The values themselves (10%, 20%, 500, 5,000 and 1,000) are product decisions, kept as named constants (Section 2.3).

5.1.3 Design

Class diagram. Figure 5.2 shows licensing and the ledger.

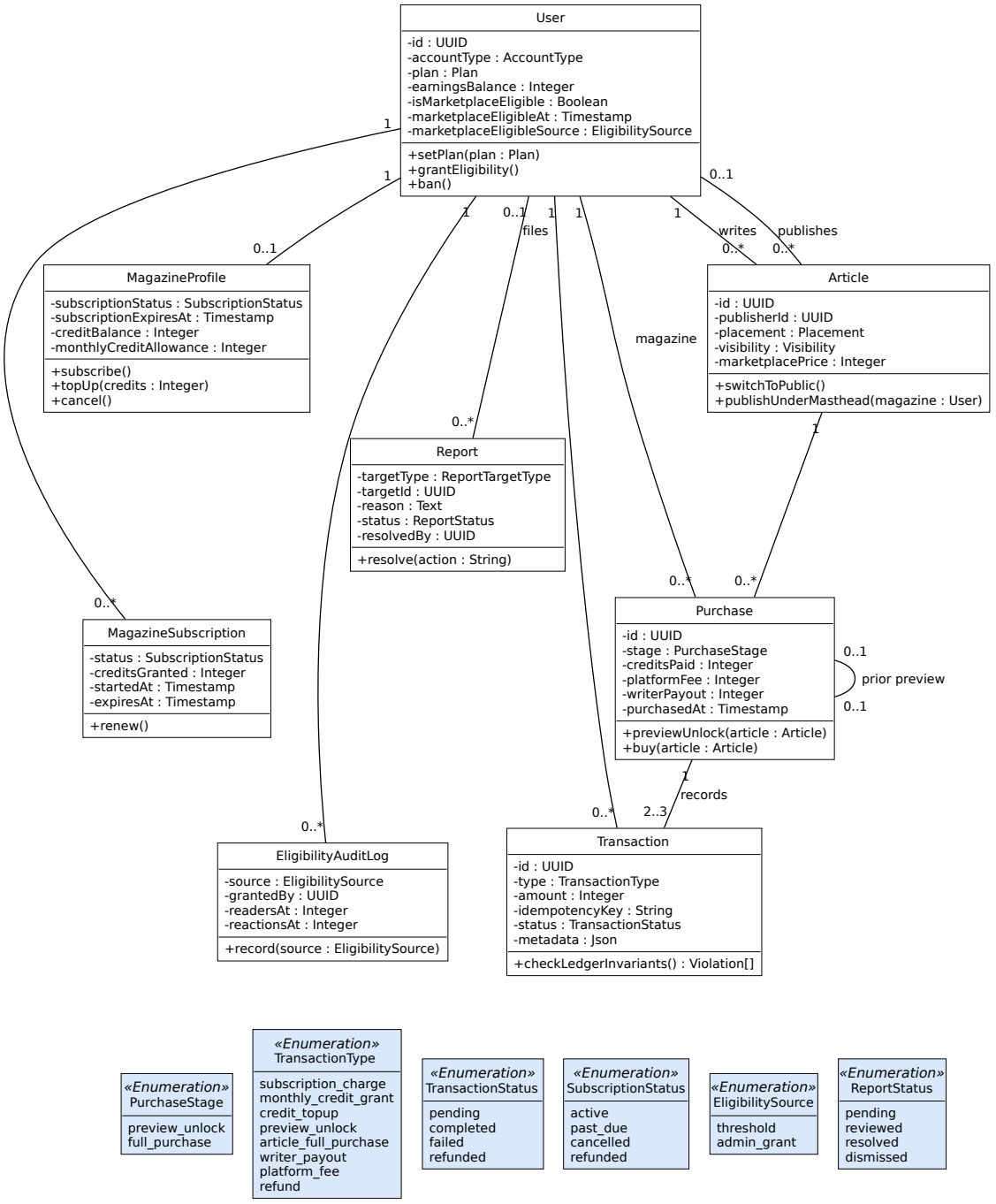


Figure 5.2 – Class diagram, Sprint 5

A `Purchase` records a stage (preview or full purchase) with the credits paid, the fee and the payout, and writes three `Transaction` rows: the magazine's debit, the writer's payout and the platform fee. No balance is stored without its explanation: credit balances and earnings are caches of these rows, which the ledger invariants check (NFR-19).

Sequence diagram: the purchase. Figure 5.3 shows the three calls of a licence.

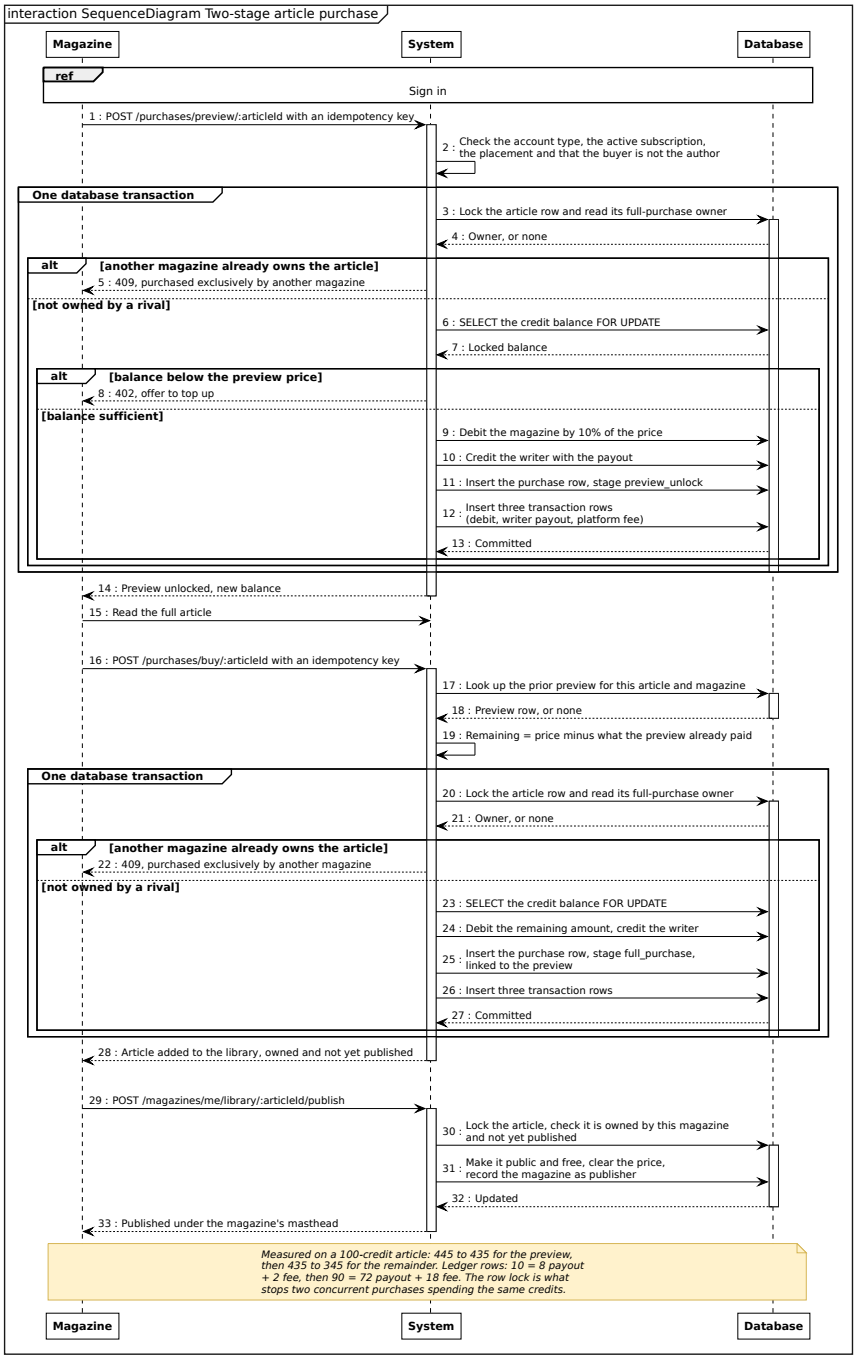


Figure 5.3 – Sequence diagram: preview, purchase and publication

Each stage is one transaction: it locks the article (409 if already owned) and the credit balance (402 if too low), moves the credits and writes the ledger rows. The row lock rules out a double purchase or double spend (NFR-20); the idempotency key, a retried request charging twice (NFR-06).

Activity diagram: the eligibility gate. Figure 5.4 shows how a writer becomes eligible.

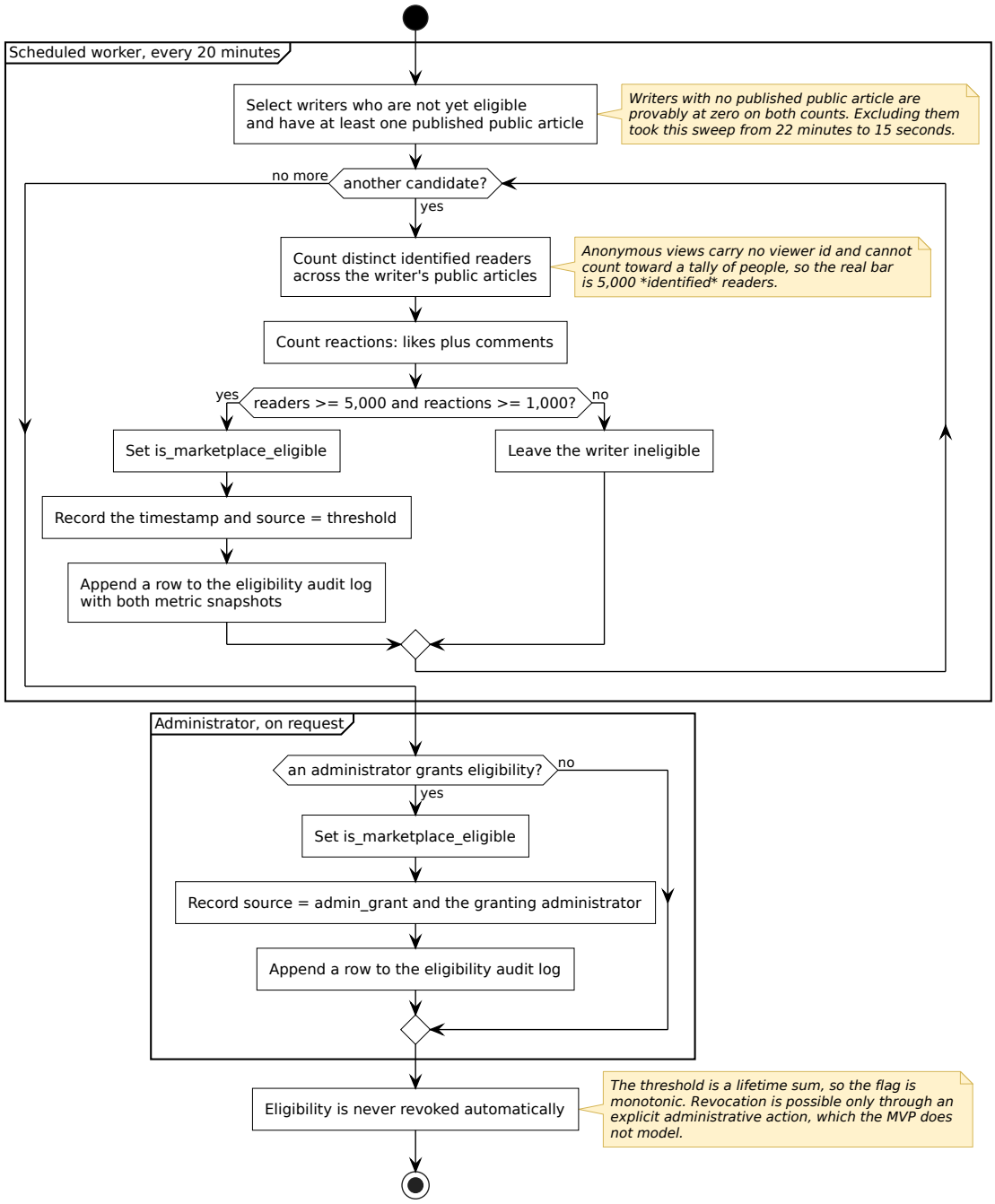


Figure 5.4 – Activity diagram: the marketplace eligibility gate

After each aggregation the worker marks the writers above both thresholds eligible and logs the metric snapshot; an administrator can also grant eligibility, with the same audit entry. Comment likes and reposts do not count, so other people's activity cannot move a threshold. Every eligible writer can be traced to the numbers or the person that made them eligible.

Sequence diagram: moderation. Figure 5.5 shows the two sources of reports.

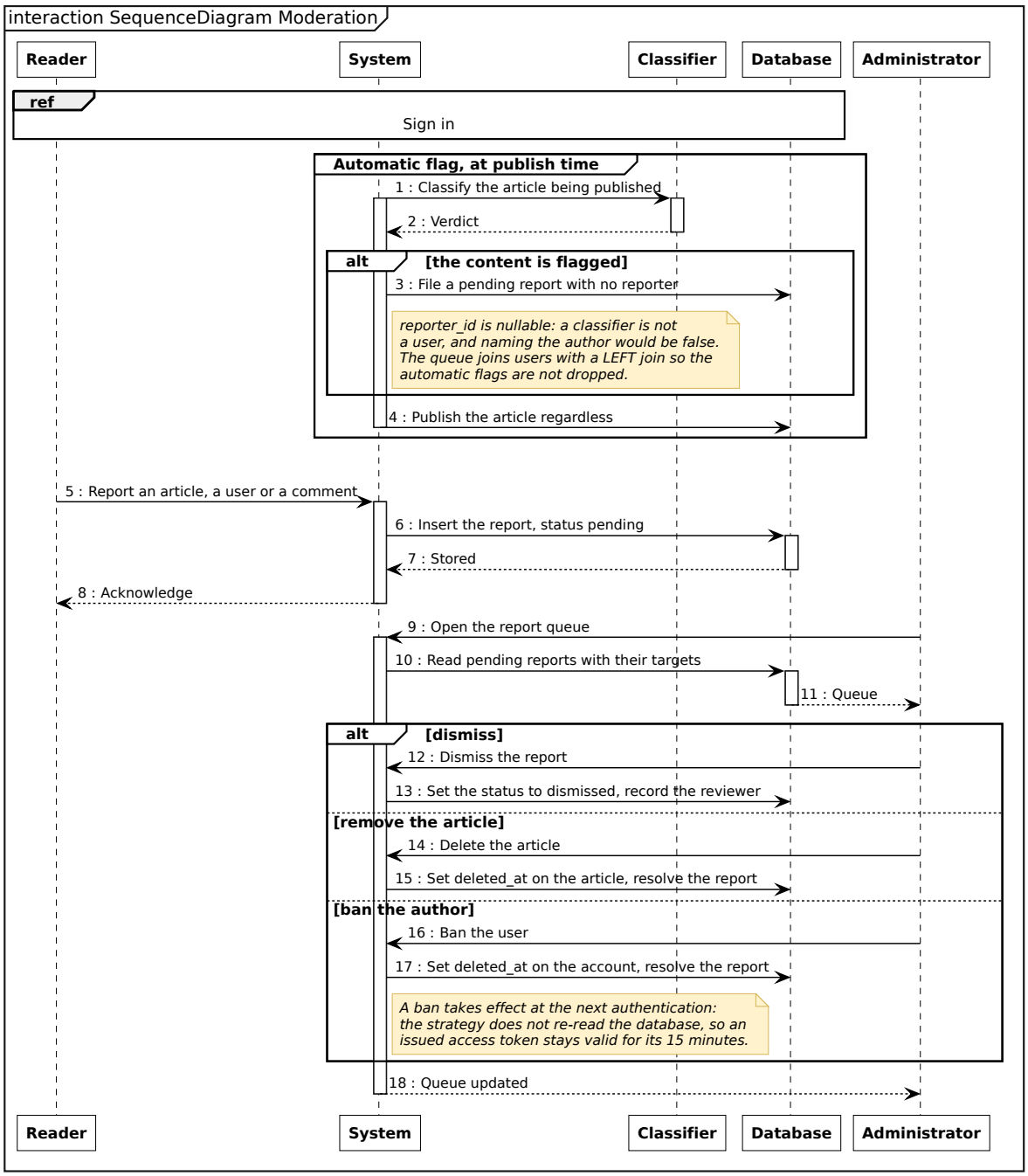


Figure 5.5 – Sequence diagram: reporting and moderation

After publication a classifier reviews the article and, if it flags it, files a pending report without blocking publication; readers report articles, users or comments. The administrator dismisses, removes the article or bans the author (in effect within fifteen minutes, NFR-09). A false positive costs a dismissed report, never a refused article; Section 7.4 measures the classifier.

Activity diagram: the article access decision. Figure 5.6 shows the decision taken server-side for every article body.

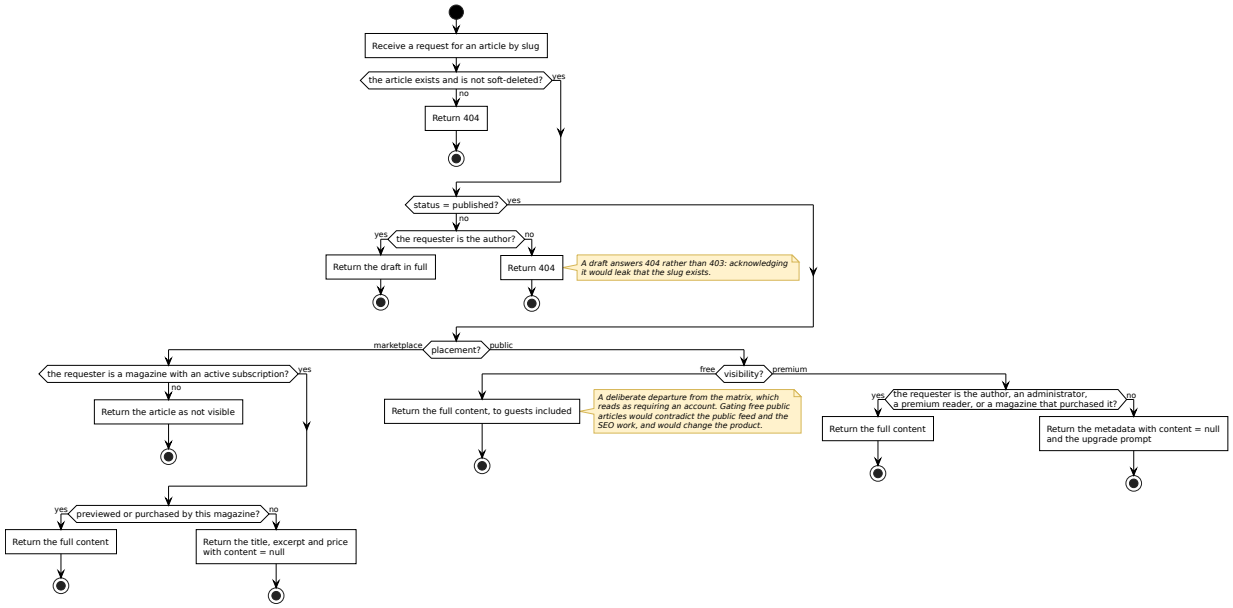


Figure 5.6 – Activity diagram: the article access decision

The author and administrators always read. A marketplace body reaches only a subscribed magazine that previewed or bought it, a premium body only the premium plan or a subscribed magazine; otherwise the content is `null`. The server decides, so a modified client cannot read what it has not paid for.

State diagram: the article lifecycle. Figure 5.7 shows the states an article goes through.

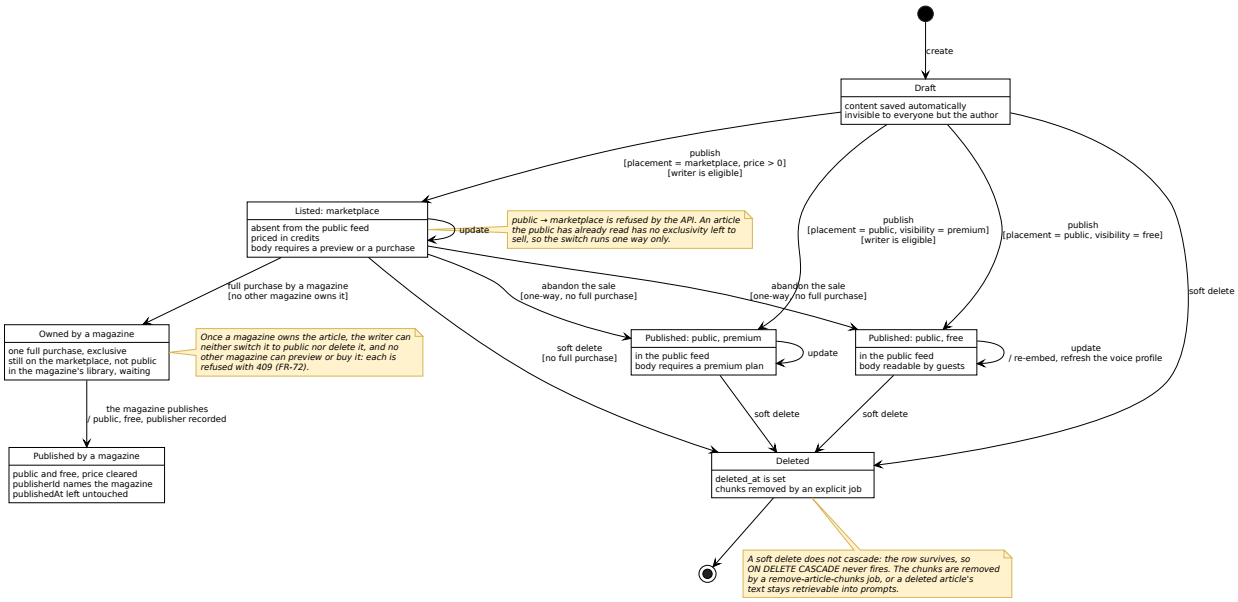


Figure 5.7 – State diagram: the article lifecycle

A draft is published as free, premium or listed. A listing can become public (never the reverse) or be deleted while unsold; a purchase makes it *owned*, and the magazine's publication public. No transition can destroy what a magazine paid for, which enforces the exclusivity of Table 5.2.

5.1.4 Realisation

The premium plan comes first, because it is what a reader meets. Figure 5.8 shows a premium article opened by a visitor.

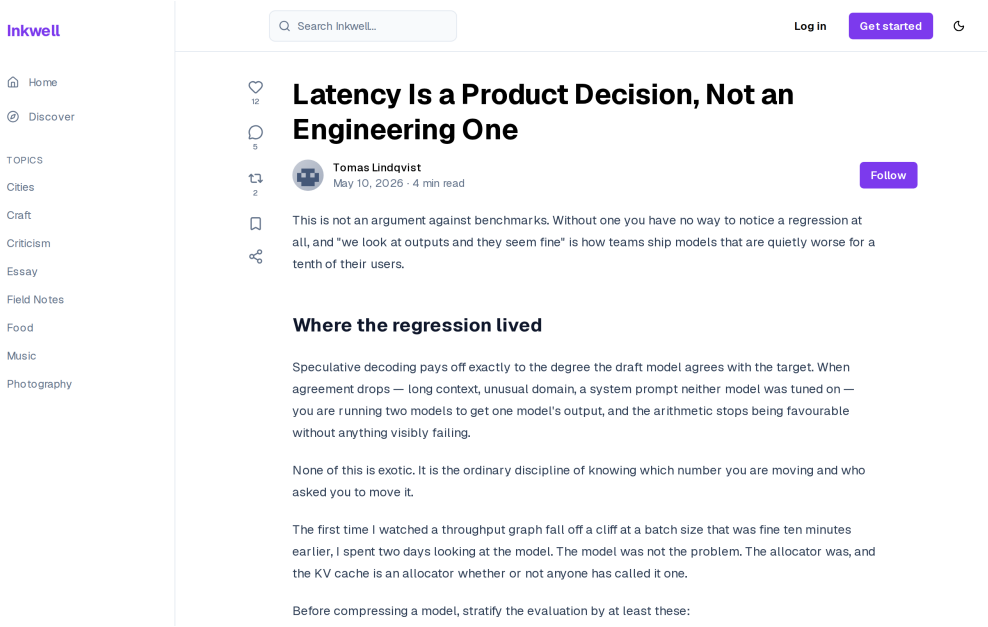


Figure 5.8 – A premium article behind the paywall

The visitor sees the title, the author and the opening; the body is withheld by the access decision of Figure 5.6 (FR-03, FR-16). It never reaches the browser, so inspecting the page cannot reveal it.

Figure 5.9 shows the plans offered to a reader.

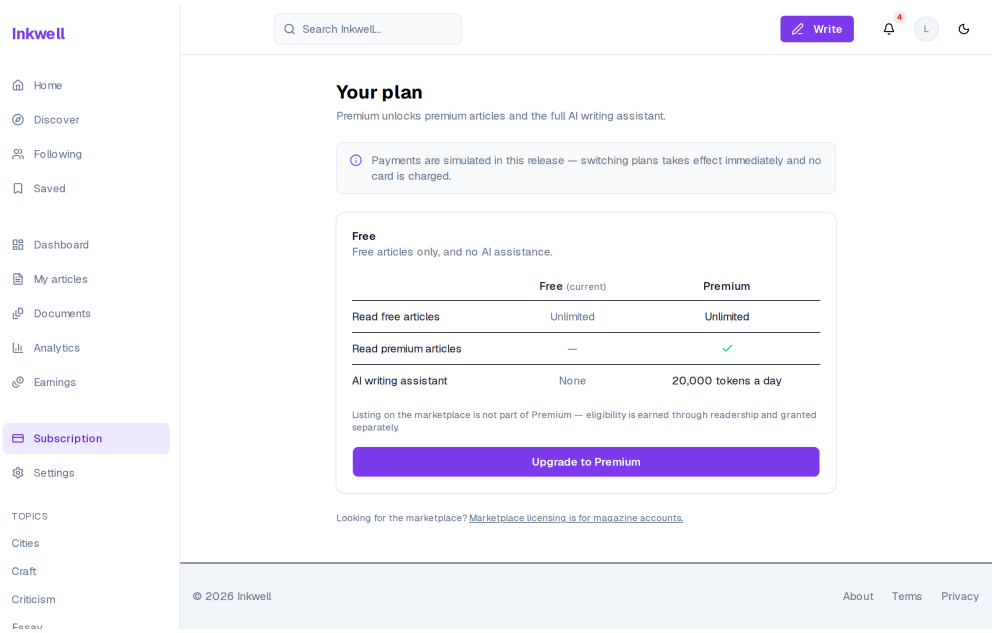


Figure 5.9 – The plans: free and premium

The comparison states the difference between the plans (premium articles and the AI assistant with 20,000 tokens a day), and the banner states that payments are simulated in this release (FR-15). The page also says

that listing on the marketplace is not part of premium: eligibility is earned, not bought, which is the rule of Table 5.2.

The magazine's side starts with its subscription. Figure 5.10 shows a subscribed magazine.

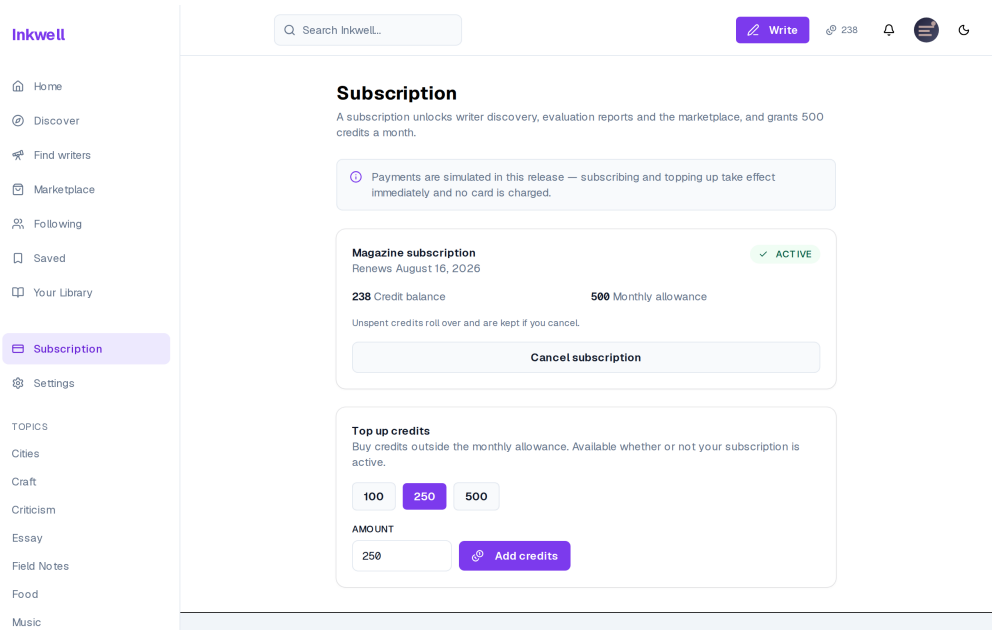


Figure 5.10 – A magazine's subscription and credits

The page shows the active subscription and its renewal date, the credit balance (238) and the monthly allowance (500), and offers a top-up of 100, 250 or 500 credits (FR-41, FR-42, FR-43). The note that unspent credits roll over and are kept on cancellation is the subscription rule of Table 5.2, stated where the magazine decides.

Figure 5.11 shows the marketplace and Figure 5.12 a listing before preview.

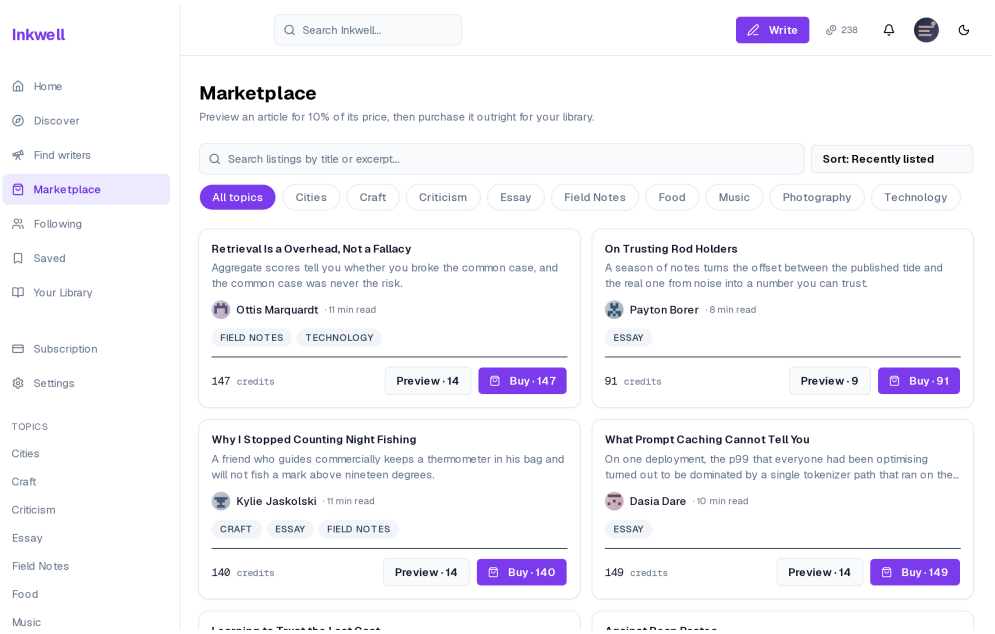


Figure 5.11 – The marketplace, with preview and purchase prices

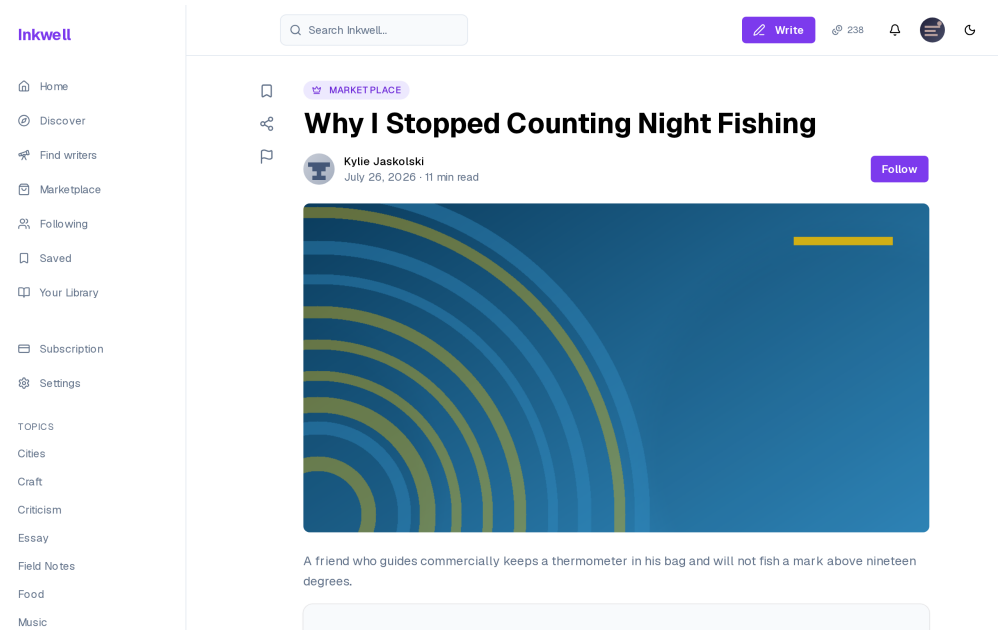


Figure 5.12 – A listing before preview

Each listing shows its writer, topics and price, and two buttons: *Preview* at 10% of the price (14 credits for an article at 140) and *Buy* at the full price (FR-47). The listing page shows the excerpt only; the body stays locked, as Figure 5.6 requires, until the magazine pays.

Figure 5.13 shows the confirmation of a preview unlock.

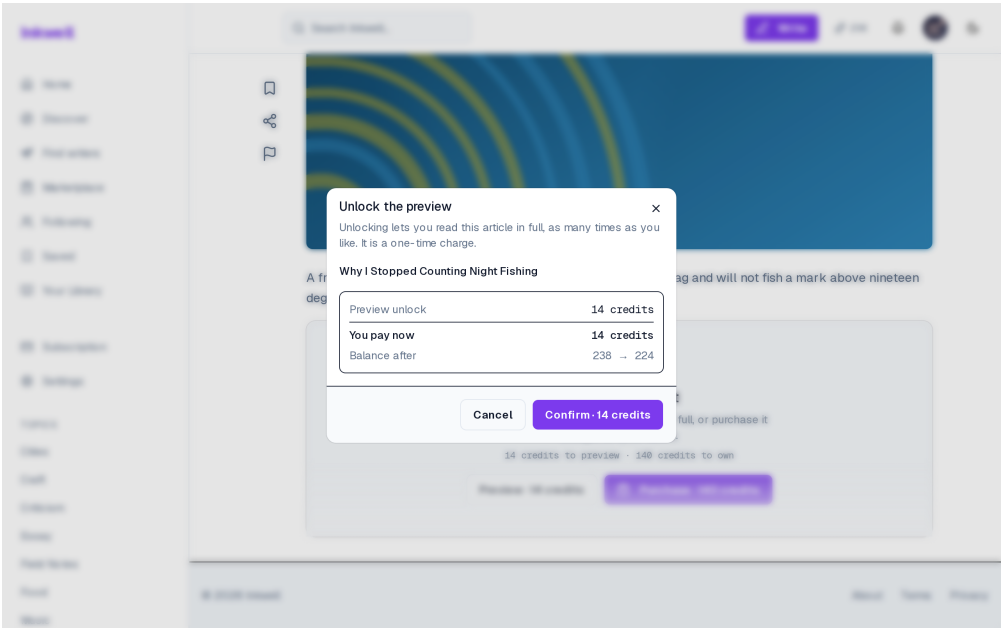


Figure 5.13 – Confirming a preview unlock: price and balance after

Before any credit moves, the dialog states what is bought, what is paid now and the balance after (238 → 224), and requires an explicit *Confirm* (FR-48). It is the screen of the first call of Figure 5.3: the figures it shows are the ones the transaction will write, so the magazine never discovers a charge after the fact.

After a full purchase the article lands in the magazine's library. Figure 5.14 shows it and Figure 5.15 the act of publishing from it.

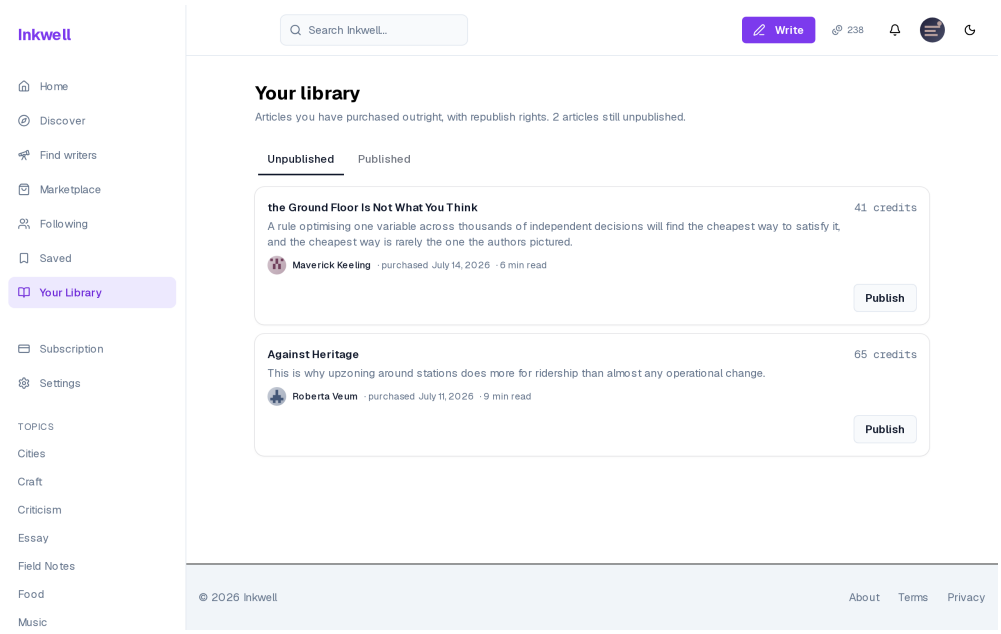


Figure 5.14 – The magazine's library: unpublished and published

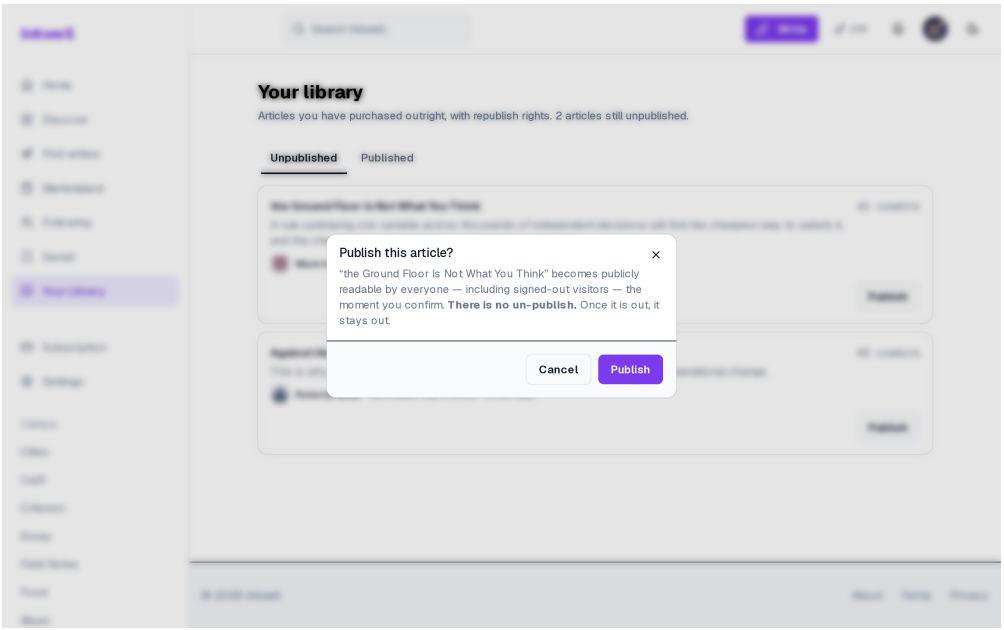


Figure 5.15 – Publishing a purchased article under the magazine's masthead

The library separates what is owned but unpublished from what has been published, with the price paid and the purchase date of each article, and offers *Publish* only on the first tab (FR-50, FR-74). Publishing is confirmed separately (FR-73): buying and publishing are two decisions, as Table 5.2 requires.

Once published, the article becomes public. Figure 5.16 shows it, and Figure 5.17 the magazine's public profile.



Figure 5.16 – A licensed article naming both the writer and the magazine

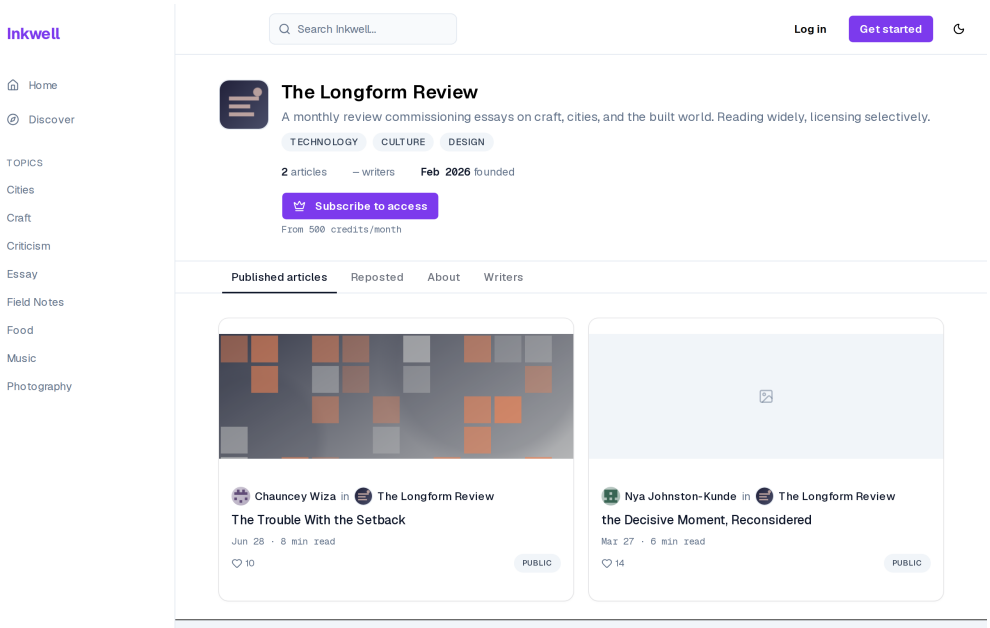


Figure 5.17 – A magazine's public profile and its published articles

The licensed article names both the writer and the publishing magazine, each linking to its profile (FR-76), and the magazine's profile lists the articles it has published (FR-75). These two requirements are attributed to Sprint 5 because a licensed article exists only from this sprint on: the licence buys the magazine a shelf, not just a file.

The writer sees the other side of the same transactions. Figure 5.18 shows the earnings and Figure 5.19 a sold article.

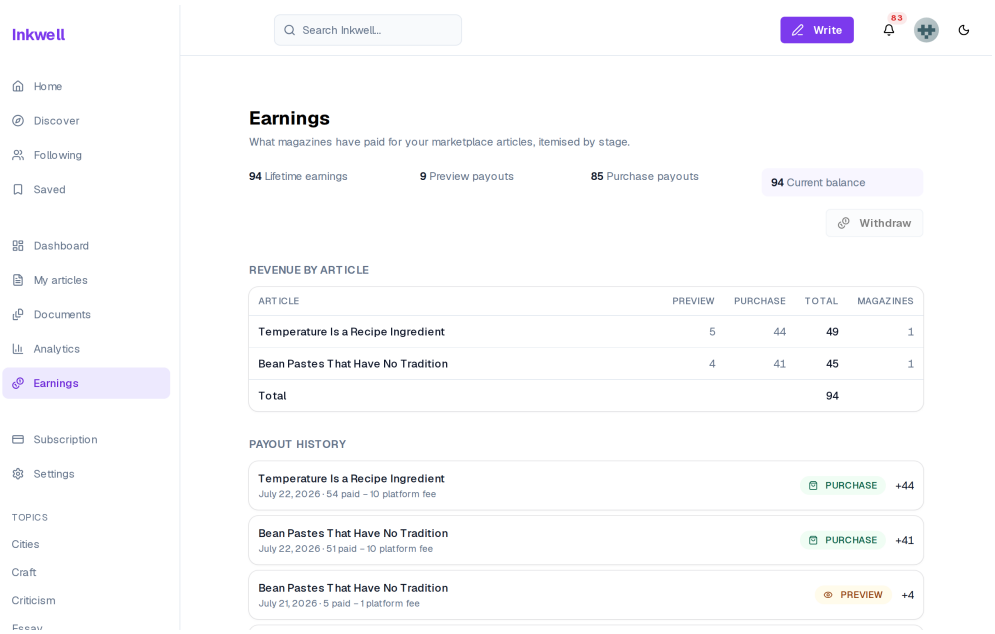


Figure 5.18 – A writer's earnings, itemised by preview and purchase

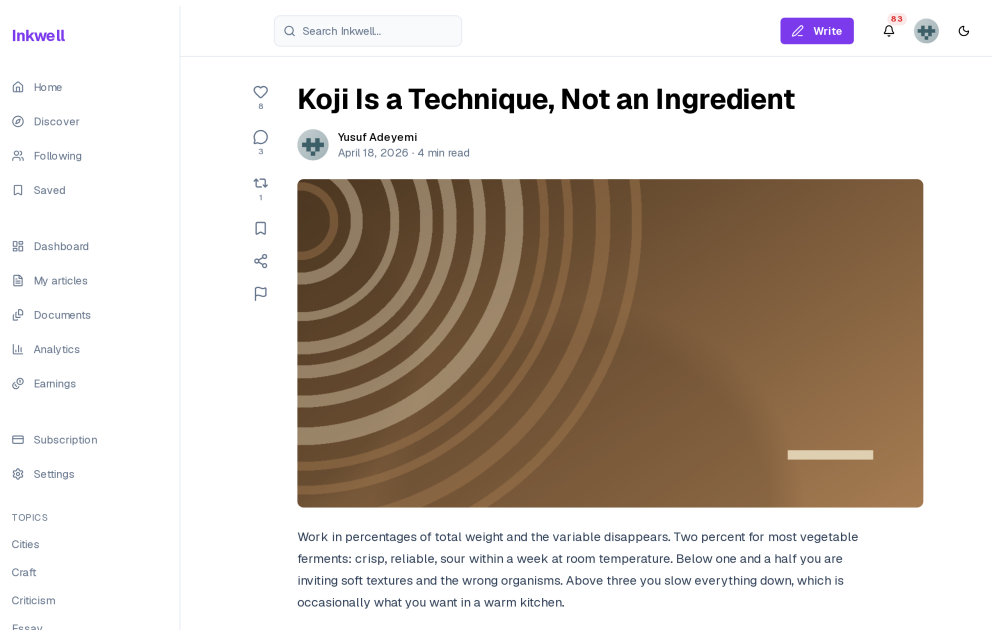


Figure 5.19 – A sold article, as its author sees it

Earnings are itemised by article and stage: 54 credits paid, 10 of fee (20% rounded down), 44 to the writer (FR-37). The arithmetic of Table 5.2 is visible line by line, as US-42 asks. A sold article names its owner and can no longer be withdrawn or deleted (FR-72).

The administrator closes the sprint. Figure 5.20 shows a manual eligibility grant, Figure 5.21 user management and Figure 5.22 the report queue.

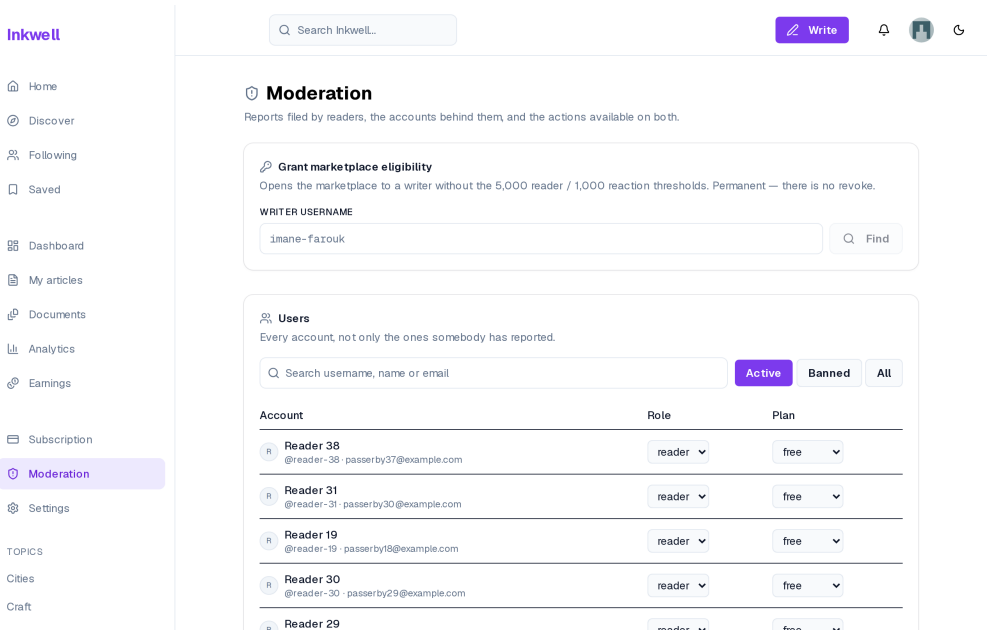


Figure 5.20 – Administration: manual eligibility grant

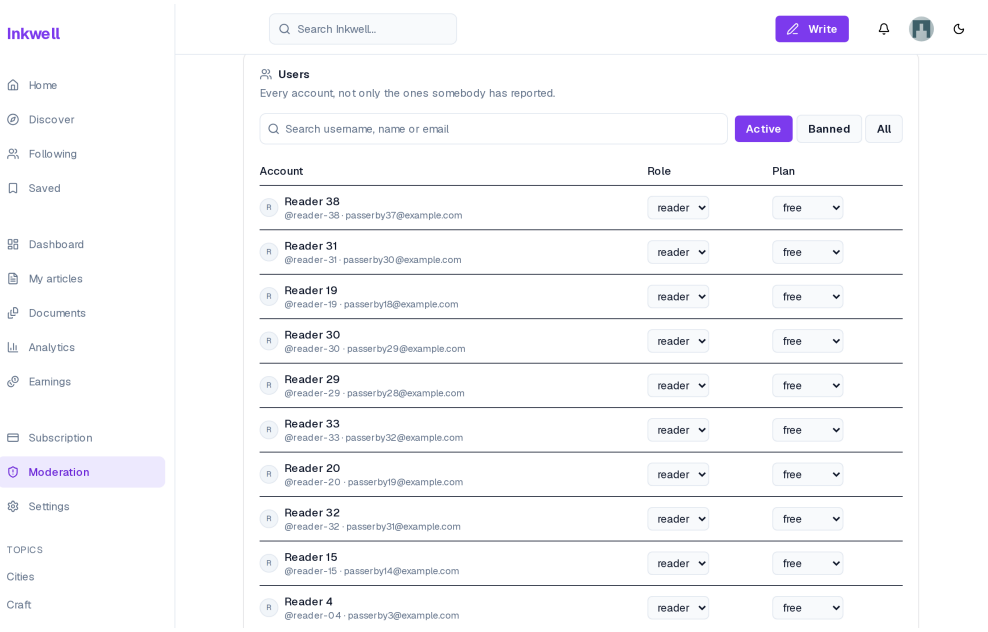


Figure 5.21 – Administration: user management

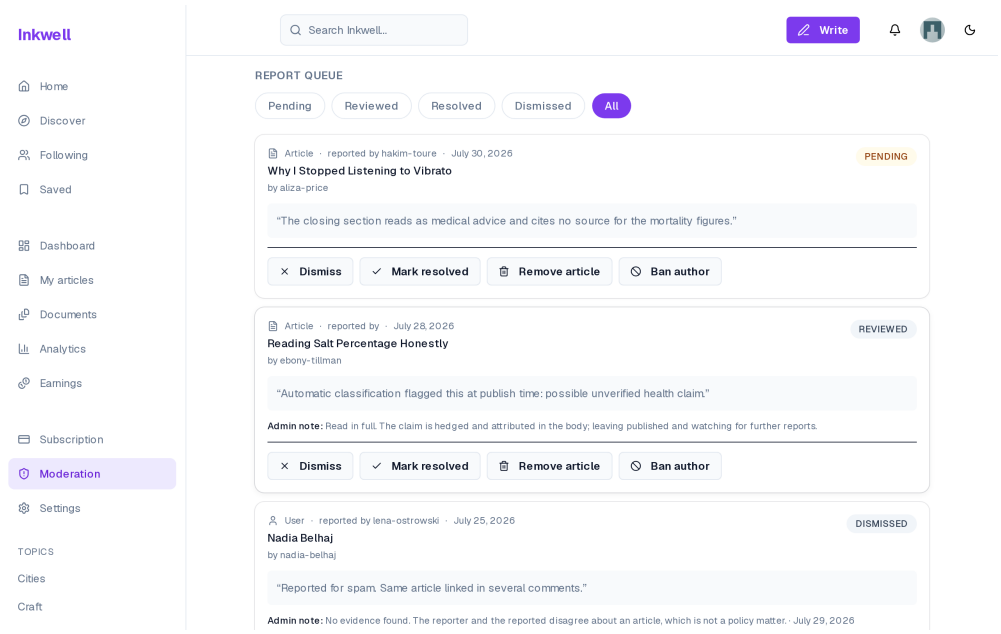


Figure 5.22 – Administration: the report queue

The grant records who made a writer eligible (FR-55); the user table searches accounts and edits a plan (FR-53, FR-54); the report queue offers the four actions of Figure 5.5 (FR-51, FR-52). Its second report has no reporter: the classifier filed it (FR-57), and the administrator's note explains why the article stayed: the decision the design reserves for a person.

5.2 Sprint 6: Tests, demonstration and deployment

5.2.1 Sprint backlog

The goal of Sprint 6 is to prove the business rules with tests, to make the product demonstrable on a realistic corpus, and to make it deployable with one command and an automated pipeline. Table 5.3 lists the stories.

Table 5.3 – Sprint 6 backlog

ID	User story (goal)	Priority	Points
US-52	Apply migrations automatically before the application starts	Must	5
US-53	Seed a believable corpus	Must	5
US-54	Lower the eligibility thresholds under a flag	Must	2
US-55	Serve the product at a public URL over TLS	Must	8
	4 stories		**20**

The sprint commits only 20 points because its main work is not expressed as user stories: writing the tests that prove the rules of Sprint 5 adds no story, but it is what the definition of done of the previous sprints relies on. The sprint is technical on purpose. Placed after the marketplace and before the last release, it consolidates the rules that move money before more features are built on top of them.

5.2.2 Analysis

A technical sprint has no new actor-facing use case; its analysis states what must be verified and what must be deployable. Three needs follow from the previous sprints.

- **Proof of the money rules.** The ledger, the prices and the exclusivity of Sprint 5 must be shown correct by tests that fail when the code is wrong, not merely pass when it is right (NFR-06, NFR-19, NFR-20, NFR-31).
- **A demonstrable product.** A demonstration needs readers, writers above and below the eligibility thresholds, magazines with and without a subscription, purchases and history, none of which an empty database has (US-53, US-54).
- **A deployable stack.** The services must start in the right order, the schema must be migrated before the application, and the product must be reachable over TLS (NFR-21, US-52, US-55).

5.2.3 Design

Test strategy. Figure 5.23 shows the strategy.

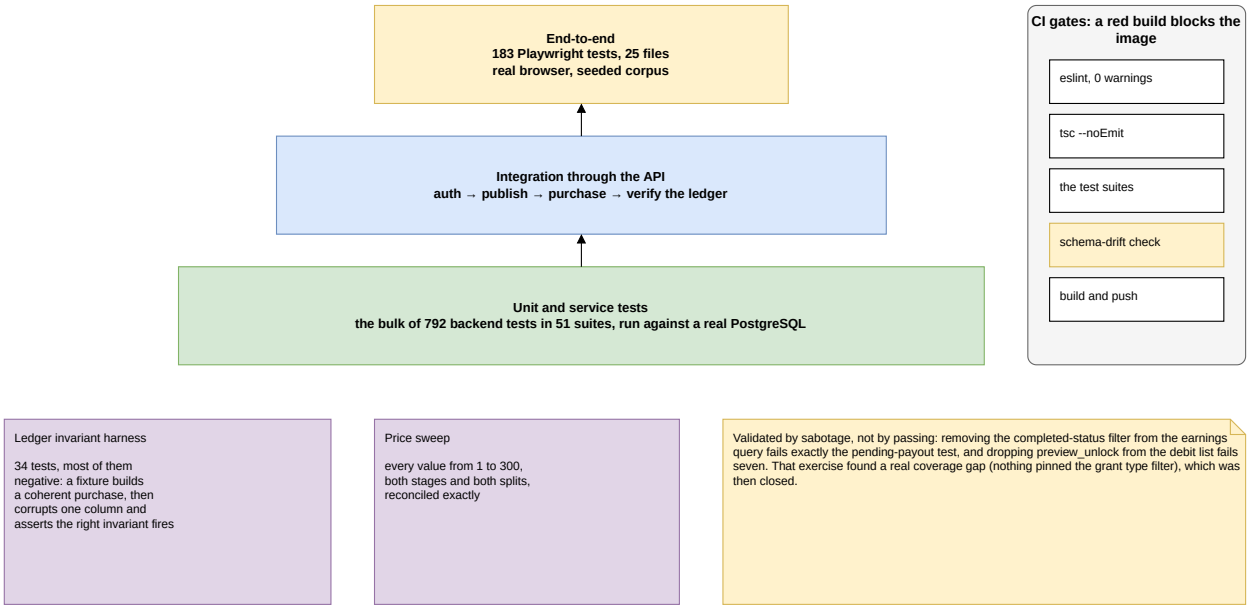


Figure 5.23 – Test strategy

Unit and service tests run against a real database, each in a rolled-back transaction; **integration tests** go through the API; **end-to-end tests** drive a browser. The **ledger harness** and a **price sweep** from 1 to 300 credits guard the money. The database is never mocked: the guarantees under test live in it.

Deployment. The deployment follows the physical architecture of Figure 2.3 and the pipeline of Figure 2.4: multi-stage images for the API and worker (one image, two entry points) and for the Next.js server in standalone mode; a production Compose file in which only nginx is published, with certificates from Let’s Encrypt; a one-shot migration service that must succeed before the API and worker start; and health and readiness endpoints checked by Compose and by the deployment workflow.

5.2.4 Realisation

Test results. Table 5.4 gives the state of the test suites at the end of the project.

Table 5.4 – Test results

Suite	Scope	Result
Backend (Jest)	52 suites: services, controllers, ledger, purchases, documents, voice, AI pipeline	**796 tests, all passing**
Ledger invariant harness	balance and ledger invariants, seed ledger	34 tests, included above
Price sweep	prices 1–300, both stages, both splits	included above
Backend line coverage	statements executed by the suite	48.5% of lines, 40.6% of branches
End-to-end (Playwright)	25 spec files over the seeded corpus	183 tests, outside the CI gate
Static gates	`tsc --noEmit` and `eslint --max-warnings=0`, both repositories	pass

The 796 tests are not spread evenly, and the coverage per module shows where they are. The moderation module is covered at 92% of its lines, the purchases at 77%, the shared utilities at 73% and the data layer, where the ledger checks live, at 67%: the code that moves money or removes content is the most tested, as the strategy intended. The AI module (21%), authentication (28%) and the analytics aggregation (0%) are the least covered: their logic is largely SQL and provider calls, verified by the end-to-end suite and by the measurements of Chapter 7 rather than by unit tests. Line coverage of 48.5% overall is therefore not a uniform level of confidence but a concentrated one, and extending it to the aggregation queries is the first improvement listed in the conclusion. The harness was itself validated by sabotage: removing a filter from the earnings query or a stage from the debit list fails exactly the tests that should fail, an exercise that found one coverage gap, which was then closed.

Demonstration corpus. A seed command builds a believable dataset in seconds and can rebuild it from scratch: its full preset creates **100 writers and 989 articles**, **50 magazines** (30 with a live subscription), **150 readers**, 110 marketplace listings with real purchases and ledger rows, and more than 15,000 likes, 2,700 reposts and 8,000 comments, with generated cover art and avatars. A fixed cast of named accounts makes every demonstration path reproducible, and a flag lowers the eligibility thresholds so that the gate can be crossed live (US-54). Every seeded date can be bounded below, so the corpus never predates the platform. The same corpus is the one evaluated in Chapter 7.

Deployment. The production stack builds, migrates and starts with one command, and its pipeline runs from a push to a running service (US-52). The domain `inkwell-ai.me` is registered, but no server was provisioned during the project, so the product is not yet reachable at a public address: **US-55 is only partly delivered**: its configuration, TLS setup and pipeline are done, its hosting is not, and its 8 points are not counted as delivered in the velocity of Section 7.7.

Conclusion

Release 3 delivered the marketplace end to end: subscriptions and credits, the eligibility gate, the exclusive two-stage purchase, the magazine's own publication and the writer's itemised earnings, all on a ledger whose invariants are tested after every operation. It also delivered moderation, a test suite of 796 backend tests concentrated on the rules that move money, a demonstration corpus and a production-ready stack whose public hosting remains to be done. The last release deepens the social layer and reworks the assistant.

CHAPTER 6

Release 4: social depth, the reworked assistant and documents

Introduction

The last release deepens what the platform already did. Sprint 7 completes the social layer with comment likes, follower lists, blocking, saving and sharing. Sprint 8 reworks the assistant: it now decides on each request whether to answer or to write into the document, shows its real pipeline as it runs, and accepts and speaks voice; the feed card becomes interactive. Sprint 9 gives the writer a private library of reference documents that the assistant can draw on and cite by page.

6.1 Sprint 7: Comment likes, lists, block, save and share

6.1.1 Sprint backlog

The goal of Sprint 7 is that readers like comments, browse and act on follower lists, block accounts, keep articles for later and share them outside the platform. Table 6.1 lists the stories.

Table 6.1 – Sprint 7 backlog

ID	User story (goal)	Priority	Points
US-56	Block an account	Must	5
US-57	Save an article to come back to it later	Must	5
US-58	Pass an article on to people outside the platform	Should	3
US-73	Like a comment	Must	2
US-74	Browse who follows an account and whom it follows, and follow from the list	Must	3
	5 stories		**18**

The sprint commits 18 points, a light load: its five stories reuse the interaction model of Sprint 3 (each is a new many-to-many association with a notification), so they were small to estimate and to build. The sprint was kept light deliberately, as the start of the release that runs alongside the writing of this report.

6.1.2 Analysis

Figure 6.1 gives the use cases of Sprint 7.

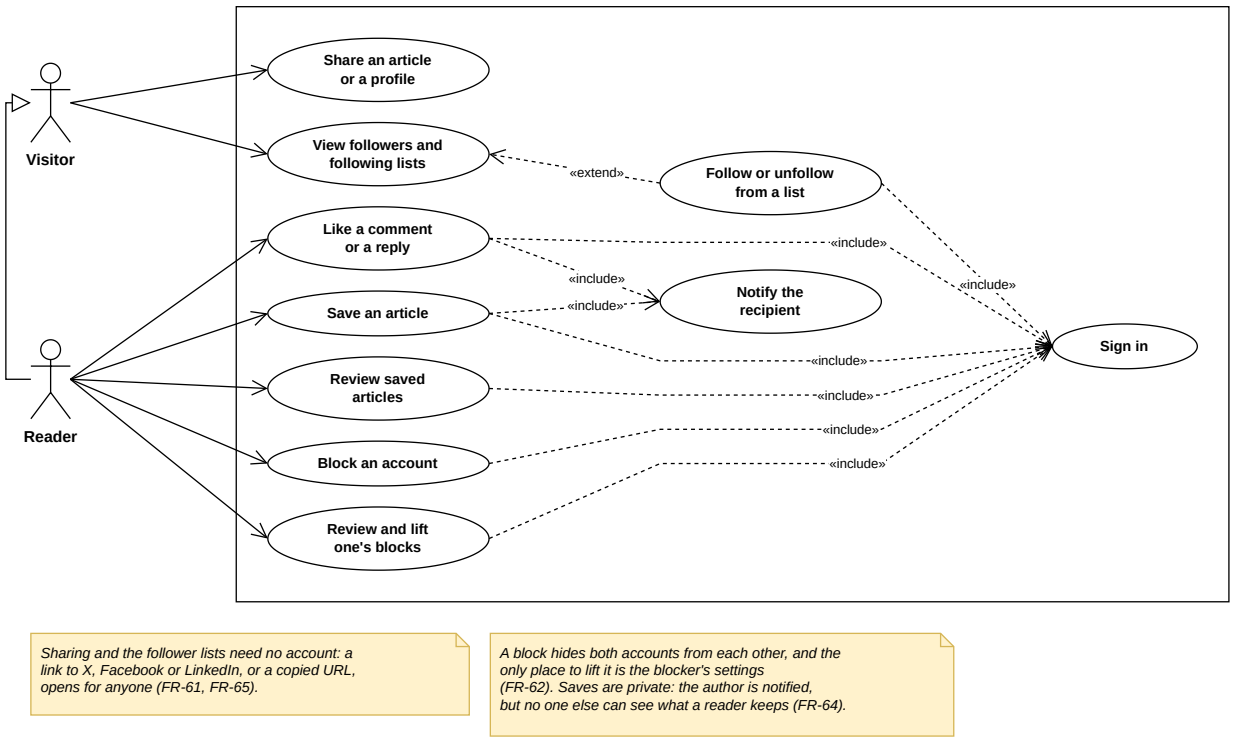


Figure 6.1 – Use case diagram, Sprint 7

Sharing and the follower lists need no account; following from a list extends viewing it; liking a comment and saving include a notification. A block hides two accounts from each other and is lifted only in the blocker’s settings, and saves are private. Liking, following and sharing are public signals; saving and blocking are private choices only their author sees.

6.1.3 Design

Figure 6.2 shows the three new associations and the notification types they emit.

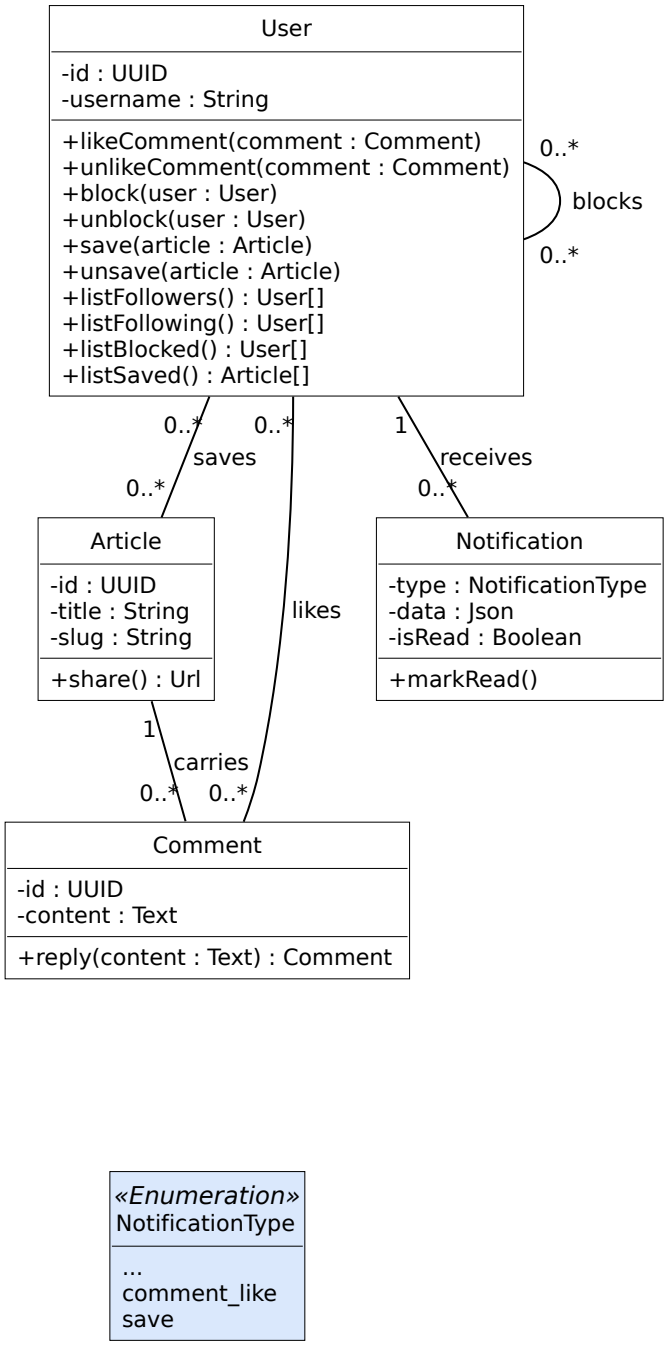


Figure 6.2 – Class diagram, Sprint 7

Comment likes, blocks and saves are many-to-many associations created by the operations this sprint adds to `User`, each a join table with a unique pair, so a repeated request is a no-op; a block works both ways, and comment likes never count towards eligibility. No existing class changed: the social layer grew by association only, without reopening the marketplace rules.

6.1.4 Realisation

Figure 6.3 shows the Saved shelf of a reader.

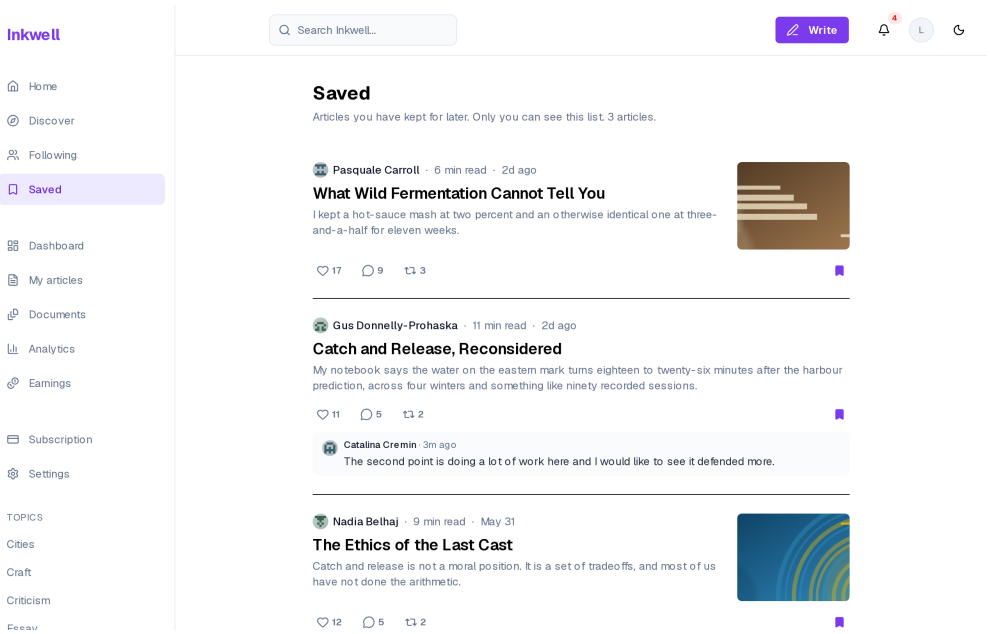


Figure 6.3 – The Saved shelf

The page states that only the reader can see the list, and every card keeps its controls, with the save marker filled (FR-63, FR-64). A saved article stays one click away from the sidebar, which is what US-57 asks.

Figure 6.4 shows the blocked accounts in settings.

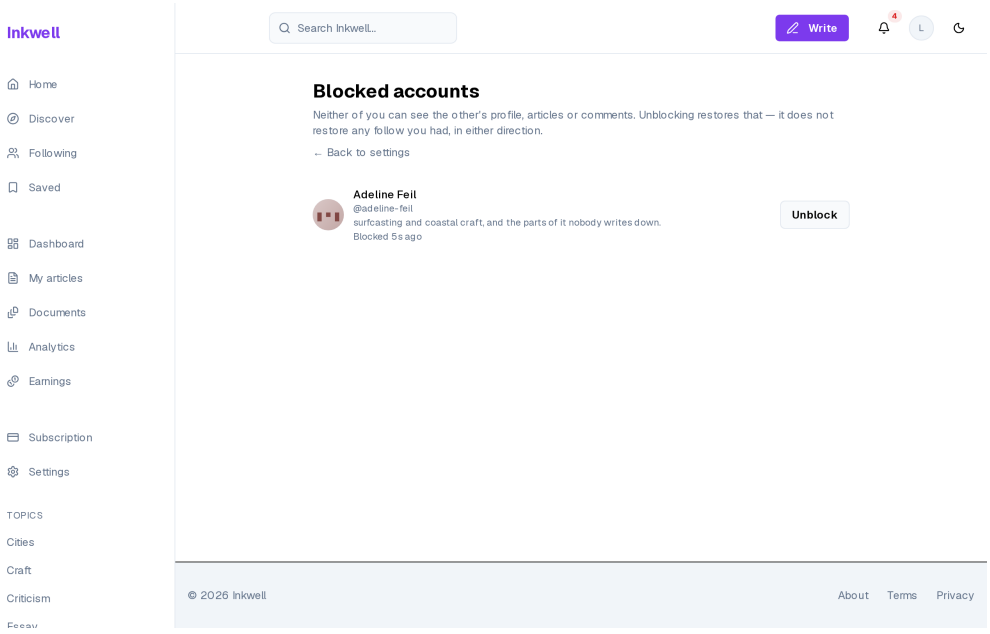


Figure 6.4 – Blocked accounts, in settings

Blocks are listed in one place, each with *Unblock* (FR-62), under a sentence that states the rule: neither account sees the other's profile, articles or comments, and unblocking does not restore a previous follow. Putting the list in settings rather than on profiles follows from the analysis: a block is a private decision, so it is reviewed privately.

Figure 6.5 shows the feed restricted to followed accounts, and Figure 6.6 a follower list being searched.

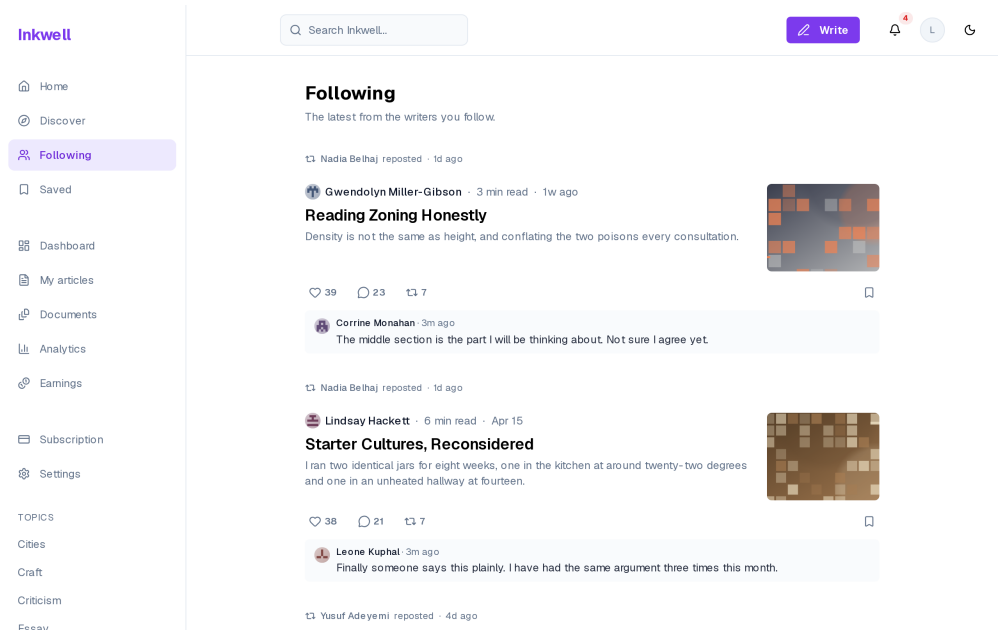


Figure 6.5 – The following feed

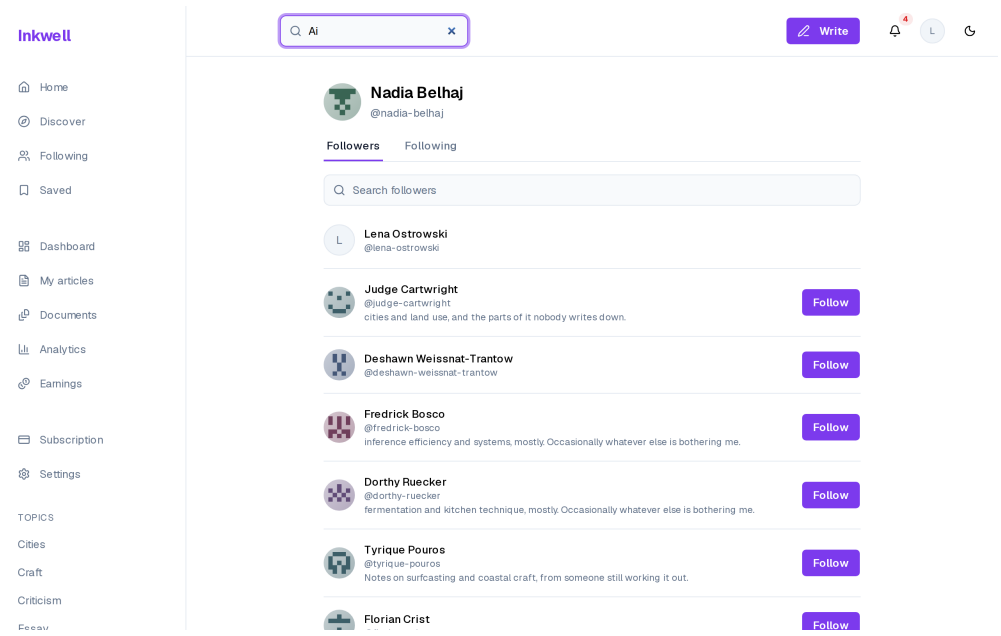


Figure 6.6 – A follower list, searched

The following feed shows only the writers a reader follows, and the follower lists can be browsed, searched and acted on, including following back from the list itself (FR-61). Both turn the follow relationship of Sprint 3 into a way to discover people through people.

6.2 Sprint 8: Live cards, the assistant that writes, and voice

6.2.1 Sprint backlog

The goal of Sprint 8 is that readers react, repost and comment from the feed card, see recent comments and preview any byline; and that the assistant decides whether to answer or to write into the document, writes

into a protected range the writer keeps or discards, shows its steps as they run, sits in a panel docked beside the editor, accepts dictated prompts and reads its replies aloud. Table 6.2 lists the stories.

Table 6.2 – Sprint 8 backlog

ID	User story (goal)	Priority	Points
US-59	React to an article from the feed	Must	5
US-60	See what an account has reposted	Should	3
US-61	Reply in the moment from the feed	Could	3
US-62	Keep reading as I scroll	Must	5
US-65	See who a writer or magazine is from where their name appears	Should	3
US-66	Tell the assistant what to write and watch it appear in my draft, then keep or discard it	Must	8
US-67	See what the assistant is doing while it works, and minimize it without losing sight of that	Should	3
US-68	Say my request instead of typing it	Should	5
US-69	Hear the assistant’s recap read out	Could	3
	9 stories		**38**

The sprint commits 38 points in two halves of similar size: the live feed card and its neighbours on one side, the reworked assistant and voice on the other. The largest story, US-66 (8 points), is the write into the document, which needed a new editor extension as well as a new kind of model call.

6.2.2 Analysis

Figure 6.7 gives the use cases of Sprint 8.

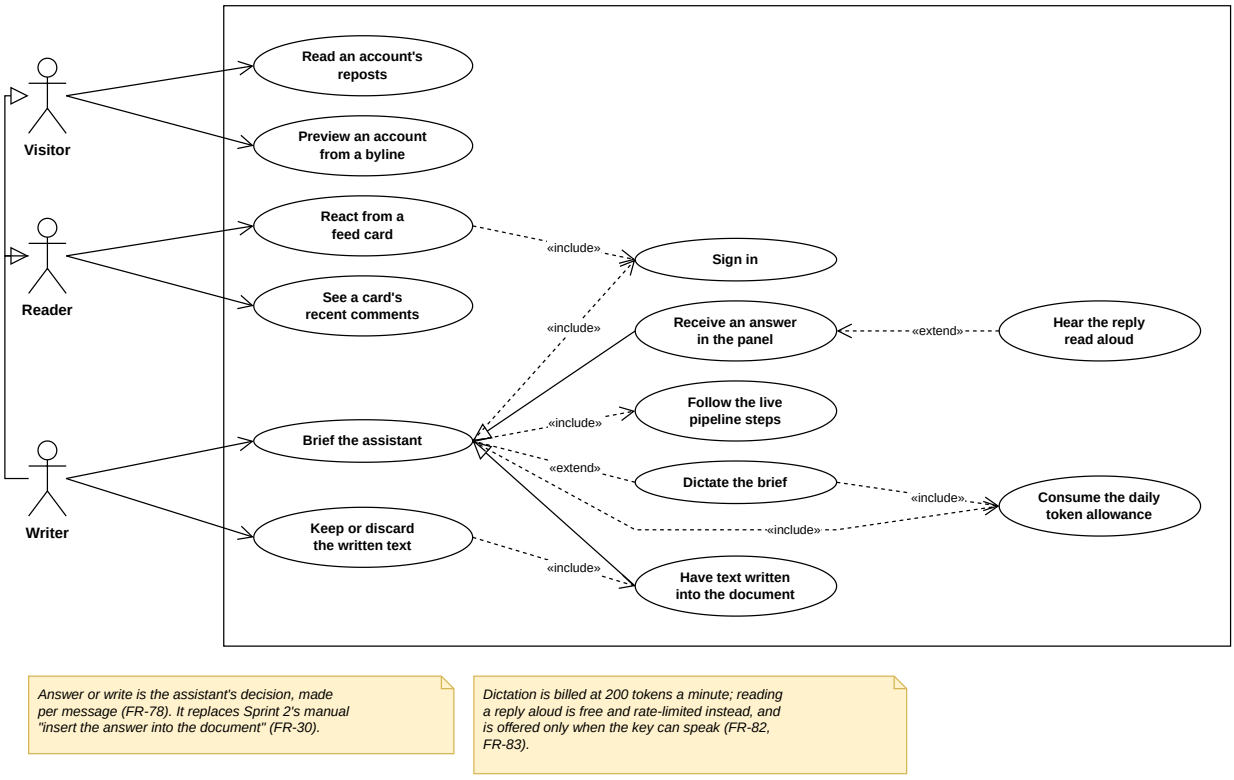


Figure 6.7 – Use case diagram, Sprint 8

Briefing the assistant specialises into an answer or a write into the document, and includes the live steps and the allowance; dictation and replies read aloud extend it. The assistant chooses per message, replacing the manual insert of Sprint 2 (FR-78 supersedes FR-30).

6.2.3 Design

Sequence diagram: one assistant turn. Figure 6.8 shows a turn.

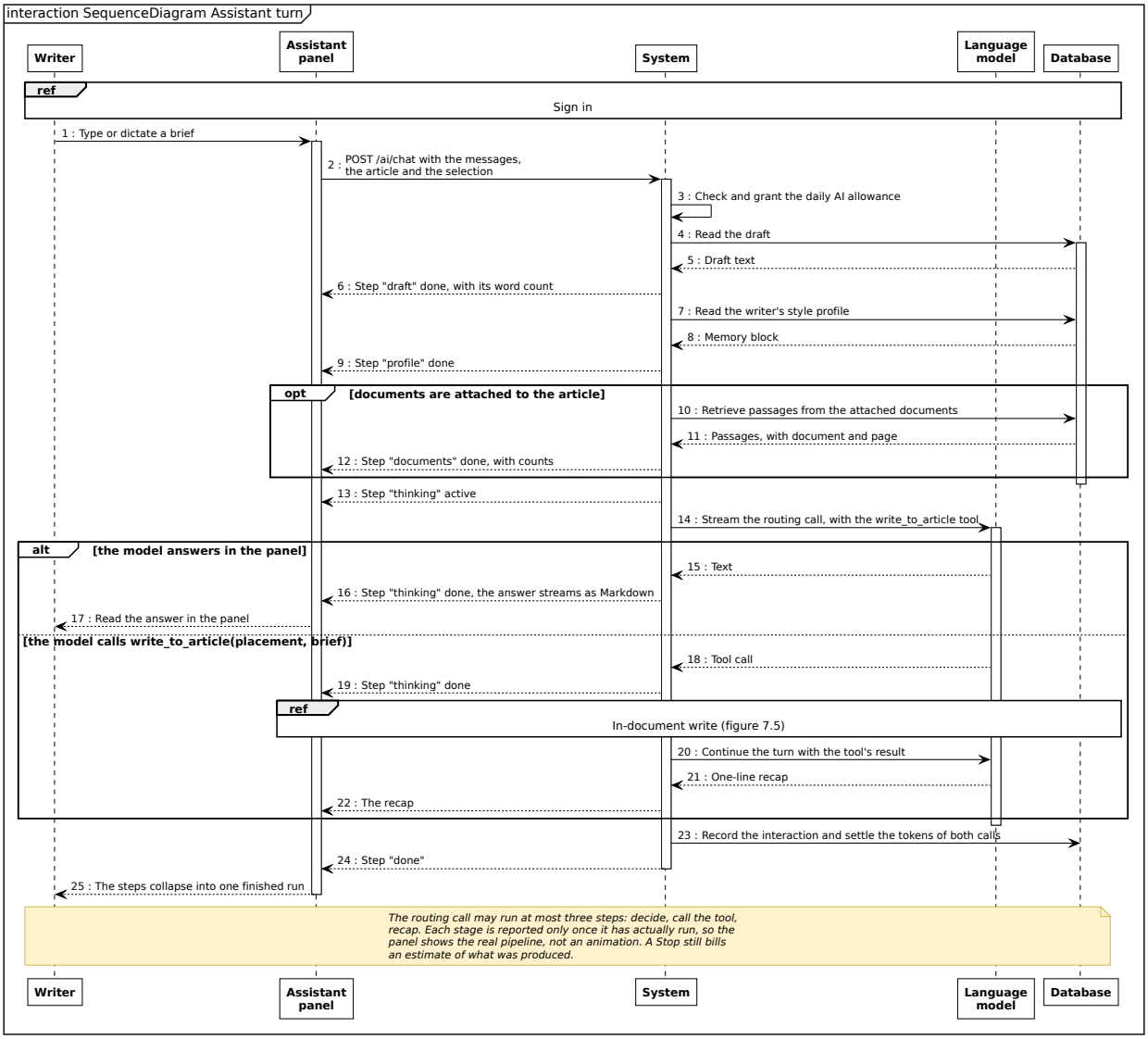


Figure 6.8 – Sequence diagram: one assistant turn

The system reads the draft, the voice profile and any documents, reporting each stage. One routing call answers in the panel or calls `write_to_article`, then adds a recap; tokens are settled at the end. The panel shows real steps because each stage is part of the response stream (FR-80).

Sequence diagram: writing into the document. Figure 6.9 shows the write.

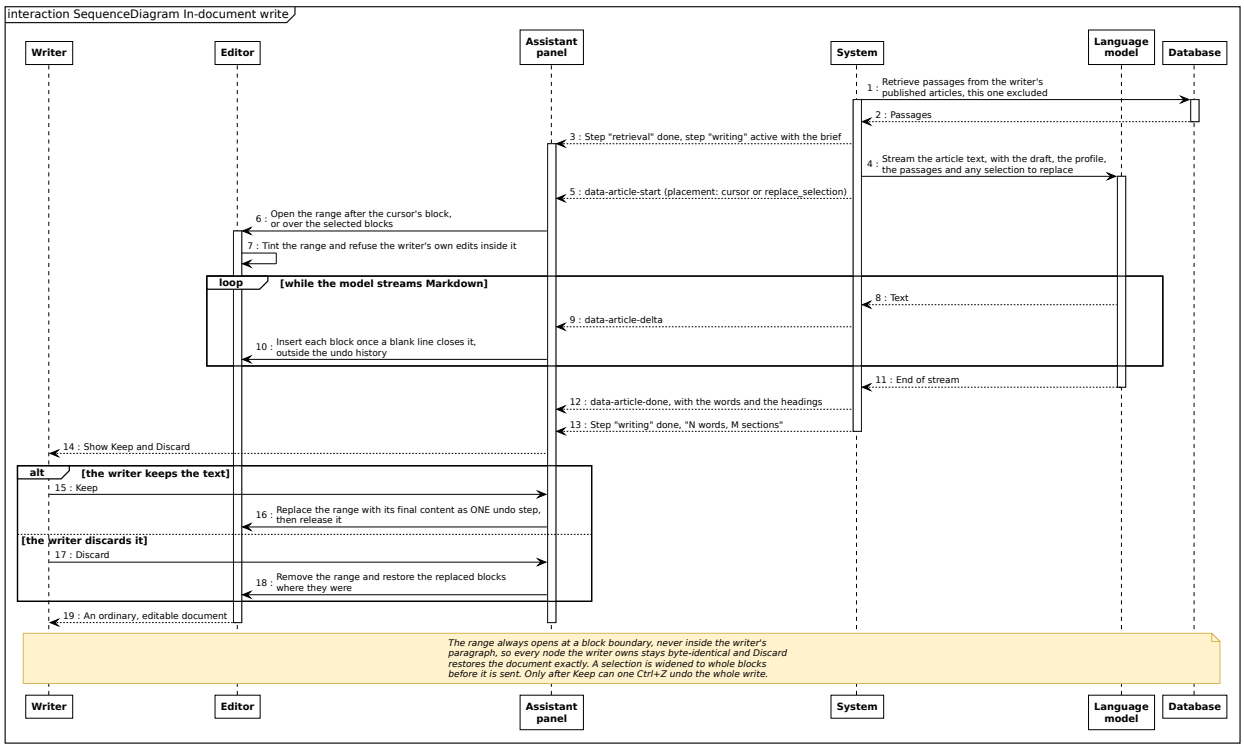


Figure 6.9 – Sequence diagram: writing into the document

The text streams into a protected range opened at a block boundary, block by block. The writer then keeps it, as one undo step, or discards it, restoring exactly what was there. It is exact because the range starts at a block boundary and touches no node the writer owns.

Sequence diagram: voice. Figure 6.10 shows the voice round trip.

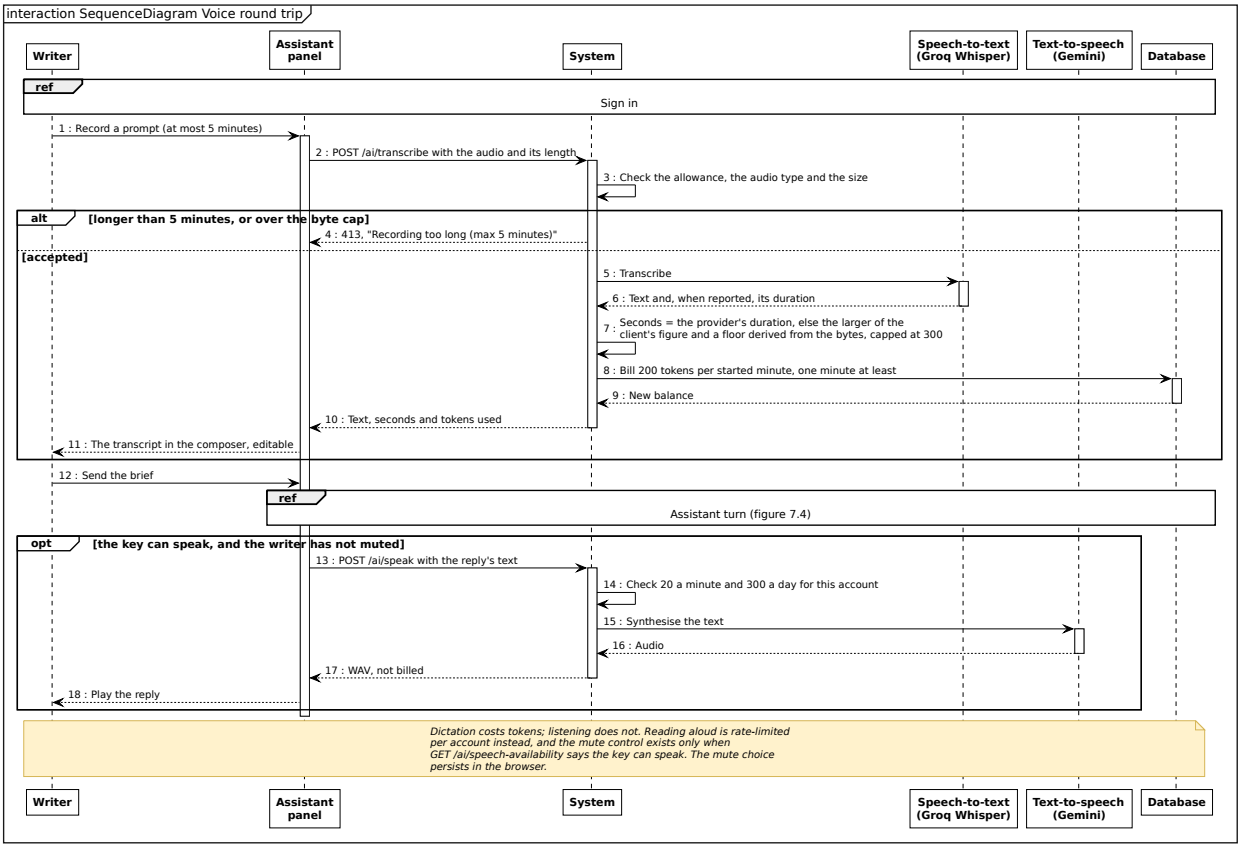


Figure 6.10 – Sequence diagram: the voice round trip

A recording of up to five minutes is transcribed with Whisper and billed 200 tokens per started minute, the transcript landing in the composer. The duration billed is the provider's, or the larger of the client's figure and a floor from the file size: the client's claim is never trusted alone.

6.2.4 Realisation

Figure 6.11 shows the Reposted tab of a profile.

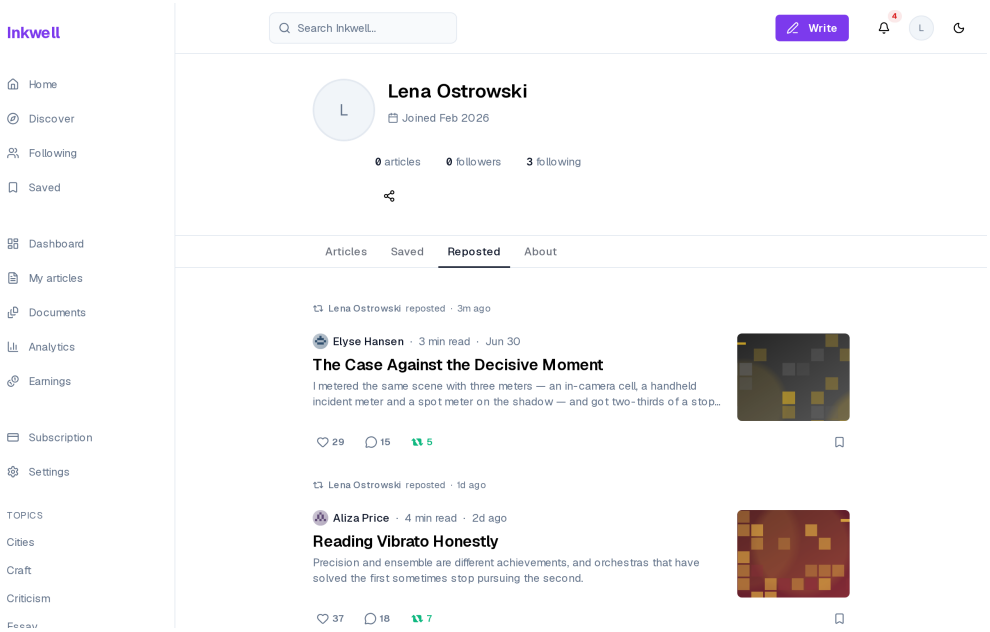


Figure 6.11 – The Reposted tab of a profile

Any account's reposts can be read from its profile, without an account (FR-67): what a person passes on is part of how others judge their taste.

Figure 6.12 shows a write awaiting the writer's decision.

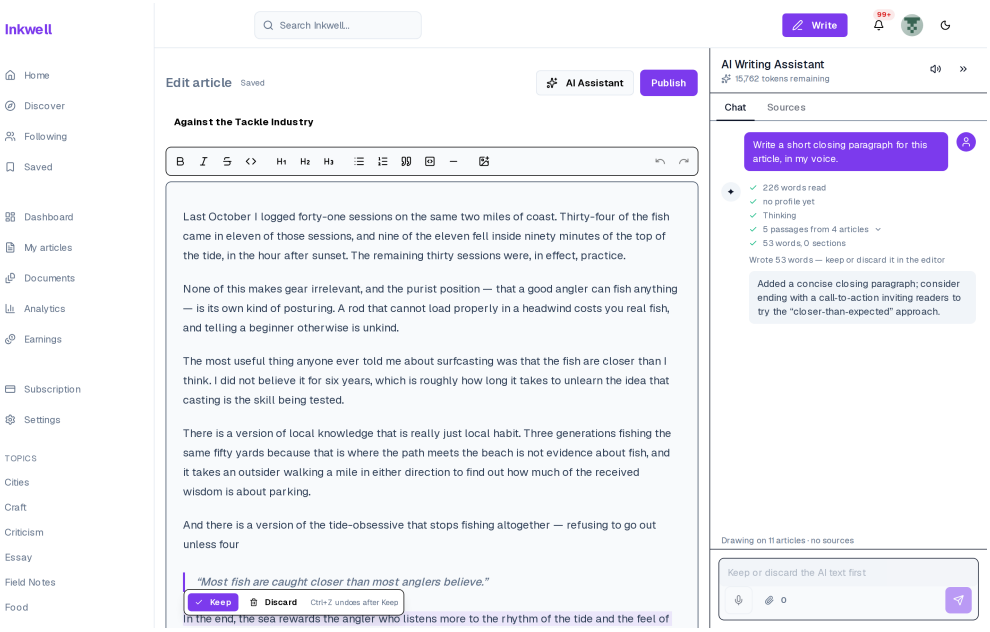


Figure 6.12 – A write awaiting Keep or Discard

Asked for a closing paragraph "in my voice", the assistant chose to write (FR-78): the panel lists its steps and a recap, and the tinted paragraph offers *Keep* and *Discard* (FR-79, FR-80). The composer stays disabled until the writer decides.

Figure 6.13 shows the live step on the collapsed panel.

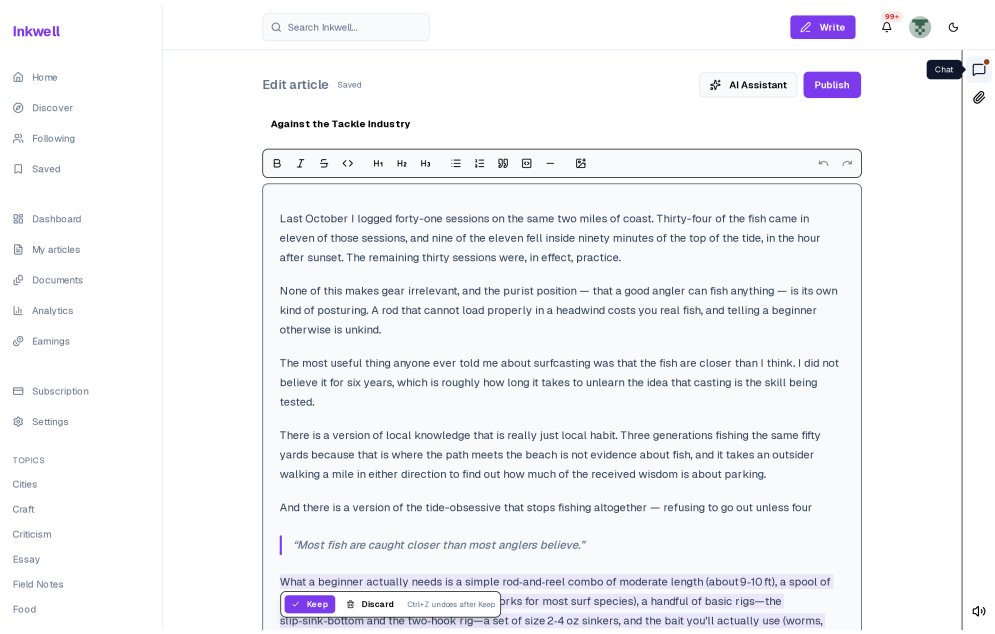


Figure 6.13 – The live step on the collapsed panel

With the panel collapsed to a rail, the editor takes the full width and a write still streams into the document; the rail keeps the chat, the sources and the voice controls one click away, and its marker signals activity (FR-81). A slow step is therefore never mistaken for a frozen screen, as US-67 requires.

Figure 6.14 shows a prompt being recorded.

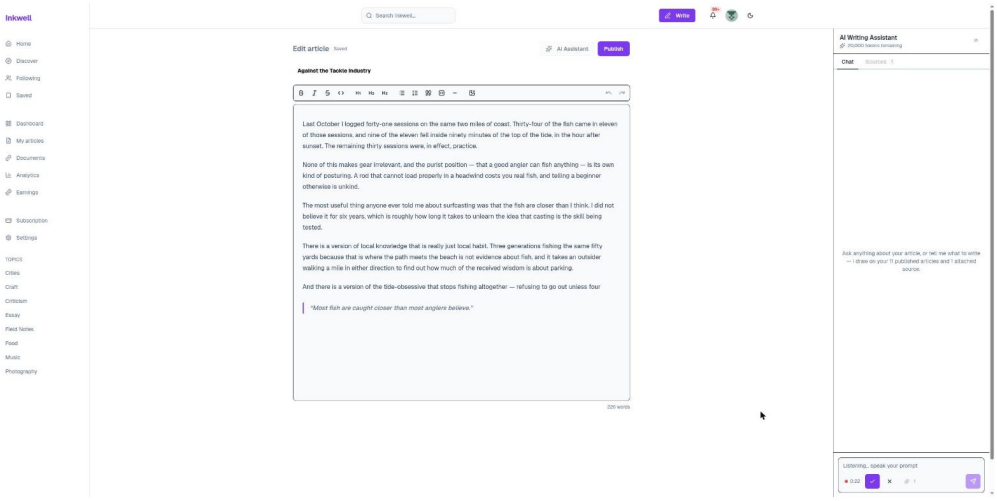


Figure 6.14 – Recording a prompt

While recording, the composer shows *Listening… speak your prompt*, the elapsed time and controls to confirm or cancel; the transcript then fills the composer, editable (FR-82). Dictation uses the same allowance as typing: voice is a way to write a prompt.

6.3 Sprint 9: Document library and citations

6.3.1 Sprint backlog

The goal of Sprint 9 is that a writer uploads reference documents into a private library, attaches them to an article, and that the assistant draws on them for answers and writes, citing each passage by document and page. Table 6.3 lists the stories.

Table 6.3 – Sprint 9 backlog

ID	User story (goal)	Priority	Points
US-70	Upload the documents I am working from	Must	8
US-71	Choose which documents apply to an article	Must	3
US-72	See which document and page a claim came from	Should	5
	3 stories		**16**

The sprint commits 16 points, the lightest load of the project after Sprint 7: the ingestion reuses the chunking and embedding pipeline of Sprint 4, so most of the work is the extraction of text page by page and the citation format. It ran alongside the end of the writing of this report.

6.3.2 Analysis

Figure 6.15 gives the use cases of Sprint 9.

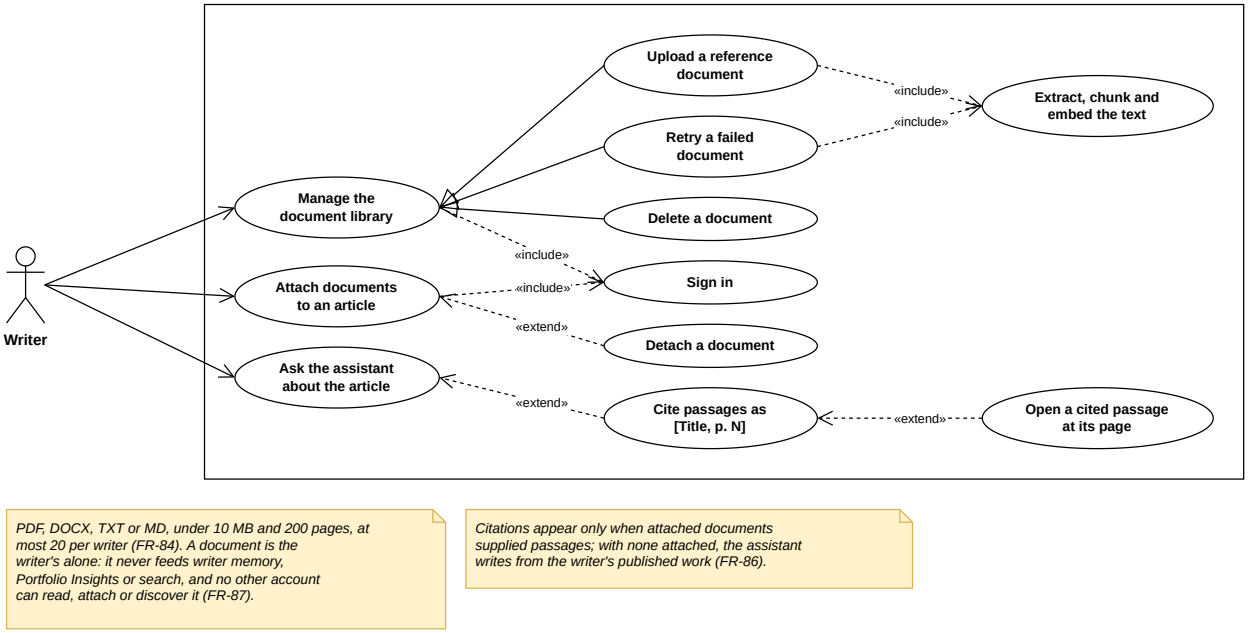


Figure 6.15 – Use case diagram, Sprint 9

Managing the library specialises into uploading, retrying and deleting; uploading includes extracting, chunking and embedding. Attaching is extended by detaching, and asking the assistant by citing *[Title, p. N]*. Documents are PDF, DOCX, TXT or Markdown, under 10 MB and 200 pages, 20 per writer. A document is private: it never feeds the voice profile, insights or search (FR-87).

6.3.3 Design

Class diagram. Figure 6.16 shows the library.

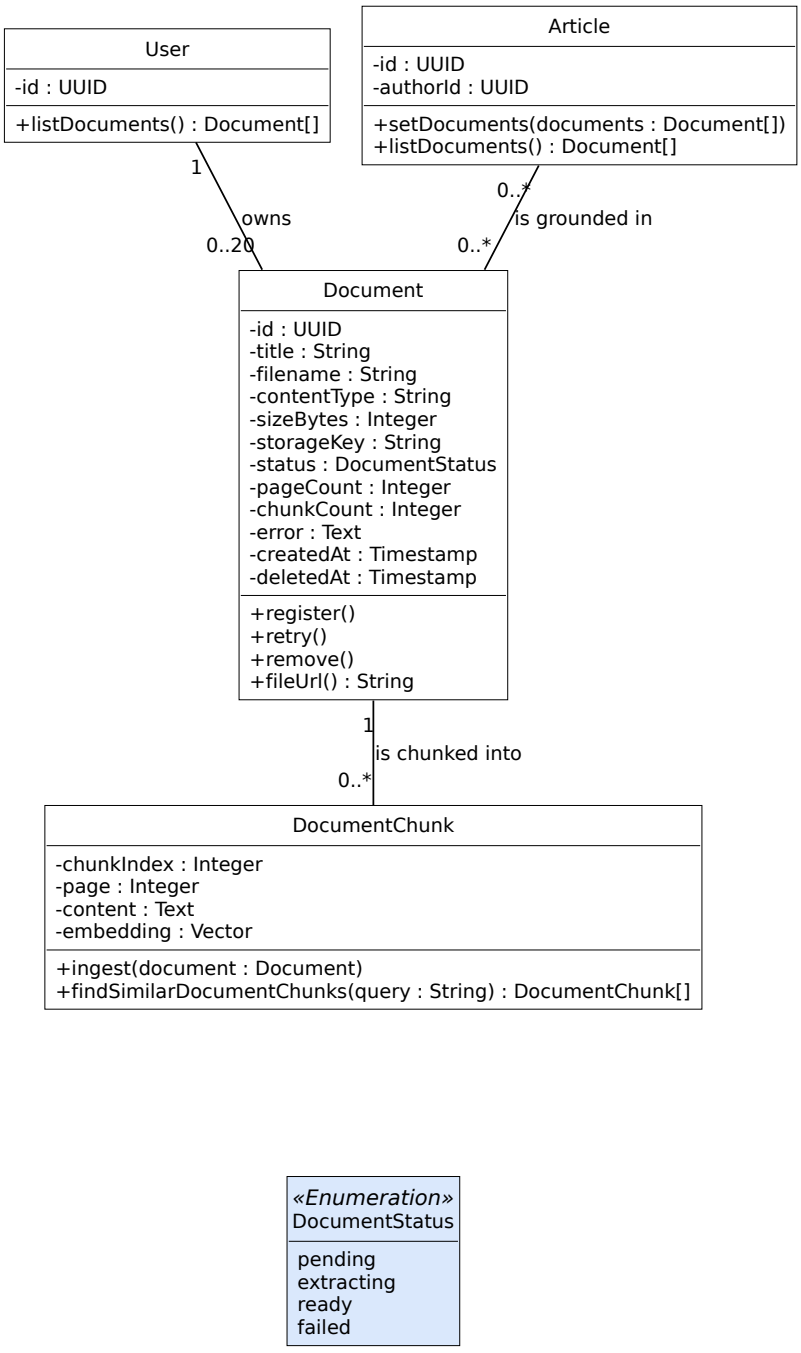


Figure 6.16 – Class diagram, Sprint 9

A `Document` carries its status, counts and a readable error; each `DocumentChunk` keeps its page, and documents attach to articles many-to-many. The status only moves forward, except a retry. A citation is a property of the data: the page travels with the chunk to the prompt.

Sequence diagram: ingestion. Figure 6.17 shows a document's path, the companion of the publishing pipeline of Figure 4.14.

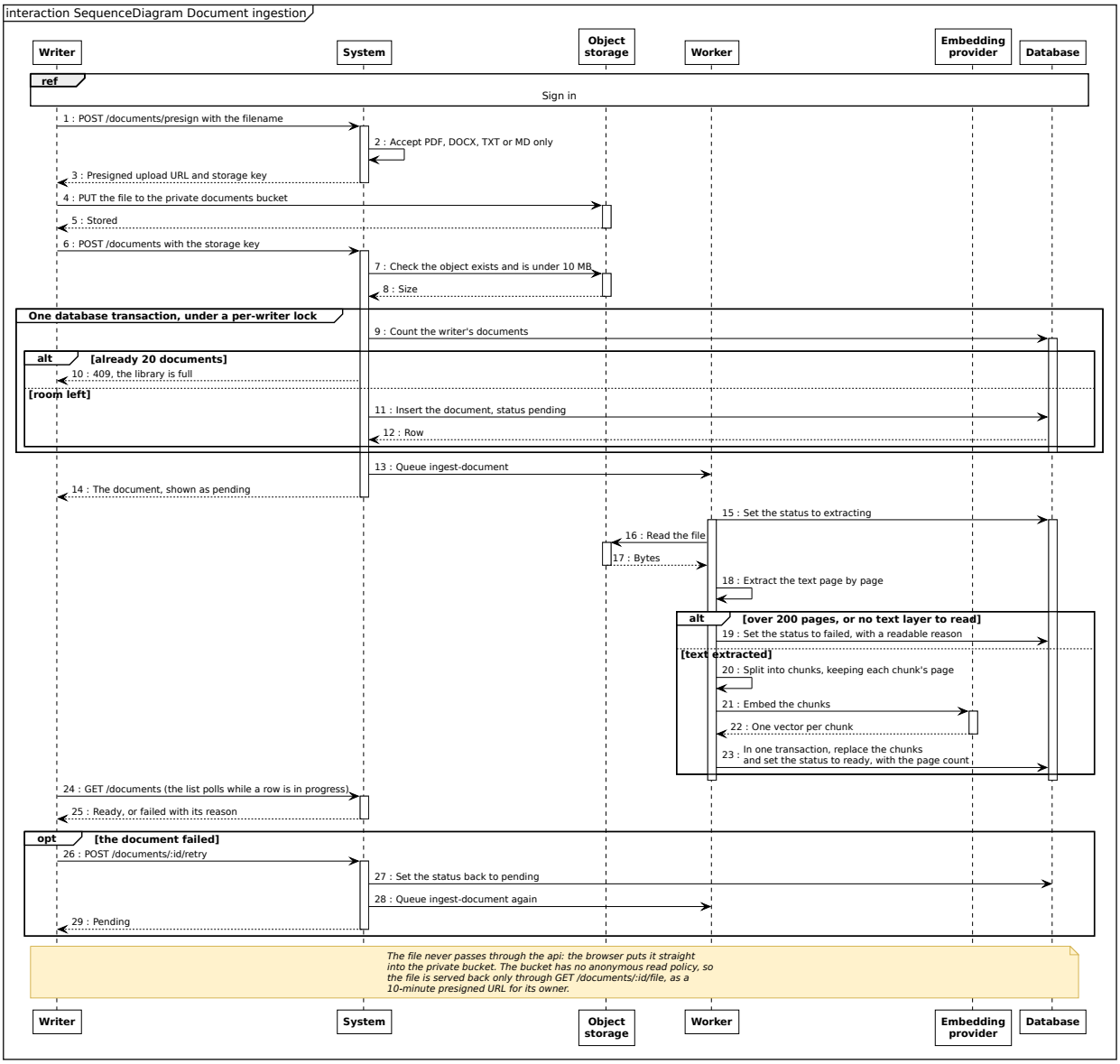


Figure 6.17 – Sequence diagram: document upload and ingestion

The file goes straight to a private bucket through a presigned URL, never through the API. Registration checks it and the 20-document limit; the worker extracts the text page by page and chunks, embeds and marks it ready in one transaction, or refuses it with a reason. No document is half usable.

6.3.4 Realisation

Figure 6.18 shows the document library.

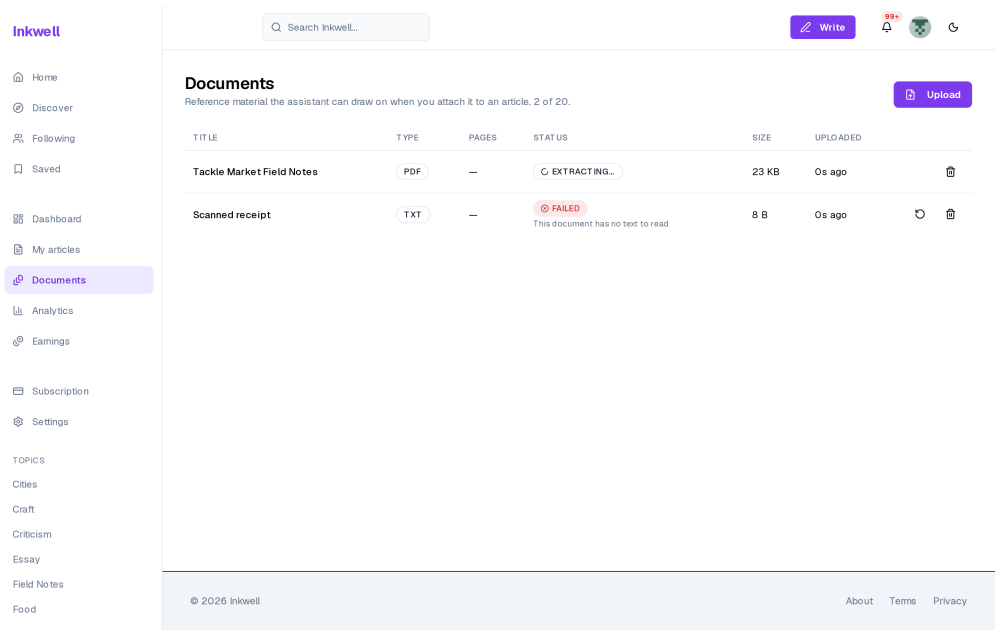


Figure 6.18 – The document library, one document refused with its reason

The library lists each document's type, pages, status, size and upload time against the limit of 20 (FR-84). One document is extracting; the other is refused ("This document has no text to read") with a retry: failures are stated in words the writer can act on.

Figure 6.19 shows the attachment of a document from the Sources tab of the assistant.

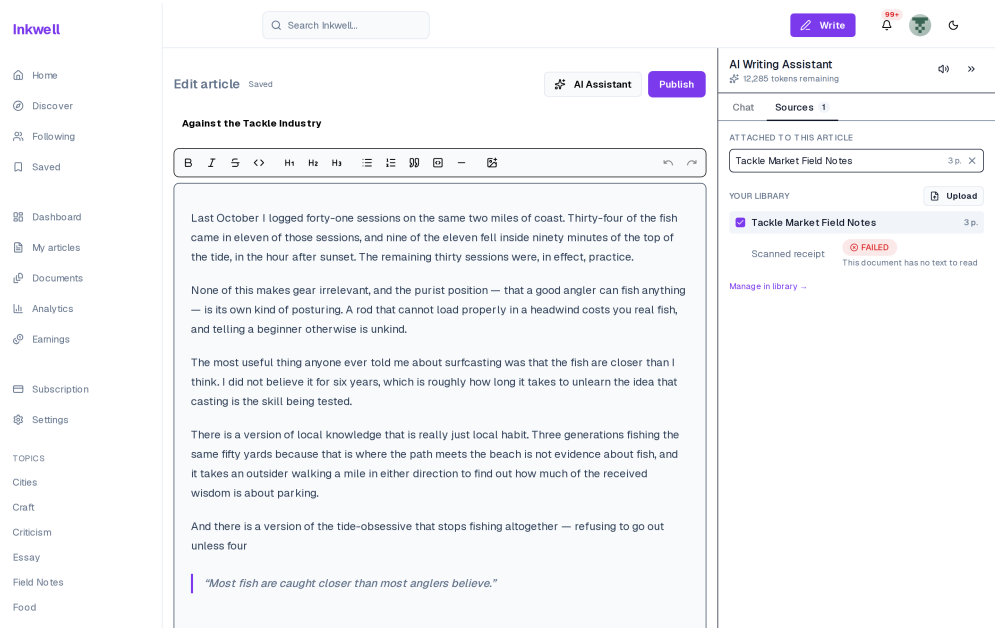


Figure 6.19 – Attaching a document from the Sources tab

The tab separates the documents attached to this article from the writer's whole library, where a checkbox attaches or detaches each ready document and a failed one is shown with its reason; the attachment is saved immediately (FR-85). Attaching per article is what keeps the assistant's material relevant: notes for one piece are not fed into another.

Figure 6.20 shows a reply citing its source.

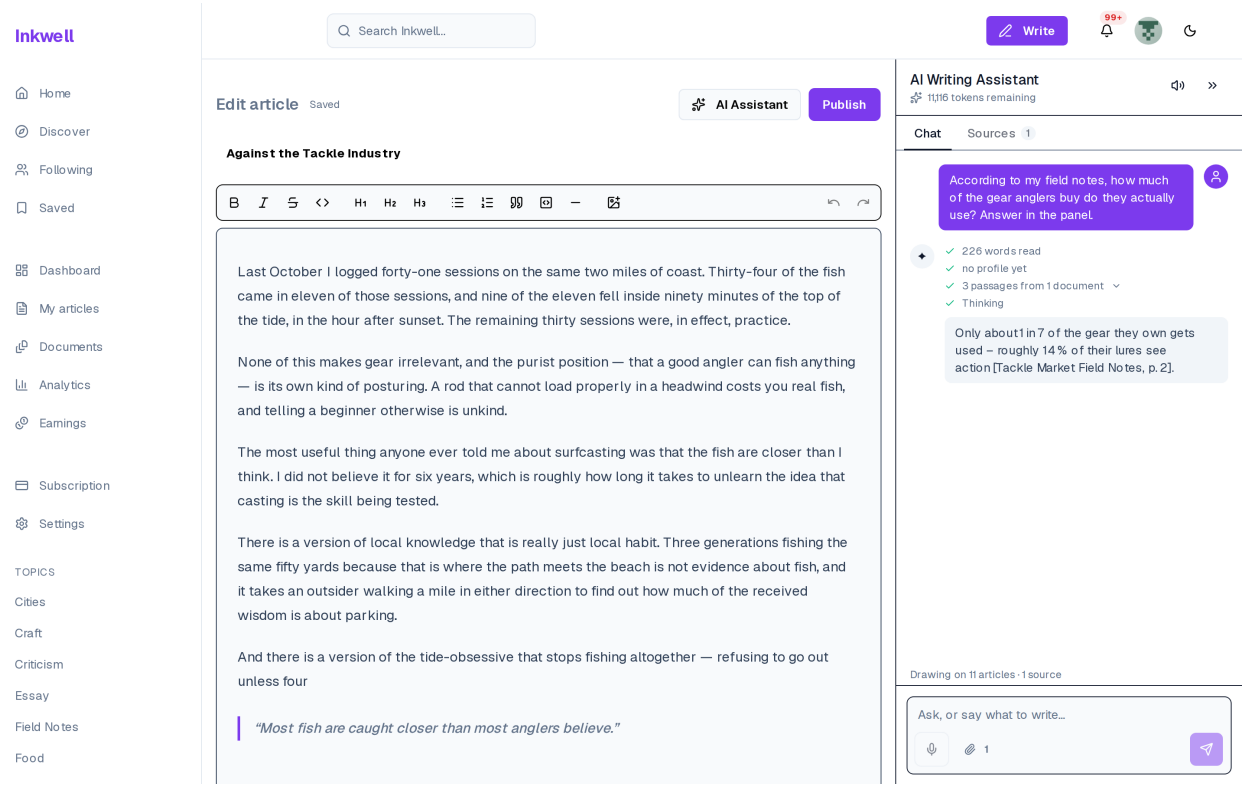


Figure 6.20 – A reply citing its source as [Title, p. N]

Asked a factual question, the assistant retrieved three passages from one document and answered with the figure and its source, *[Tackle Market Field Notes, p. 2]* (FR-86). The writer can check a claim before it goes out under their name, the purpose of US-72.

Conclusion

Release 4 completed the social layer, turned the assistant from a chat beside the editor into a collaborator that writes into the document under the writer’s control, added voice in both directions, and gave the writer a private library of sources the assistant can cite to the page. The next chapter evaluates the result.

CHAPTER 7

Evaluation

Introduction

The release chapters showed what the platform does; this chapter measures how well it does it. It evaluates the three components whose quality cannot be read off a screen (the retrieval behind the assistant, the hybrid search and the moderation classifier), then the performance and accessibility of the platform, and finally the delivery itself: the velocity of the sprints against the plan, and a retrospective of each release. Every measurement was taken on the workstation of Table 2.1, with the full stack running in Docker, and every script and result is kept in the backend repository (`eval/`) so that it can be repeated.

7.1 Evaluation protocol

Corpus. Retrieval and search were evaluated on the demonstration corpus of Section 5.2, restricted to six writers, one for each of the corpus's six subjects (coastal craft, inference systems, cities, fermentation, orchestral music, analogue photography): **56 published articles split into 1,202 embedded passages**. The restriction is imposed by the embedding provider's free tier, which allows about a thousand embeddings a day; it keeps the evaluation within one day's quota.

Questions. The evaluation set is a *known-item* set: 30 target passages chosen from the corpus, five per subject, each asked twice: once with the passage's own distinctive words (the *keyword* query) and once as a paraphrase that avoids them (the *paraphrase* query). A passage or an article is relevant when it contains the target passage. Three targets turned out to appear in none of their writer's embedded articles and were dropped, leaving **27 targets and 54 queries**; because the seeded corpus reuses paragraphs across a writer's articles, a target appears on average in 5.9 articles. The queries were written before any result was seen.

Metrics. The standard measures of ranked retrieval were used [42], [43], [44]:

- **hit@k**: the share of queries for which at least one relevant item is among the first k results;
- **MRR** (mean reciprocal rank): the mean of $1/r$, where r is the rank of the first relevant result (0 when there is none); 1.00 means the first result is always relevant;
- **precision@5**: the share of relevant items among the passages returned;
- **recall@10**: the share of the relevant articles found in the first ten results;
- **nDCG@10**: a rank-weighted score that rewards relevant results placed higher.

The evaluation reuses the production code: queries are embedded by the platform's own embedding service, and the rankings are the SQL of the retrieval and search services. As a check, the production retrieval call was run on every query and returned the same passages as the evaluation in 54 cases out of 54.

7.2 Retrieval behind the assistant

The assistant retrieves the five passages of the writer's own work closest to the question, provided they score above a similarity floor (Section 4.2). Two questions matter: does the right passage come back, and does the floor correctly return nothing when the question is off-topic? The second was measured by asking every

query to a writer of *another* subject, for whom the correct answer is no passage at all. Table 7.1 gives the results for floors from 0.50 to 0.70.

Table 7.1 – Retrieval of the writer's own passages, by similarity floor (54 queries)

Floor	Hit@5, keyword	Hit@5, paraphrase	Hit@5, all	MRR	Precision@5	Passages returned	Off-topic refused
0.50	0.96	0.81	0.89	0.79	0.59	5.0	0%
0.55	0.96	0.81	0.89	0.79	0.59	5.0	7%
0.60	**0.96**	**0.81**	**0.89**	**0.79**	**0.59**	**5.0**	**69%**
0.65	0.96	0.81	0.89	0.79	0.60	4.9	98%
0.70	0.93	0.74	0.83	0.75	0.64	4.1	100%

At the floor used in production (0.60), a relevant passage is among the five returned for 89% of the questions (96% when the question uses the passage's words, 81% when it paraphrases them), and it is usually the first one (MRR 0.79). The floor is the reason off-topic questions are refused: at 0.60, 69% of the questions asked to the wrong writer retrieve nothing, where a floor of 0.50 would have fed five irrelevant passages to the model every time.

The table also shows that 0.60 is not the best value. Raising the floor to **0.65** keeps every hit and refuses 98% of off-topic questions; only at 0.70 do relevant passages start to be lost. The similarities explain why: the best relevant passage scored between 0.66 and 0.84, and the best off-topic passage between 0.52 and 0.66, so the two ranges barely touch at 0.65. The measurement therefore supports raising the floor from 0.60 to 0.65, which is listed among the perspectives. Precision stays near 0.6 at every floor because a target appears in about six articles but only some of them are among the five passages returned; it is not a sign of irrelevant results.

7.3 Hybrid search

Search fuses a lexical ranking and a semantic ranking with reciprocal rank fusion (Section 4.2). Table 7.2 compares the two rankings alone and fused, with the production constant k = 60 and two other values, on the same 54 queries over the 56 articles.

Table 7.2 – Lexical, semantic and hybrid search (54 queries, 56 articles)

Ranking	Recall@10, keyword	Recall@10, paraphrase	MRR, keyword	MRR, paraphrase	nDCG@10, all
Lexical only (`ts_rank`)	0.95	0.06	0.93	0.07	0.50
Semantic only (embeddings)	1.00	1.00	0.94	0.90	0.94
Hybrid, RRF k = 10	1.00	1.00	0.98	0.90	0.95
Hybrid, RRF k = 60	**1.00**	**1.00**	**1.00**	**0.90**	**0.95**
Hybrid, RRF k = 100	1.00	1.00	1.00	0.90	0.95

The table justifies the hybrid design. Keyword search is excellent when the searcher uses the article's words (recall 0.95) and nearly useless when they do not (0.06): a paraphrase defeats it. Semantic search finds every relevant article in both cases. The fusion keeps the semantic recall and improves the ranking of keyword queries: with k = 60 the first result is relevant for every keyword query (MRR 1.00 against 0.94 for semantic

search alone), because an article that matches both by words and by meaning is ranked first. The value k = 60 recommended by the method's authors [39] is confirmed: a smaller constant is slightly worse, a larger one changes nothing.

The limits of this evaluation must be stated. The corpus is small and synthetic, built by a seed that reuses paragraphs; the queries were written by the author, who knew the corpus; and a known-item set rewards finding one passage, not judging the usefulness of an answer. The results show that retrieval and fusion behave as designed on this corpus, not how they would score on a public benchmark.

Search latency. On the same queries, the stages of a search took a median of 263 ms to embed the query, 7 ms for the lexical ranking and 9 ms for the semantic ranking: about 0.3 s in total, dominated by the call to the embedding provider. One behaviour was also measured that the design did not intend. When the provider's daily quota is exhausted, each search spends about **6.3 s** retrying the embedding three times before falling back to keyword search. The fallback works, since the results are still returned, but the retries make it slow; failing fast once the quota is known to be exhausted, and caching the embeddings of frequent queries, are listed among the perspectives.

7.4 Moderation classifier

The classifier reviews every public article after publication and files a report when it judges the content to violate one of six categories (Section 5.1). It was evaluated on a labelled set of 33 texts written for the purpose: **13 violating texts** across the six categories, and **20 safe texts**, of which twelve are *hard negatives* (war reporting, crime news, an addiction memoir, thriller fiction, a suicide-prevention message, instructions on overdose treatment) that a naive classifier would flag because of their subject. The production service was called on each text three times.

Table 7.3 – Confusion matrix of the moderation classifier (identical in the three runs)

	Classified violating	**Classified safe**
Violating (13)	13 (true positives)	0 (false negatives)
Safe (20)	1 (false positive)	19 (true negatives)

The classifier reaches a **precision of 0.93** and a **recall of 1.00** (F1 0.96), and named the right category for all 13 violating texts; the three runs gave identical results. Its single false positive is instructive: instructions for killing a landed fish quickly, classified as violence. It is a hard negative about hunting, the kind of text the instruction to judge content rather than subject is meant to pass. The policy chosen in Section 5.1 absorbs exactly this kind of error: a false positive files a report that an administrator dismisses, and never blocks the article.

The median classification took 2.2 s, which is acceptable because it runs after the publication has been answered. The evaluation also had a second outcome. Its first run showed that the model initially configured for the classifier had been withdrawn by the provider, so that every call failed and the service, which fails open by design, passed every article without a word in the logs. The classifier was moved to `gpt-oss-20b`, with an output budget large enough for a reasoning model, and the results above are those of the corrected service. The lesson is that a fail-open component needs a liveness check of its own; the daily model check of the worker now exercises the classifier at the budget it is called with.

7.5 Performance and accessibility

API latency. Table 7.4 gives the response time of the main read routes, measured inside the Compose network with ten concurrent clients and 300 requests per route, each client presenting its own address so that the per-address rate limit applied as it would to distinct readers.

Table 7.4 – Latency of the main API routes under concurrent load

Route	Median	95th percentile	Throughput
Public feed (`GET /articles`, 20 cards)	34 ms	41 ms	285 req/s
Article page (`GET /articles/:slug`)	15 ms	19 ms	652 req/s
Writer evaluation (`GET /writers/:username/evaluation`)	16 ms	20 ms	620 req/s
Writer dashboard (`GET /me/analytics`)	12 ms	17 ms	774 req/s
Search (stages, one query at a time)	≈ 0.3 s	n/a	bound by the provider quota

Every route that reads the database answers in under 50 ms at the 95th percentile on a laptop, and the two routes backed by the rollups (the evaluation report and the dashboard) are among the fastest, which is the effect NFR-15 was written for. Search is the exception, for the reason given in Section 7.3: its cost is the call to the embedding provider, not the database. The rate limit of NFR-05 was verified on the same occasion: sixty requests from one address within a minute were served, and the sixty-first was answered 429.

Lighthouse and accessibility. NFR-37 asks for a Lighthouse score above 90 and no critical accessibility violation on article pages [45]. A production build of the frontend was audited behind nginx, with Lighthouse's mobile and desktop presets (median of three runs) and with axe-core on the WCAG 2.x A and AA rules at the two designed widths [46], [47]. Table 7.5 gives the results.

Table 7.5 – Lighthouse scores and axe-core violations of the public pages

Page	Performance (mobile / desktop)	Accessibility	SEO	Critical axe violations	Serious axe violations
Article, free	63 / 96	92	100	0	6
Article, premium (paywalled)	74 / 98	91	100	0	5
Home feed	71 / 98	91	100	0	19
Search results	75 / 93	94	63	0	4

NFR-37 is **partly met**. No page has a critical accessibility violation, at either width; every page scores above 90 for accessibility, and between 93 and 98 for performance on desktop. On the mobile preset, which simulates a slow device and network, the pages score 63 to 75: their largest element is painted after about six seconds, because the page loads its fonts, cover image and interactive controls before the text settles. The serious violations are of two kinds only: text whose contrast with its background is too low, and icon-only links without an accessible name (the reaction and share buttons of the cards, which explains the higher count on the feed). Both are fixes of the design tokens and of a few components, and they are listed with the limitations.

The search page's SEO score of 63 is intended: result pages are marked `noindex`, so that search engines index the articles rather than lists of them.

7.6 Test coverage

Table 5.4 in Section 5.2 gave the test suites; their coverage was interpreted there. In short, the 796 backend tests execute 48.5% of the backend's lines, concentrated on the rules that move money or remove content (moderation 92%, purchases 77%), while the analytics aggregation is verified by the measurements above rather than by unit tests.

7.7 Delivery: velocity and retrospective

Definition of done. A story counted as delivered when its code had been reviewed against the specification, the type check and the linter passed with no warning, the backend tests passed in the container, the feature had been demonstrated in a browser, and the specification had been updated to match what was built.

Velocity. Table 7.6 compares the points committed in each sprint with the points delivered under that definition.

Table 7.6 – Committed and delivered points per sprint

Sprint	Committed	Delivered	Gap
Sprint 1	45	45	
Sprint 2	45	45	
Sprint 3	44	44	
Sprint 4	52	52	
Sprint 5	85	85	
Sprint 6	20	12	US-55 (8 points): configuration ready, server not provisioned
Sprint 7	18	18	
Sprint 8	38	38	
Sprint 9	16	16	
Total	**350**	**342**	98% of the committed points

The plan was followed sprint for sprint: 98% of the points were delivered, and the only gap is the public hosting of US-55. The table shows, however, that the loads were uneven, from 16 to 85 points for sprints of the same length. That spread is a consequence of estimating alone: without a team to challenge a figure, the marketplace of Sprint 5 was planned as one block because its stories share one ledger, and the light sprints of Release 4 reflect stories that reused existing pipelines and a release that ran alongside the writing of this report. A velocity averaged over the project (about 38 points per sprint) would have been the better planning figure, and would have split Sprint 5 in two.

Retrospective. Table 7.7 summarises what each release taught.

Table 7.7 – Retrospective by release

Release	What worked	What to improve	Action taken
R1	The ledger checks and the one-command stack existed before any feature that needed them	The editor waited six weeks for the design track	Later releases built on the design system and needed no separate design track

Release	What worked	What to improve	Action taken
R2	Rollups kept dashboards fast as events accumulated	The embedding provider’s quota limits backfills and search under load	Embed one writer at a time; a query-embedding cache listed as a perspective
R3	Every money rule is tested, including tests designed to fail under sabotage	Sprint 5 was overloaded; the aggregation has no unit tests; hosting was not done	Plan by measured velocity; aggregation tests listed as the first perspective
R4	Voice and documents reused the AI pipeline, so they were small	A provider withdrew a model and the fail-open classifier hid it	A daily check exercises every model as it is called

The retrospective shows a pattern. The rules written down early (the ledger invariants, the access decision, the definition of done) held for the whole project; what went wrong came from outside the code: the quotas and model changes of the AI providers, and the time an individual project leaves for hosting. Those are the points the perspectives address.

Conclusion

The evaluation confirmed the main design decisions with measurements: retrieval finds a relevant passage for 89% of questions and its floor refuses most off-topic ones; hybrid search keeps all relevant articles even when the searcher’s words differ from the author’s; the moderation classifier reaches a precision of 0.93 and a recall of 1.00; the API answers in tens of milliseconds; and 98% of the planned points were delivered. It also found what remains to be done: a higher similarity floor, a faster fallback when the embedding quota is exhausted, better mobile performance and contrast, tests for the aggregation, and the public hosting of the platform.

General conclusion and perspectives

This project set out to connect two groups the existing platforms leave apart: independent writers, who lack an assistant that writes in their own voice and a direct way to be paid for quality, and magazines, which lack objective evidence to judge a writer before buying. Inkwell.ai answers both with one platform.

Over nine two-week sprints in four releases, the project delivered:

- a **writing environment**: a rich-text editor with automatic saving and images, and an AI assistant grounded in the writer's own published work through retrieval-augmented generation, which decides per request whether to answer or to write into the document, writes into a protected range the writer keeps or discards, shows its real pipeline as it runs, accepts dictated prompts, reads its replies aloud, and cites reference documents to the page;
- an **analytics pipeline**: reading events captured in the browser, aggregated incrementally into per-article and per-writer metrics, presented to writers as dashboards and to magazines as an evaluation report completed by AI portfolio insights;
- a **marketplace**: an eligibility gate, magazine subscriptions with monthly credits, a two-stage exclusive purchase and the magazine's own publication, with every credit movement written to a ledger inside one transaction and three balance invariants asserted after every money-moving test;
- a **social layer** (likes, threaded comments, comment likes, reposts, follows, saves, blocks, sharing and live notifications) and the **moderation** console;
- the **engineering around it**: a containerised stack started by one command, a continuous-integration pipeline with strict typing, zero lint warnings and a schema-drift check, 796 backend tests, a demonstration corpus and a production-ready stack.

The evaluation of Chapter 7 measured the result rather than asserting it: retrieval returns a relevant passage for 89% of questions, hybrid search keeps every relevant article for paraphrased queries that keyword search misses, the moderation classifier reaches a precision of 0.93 and a recall of 1.00, the API answers in tens of milliseconds, and 98% of the planned points were delivered.

Beyond the product, the project was an exercise in engineering discipline: requirements stated per actor with stable identifiers, non-functional requirements tied to the mechanism that provides them and to the way they are verified, business rules proven by tests (including tests designed to fail when the code is sabotaged) and claims checked by measurement, which in two cases corrected the project itself.

Limitations

Some points remain open and are stated as such.

1. **Public hosting.** The production stack, its TLS configuration and its delivery pipeline are ready and the domain is registered, but no server was provisioned, so US-55 is only partly delivered.
2. **Background job retries.** No queue is configured with retries, so a job that fails (an embedding during a provider outage, for instance) is not attempted again (NFR-23).
3. **Pagination.** Lists are paginated by offset; a cursor-based scheme would stay stable when new rows are inserted while a reader scrolls (NFR-14).
4. **Readiness.** The readiness endpoint checks PostgreSQL but not Redis (NFR-25).

5. **Mobile performance and accessibility.** On the mobile preset the article pages score 63 to 74 for performance, and the public pages carry serious, though not critical, accessibility violations: insufficient contrast and icon-only links without a name (NFR-37).
6. **Session revocation.** Refresh tokens are stateless: a password change does not revoke one already issued, which stays valid until its seven days expire. A ban is enforced at the next refresh, within fifteen minutes.
7. **Test coverage of the analytics.** The aggregation queries have no unit tests; they are verified only end to end and by measurement.
8. **Search under quota exhaustion.** When the embedding provider's quota is exhausted, search falls back to keywords only after about six seconds of retries.

Perspectives

- **Go live** on a provisioned server, then measure real traffic against the performance requirements.
- **Harden the background work**: retries with back-off on every queue, and a readiness check that includes Redis.
- **Tune retrieval and search with the measurements of Chapter 7**: raise the similarity floor to 0.65, cache the embeddings of frequent queries, and fail fast to keyword search once the provider's quota is known to be exhausted.
- **Fix the accessibility findings** (contrast tokens and accessible names on icon links) and improve the mobile loading of article pages.
- **Test the aggregation** with fixtures of known events and expected metrics.
- **Real payments** in place of the simulated ones, with the ledger already designed to record them, and **revocable sessions** by storing refresh tokens server-side.
- **Richer AI collaboration**: dictating whole articles, style learning beyond the current voice profile, and feedback on weak sections.
- **Wider audience**: multilingual publishing and a native mobile application on the existing 375 px designs.

References

[1] Goldman Sachs Research, ‘The creator economy could approach half-a-trillion dollars by 2027’. 2023. Accessed: 2026. [Online]. Available: https://www.goldmansachs.com/insights/articles/the-creator-economy-could-approach-half-a-trillion-dollars-by-2027

[2] Grammarly, ‘Grammarly: Generative AI writing assistance’. 2026. Accessed: 2026. [Online]. Available: https://www.grammarly.com/ai

[3] Notion Labs, ‘Notion AI’. 2026. Accessed: 2026. [Online]. Available: https://www.notion.com/product/ai

[4] beehiiv, ‘Beehiiv AI: Writing assistant in the post editor’. 2026. Accessed: 2026. [Online]. Available: https://www.beehiiv.com/features/artificial-intelligence

[5] Medium, ‘Medium partner program earnings calculation’. 2026. Accessed: 2026. [Online]. Available: https://help.medium.com/hc/en-us/articles/360036691193

[6] Substack, ‘Substack: A new economic engine for culture’. 2026. Accessed: 2026. [Online]. Available: https://substack.com/about

[7] Ghost Foundation, ‘Ghost: The open-source publishing platform for paid memberships and newsletters’. 2026. Accessed: 2026. [Online]. Available: https://ghost.org/features/

[8] Contently, ‘Contently: Content marketing platform and freelance creative network’. 2026. Accessed: 2026. [Online]. Available: https://contently.com

[9] Upwork, ‘Hire freelance writers on Upwork’. 2026. Accessed: 2026. [Online]. Available: https://www.upwork.com/hire/writers/

[10] Google, ‘NotebookLM goes global with slides support and better ways to fact-check’. 2024. Accessed: 2026. [Online]. Available: https://blog.google/technology/ai/notebooklm-goes-global-support-for-websites-slides-fact-check/

[11] OpenAI, ‘File uploads with ChatGPT: Frequently asked questions’. 2026. Accessed: 2026. [Online]. Available: https://help.openai.com/en/articles/8555545-file-uploads-faq

[12] Perplexity AI, ‘What are Spaces?’ 2026. Accessed: 2026. [Online]. Available: https://www.perplexity.ai/help-center/en/articles/10352961-what-are-spaces

[13] W. W. Royce, ‘Managing the development of large software systems’, in *Proceedings of IEEE WESCON*, 1970, pp. 1–9.

[14] K. Beck *et al.*, ‘Manifesto for agile software development’. 2001. Accessed: 2026. [Online]. Available: https://agilemanifesto.org

[15] I. Sommerville, *Software engineering*, 10th ed. Boston: Pearson, 2016.

[16] K. Schwaber and J. Sutherland, ‘The scrum guide: The definitive guide to scrum: The rules of the game’. 2020. Accessed: 2026. [Online]. Available: https://scrumguides.org/scrum-guide.html

[17] Agile Business Consortium, ‘The DSDM agile project framework (2014 onwards), chapter 10: MoSCoW prioritisation’. 2014. Accessed: 2026. [Online]. Available: https://www.agilebusiness.org/dsdm-project-framework/moscow-prioririsation.html

[18] M. Cohn, *Agile estimating and planning*. Upper Saddle River, NJ: Prentice Hall, 2005.

[19] Object Management Group, ‘OMG unified modeling language (OMG UML), version 2.5.1’. 2017. Accessed: 2026. [Online]. Available: https://www.omg.org/spec/UML/2.5.1

[20] OWASP Foundation, ‘Password storage cheat sheet’. 2026. Accessed: 2026. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

[21] M. B. Jones, J. Bradley, and N. Sakimura, ‘JSON web token (JWT)’, Internet Engineering Task Force, RFC 7519, 2015. doi: 10.17487/RFC7519.

[22] Yu. A. Malkov and D. A. Yashunin, ‘Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs’, *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020, doi: 10.1109/TPAMI.2018.2889473.

[23] The PostgreSQL Global Development Group, ‘PostgreSQL 16 documentation, chapter 12: Full text search’. 2023. Accessed: 2026. [Online]. Available: https://www.postgresql.org/docs/16/textsearch.html

[24] Docker, ‘Docker compose documentation’. 2026. Accessed: 2026. [Online]. Available: https://docs.docker.com/compose/

[25] Microsoft, ‘Playwright documentation’. 2026. Accessed: 2026. [Online]. Available: https://playwright.dev/docs/intro

[26] Vercel, ‘Next.js documentation’. 2026. Accessed: 2026. [Online]. Available: https://nextjs.org/docs

[27] Devographics, ‘State of JavaScript 2024: libraries’. 2025. Accessed: 2026. [Online]. Available: https://2024.stateofjs.com/en-US/libraries/

[28] Tiptap, ‘Tiptap editor documentation’. 2026. Accessed: 2026. [Online]. Available: https://tiptap.dev/docs

[29] Vercel, ‘AI SDK documentation’. 2026. Accessed: 2026. [Online]. Available: https://ai-sdk.dev/docs

[30] NestJS, ‘NestJS documentation’. 2026. Accessed: 2026. [Online]. Available: https://docs.nestjs.com

[31] TechEmpower, ‘Web framework benchmarks, round 23’. 2025. Accessed: 2026. [Online]. Available: https://www.techempower.com/benchmarks/

[32] A. Kane, ‘Pgvector: Open-source vector similarity search for Postgres’. 2026. Accessed: 2026. [Online]. Available: https://github.com/pgvector/pgvector

[33] M. Aumüller, E. Bernhardsson, and A. Faithfull, ‘ANN-Benchmarks: A benchmarking tool for approximate nearest neighbor algorithms’, *Information Systems*, vol. 87, p. 101374, 2020, doi: 10.1016/j.is.2019.02.006.

[34] Drizzle Team, ‘Drizzle ORM documentation’. 2026. Accessed: 2026. [Online]. Available: https://orm.drizzle.team/docs/overview

[35] Taskforce.sh, ‘BullMQ documentation’. 2026. Accessed: 2026. [Online]. Available: https://docs.bullmq.io

[36] D. Hardt, ‘The OAuth 2.0 authorization framework’, Internet Engineering Task Force, RFC 6749, 2012. doi: 10.17487/RFC6749.

[37] Groq, ‘GroqCloud documentation: Supported models’. 2026. Accessed: 2026. [Online]. Available: https://console.groq.com/docs/models

[38] Google, ‘Gemini API: embeddings’. 2026. Accessed: 2026. [Online]. Available: https://ai.google.dev/gemini-api/docs/embeddings

[39] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, ‘Reciprocal rank fusion outperforms condorcet and individual rank learning methods’, in *Proceedings of the 32nd international ACM SIGIR conference on research and development in information retrieval*, 2009, pp. 758–759. doi: 10.1145/1571941.1572114.

[40] P. Lewis *et al.*, ‘Retrieval-augmented generation for knowledge-intensive NLP tasks’, in *Advances in neural information processing systems 33 (NeurIPS 2020)*, 2020, pp. 9459–9474.

[41] Y. Gao *et al.*, ‘Retrieval-augmented generation for large language models: A survey’, *arXiv preprint arXiv:2312.10997*, 2023.

[42] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to information retrieval*. Cambridge: Cambridge University Press, 2008.

[43] E. M. Voorhees, ‘The TREC-8 question answering track report’, in *Proceedings of the eighth text REtrieval conference (TREC-8)*, 1999, pp. 77–82.

[44] K. Järvelin and J. Kekäläinen, ‘Cumulated gain-based evaluation of IR techniques’, *ACM Transactions on Information Systems*, vol. 20, no. 4, pp. 422–446, 2002, doi: 10.1145/582415.582418.

[45] Google Chrome Developers, ‘Introduction to Lighthouse’. 2026. Accessed: 2026. [Online]. Available: https://developer.chrome.com/docs/lighthouse/overview

[46] Deque Systems, ‘Axe-core: Accessibility engine for automated web UI testing’. 2026. Accessed: 2026. [Online]. Available: https://github.com/dequelabs/axe-core

[47] World Wide Web Consortium, ‘Web content accessibility guidelines (WCAG) 2.2’. 2023. Accessed: 2026. [Online]. Available: https://www.w3.org/TR/WCAG22/

APPENDIX A

Functional requirements

This annex gives every functional requirement of Section 1.4, per actor, with its MoSCoW priority and the sprint or sprints that deliver it. A requirement listed under two sprints is delivered in two steps, as explained in the release chapters.

Table A.1 – Functional requirements: visitor

ID	Requirement	Priority	Sprint
FR-01	Browse the public feed, paginated, with title, excerpt, thumbnail and tags	Must	1
FR-02	Read the full body of a free public article without an account	Must	1
FR-03	Be blocked from premium and marketplace articles, with an explicit prompt to sign up	Must	1
FR-04	Register as a personal account, or as a magazine account	Must	1
FR-05	Authenticate by email/password or Google OAuth	Must	1
FR-06	Search articles and writers, and browse by tag	Should	4
FR-69	Continue any article list as the reader scrolls, with a “Load more” control that remains available and a failed page that neither discards loaded rows nor retries itself	Must	8
FR-71	Not see a marketplace listing on the writer’s public profile at all, the exceptions being the article’s own author and an administrator	Must	5
FR-75	Read a magazine’s published articles on its public profile, paginated, without an account	Must	5
FR-76	See both the writer and the publishing magazine named on a licensed article, each linking to its own profile	Must	5
FR-77	Preview a writer or magazine from any byline by hovering it (name, bio, counts and a follow control for a writer; logo, description and founding date for a magazine) without leaving the page	Should	8

Table A.2 – Functional requirements: reader

ID	Requirement	Priority	Sprint
FR-07	Manage a profile: display name, bio, avatar upload	Must	1
FR-08	Like an article, and un-like it	Must	3
FR-09	Comment on an article, and reply to a comment (threaded)	Must	3
FR-10	Delete one’s own comment	Must	3
FR-11	Repost an article	Could	3
FR-12	Follow and unfollow a writer	Should	3
FR-13	Receive notifications for follows, likes, comment likes, saves, comments and replies, delivered live	Must	3
FR-14	List past notifications and mark them read	Must	3
FR-15	Upgrade to premium, and downgrade (simulated payment)	Must	5
FR-16	*(premium plan)* Read premium-visibility articles	Must	5
FR-17	*(premium plan)* Consume a daily AI token allowance	Must	2, 5
FR-60	Like a comment or a reply, and un-like it	Must	7
FR-61	View any profile’s followers and following lists, and follow or unfollow from them	Must	7
FR-62	Block an account, and review and lift one’s blocks from settings	Must	7

ID	Requirement	Priority	Sprint
FR-63	Save an article from the feed card or from the article page, and unsave it	Must	7
FR-64	Review one's own saved articles, from the sidebar or from the Saved tab on one's own profile	Must	7
FR-65	Share an article or a writer profile to X, Facebook or LinkedIn, or copy its link, without needing an account	Should	7
FR-66	Like, repost and comment on an article from a feed card, and see whether one has already liked or reposted it	Must	8
FR-67	Read any account's reposts from the Reposted tab on its profile, without an account	Should	8
FR-68	See a feed card's most recent comments from the last hour, up to three, without opening the article	Could	8
FR-70	Be refused when liking, commenting on or reposting a marketplace listing, while saving one stays available	Must	5

Table A.3 – Functional requirements: writer

ID	Requirement	Priority	Sprint
FR-18	Create a draft and edit it in a rich-text editor (TipTap, stored as JSON)	Must	2
FR-19	Have drafts saved automatically, debounced, without a save action	Must	2
FR-20	Insert images by paste, drag or file picker, uploaded directly to object storage	Must	2
FR-21	Set a cover image, excerpt and tags	Must	2
FR-22	See live word count and estimated reading time	Should	2
FR-23	Publish an article, choosing its placement: public or marketplace	Must	1, 5
FR-24	Choose a visibility for public articles: free or premium	Must	1, 5
FR-25	Update or soft-delete an own article	Must	1
FR-26	Switch an article from marketplace to public (one-way; the reverse is refused)	Should	5
FR-27	Converse with an AI assistant grounded in the writer's own published corpus	Must	2, 4
FR-28	Apply an inline AI action to a text selection: reformulate, shorten, expand, simplify, improve	Must	2
FR-29	See which passages the assistant retrieved, grouped by source article	Should	4
FR-30	Insert an AI response into the document at the cursor	Must	2
FR-31	See the remaining AI token balance, updated after each action	Must	2
FR-32	View per-article analytics: views, unique readers, average read time, completion, engagement, retention curve	Must	3
FR-33	Rank own articles by views or engagement, server-side across the whole body of work	Should	3
FR-34	See progress toward marketplace eligibility	Must	5
FR-35	*(eligible writer)* Publish to the marketplace with a price in credits	Must	5
FR-36	*(eligible writer)* Publish premium-visibility articles	Must	5
FR-37	View earnings: lifetime, per article, itemised by preview and purchase	Must	5
FR-38	Be notified when a magazine previews or purchases an article, and when earnings are credited	Must	5
FR-78	The assistant decides, per message, whether to answer in the panel or write into the document, superseding the manual insert of FR-30	Must	8
FR-79	Written text streams into the document at the cursor or over the selection, marked and protected until the writer keeps or discards it; Keep is one undo step; Discard restores what it replaced	Must	8
FR-80	Show the pipeline's real steps as they run, with counts, and a one-line recap after a write	Must	8

ID	Requirement	Priority	Sprint
FR-81	The assistant is a panel docked beside the editor with Chat and Sources tabs; it collapses to a rail that carries the live step, and opens as a bottom sheet below desktop width	Should	8
FR-82	Record a prompt with the microphone (≤ 5 min) and have it transcribed into the assistant's input, editable, billed at 200 tokens a minute against the daily allowance	Should	8
FR-83	Hear each assistant reply read aloud, with a persisted mute; the control exists only when the key can speak	Could	8
FR-84	Upload PDF, DOCX, TXT or MD reference documents (≤ 10 MB, ≤ 200 pages, 20 per writer) into a private library, with text extracted and embedded by the worker and a readable failure reason when it cannot be	Must	9
FR-85	Attach library documents to an article from the assistant and detach them; attachments are saved immediately	Must	9
FR-86	The assistant draws on attached documents for both answers and writes, cites passages inline as [Title, p. N], and shows the passages used with a link opening the file at the page	Must	9
FR-87	A document is the writer's alone: it never feeds writer memory, Portfolio Insights or search, and another account cannot read, attach or discover it	Must	9

Table A.4 – Functional requirements: magazine

ID	Requirement	Priority	Sprint
FR-39	Register with magazine fields: name, slug, logo, website, description	Must	1
FR-40	*(without subscription)* Be stopped at a subscription wall before any marketplace surface	Must	5
FR-41	Subscribe, receiving a monthly credit allowance (simulated payment)	Must	5
FR-42	Top up credits outside the monthly grant	Should	5
FR-43	See the credit balance, the renewal date and the transaction history	Must	5
FR-44	Browse eligible writers, filtered by topic, searchable, with four sort orders	Must	3, 5
FR-45	Read a writer evaluation report: audience, content, quality panels	Must	3
FR-46	Request AI Portfolio Insights for a writer, as an explicit action	Must	4
FR-47	Browse a writer's marketplace-listed articles with prices	Must	5
FR-48	Unlock a preview for 10% of the price, and read the full article	Must	5
FR-49	Purchase the remainder (90%) and gain republish rights	Must	5
FR-50	See the curated library of fully purchased articles	Must	5
FR-72	Be refused when previewing, buying, switching to public, or deleting an article another magazine has already purchased exclusively	Must	5
FR-73	Publish an article it has purchased, under its own masthead, making it public and free, clearing the price, and recording the publisher	Must	5
FR-74	See the library split into what has been published and what is still waiting, and act only on the second	Must	5

Table A.5 – Functional requirements: administrator

ID	Requirement	Priority	Sprint
FR-51	Review a queue of reports on articles, users and comments	Must	5
FR-52	Dismiss a report, delete the reported article, or ban its author	Must	5
FR-53	List and search users, filtered by role and by active/banned	Should	5
FR-54	Edit a user's plan (both directions), logged	Should	5
FR-55	Grant marketplace eligibility manually, with an audit-log entry	Must	5

Table A.6 – Functional requirements: cross-cutting

ID	Requirement	Priority	Sprint
FR-56	Every article body access decision is taken server-side against the article access decision (the activity diagram of Sprint 5); locked content arrives as null	Must	5
FR-57	Content is classified at publish time; a flag files a pending report but never blocks publication	Should	5
FR-58	Any credit movement writes ledger rows inside one transaction, under a row lock	Must	5
FR-59	Report an article, a user or a comment	Must	5

APPENDIX B

Product backlog

This annex gives the full product backlog of Section 1.8, epic by epic, sorted by identifier. Each story is stated as *"As an ⟨actor⟩, I want ⟨goal⟩ so that ⟨benefit⟩"*; the actor column gives the actor of Table 1.2, and the technical stories of E9 have none.

E1: Accounts and authentication

ID	User story	Actor	Priority	Points	Sprint
US-01	Register with an email and a password so that I can have an identity on the platform	A1	Must	3	1
US-02	Sign in with Google so that I do not manage another password	A1	Should	3	1
US-03	Register as a magazine with my branding so that writers recognise my publication	A1	Must	5	1
US-04	Stay signed in across a long writing session so that I do not lose a draft	A3	Must	3	1
US-05	Edit my profile and avatar so that my public page represents me	A2	Should	3	1

E2: Writing and publishing

ID	User story	Actor	Priority	Points	Sprint
US-06	Write in a rich editor so that structure and emphasis survive publication	A3	Must	8	2
US-07	Have my draft saved as I type so that I never lose work	A3	Must	3	2
US-08	Paste or drag an image into the article so that illustrating is not a detour	A3	Must	5	2
US-09	Set a cover image so that my article looks deliberate in the feed	A3	Should	3	2
US-10	Tag an article so that readers find it by topic	A3	Should	2	1
US-11	Choose at publish time whether an article is public or for sale	A3	Must	5	1
US-12	Restrict an article to premium readers so that my best work is worth a subscription	A3	Should	3	5
US-13	See word count and reading time so that I can judge length as I write	A3	Could	1	2

E3: AI assistance

ID	User story	Actor	Priority	Points	Sprint
US-14	Ask an assistant that has read my published work so that suggestions sound like me	A3	Must	13	2, 4
US-15	Reformulate, shorten, expand, simplify or improve a selection so that editing is one click	A3	Must	8	2
US-16	Insert an answer into my document so that I do not copy by hand	A3	Must	2	2

ID	User story	Actor	Priority	Points	Sprint
US-17	See which of my passages the assistant used so that I can trust the answer	A3	Should	5	4
US-18	See my remaining allowance so that I can pace my usage	A2	Must	2	2
US-19	Have my voice profile extracted from my corpus so that the assistant stays consistent when retrieval finds nothing	A3	Should	8	4
US-66	Tell the assistant what to write and watch it appear in my draft, then keep or discard it, so that generation happens where I write rather than in a chat I copy from	A3	Must	8	8
US-67	See what the assistant is doing while it works, and minimize it without losing sight of that, so that a slow step is not a frozen screen	A3	Should	3	8
US-68	Say my request instead of typing it so that I can brief the assistant while my hands are busy or my thoughts are faster than my typing	A3	Should	5	8
US-69	Hear the assistant’s recap read out so that I can keep my eyes on the draft, and mute it when I would rather not	A3	Could	3	8
US-70	Upload the documents I am working from so that the assistant can use their facts without my retyping them	A3	Must	8	9
US-71	Choose which documents apply to an article so that a piece on tides is not fed my notes on fermentation	A3	Must	3	9
US-72	See which document and page a claim came from so that I can check it before it goes out under my name	A3	Should	5	9

E4: Discovery and search

ID	User story	Actor	Priority	Points	Sprint
US-20	Search by keyword so that I find an article I half-remember	A1	Must	5	4
US-21	Find articles that match my meaning even when they share none of my words	A1	Must	8	4
US-22	Browse by tag so that I can follow a topic	A1	Should	2	4
US-23	Find writers, not only articles, so that I can follow a person	A1	Should	3	4

E5: Social interactions and notifications

ID	User story	Actor	Priority	Points	Sprint
US-24	Like an article so that I can signal quality	A2	Must	2	3
US-25	Comment and reply in a thread so that discussion has structure	A2	Must	5	3
US-26	Follow a writer so that I keep up with them	A2	Should	3	3
US-27	Repost an article so that my followers see it	A2	Could	2	3
US-28	Be told the moment someone reacts to my work, without reloading	A3	Must	5	3
US-29	Review notifications I missed so that nothing is lost between sessions	A2	Must	3	3
US-57	Save an article to come back to it later, without anyone else seeing what I keep	A2	Must	5	7
US-58	Pass an article on to people outside the platform so that they can read it without signing up	A1	Should	3	7
US-59	React to an article from the feed so that I can engage without losing my place	A2	Must	5	8

ID	User story	Actor	Priority	Points	Sprint
US-60	See what an account has reposted so that I can judge what they rate	A2	Should	3	8
US-61	Reply in the moment from the feed so that a fresh conversation is easy to join	A2	Could	3	8
US-62	Keep reading as I scroll so that a list does not end at an arbitrary point I have to click past	A1	Must	5	8
US-65	See who a writer or magazine is from where their name appears so that I can decide to follow without leaving the feed	A1	Should	3	8
US-73	Like a comment so that I can credit a good reply without writing one	A2	Must	2	7
US-74	Browse who follows an account and whom it follows, and follow from the list, so that I find people through people	A1	Must	3	7

E6: Analytics

ID	User story	Actor	Priority	Points	Sprint
US-30	See how many people read an article and how far they got so that I can write better	A3	Must	8	3
US-31	Rank my articles by performance so that I learn what works	A3	Should	3	3
US-32	Evaluate a writer on audience, content and quality so that I can commission with evidence	A4	Must	13	3
US-33	Read an AI assessment of a writer’s voice and range so that I can shortlist quickly	A4	Must	8	4

E7: Marketplace and subscriptions

ID	User story	Actor	Priority	Points	Sprint
US-34	Upgrade to premium so that I can read everything and use the assistant	A2	Must	5	5
US-35	Subscribe as a magazine so that I can access the marketplace	A4	Must	8	5
US-36	Receive a monthly credit budget so that spending is predictable	A4	Must	5	5
US-37	Top up credits so that a good month is not capped	A4	Should	3	5
US-38	Pay 10% to read an article in full so that I can judge before buying	A4	Must	8	5
US-39	Pay the remainder to acquire republish rights so that I can publish it	A4	Must	8	5
US-40	Keep purchased articles in a library so that my acquisitions are in one place	A4	Must	3	5
US-41	List an article for sale at my own price so that I am paid for exclusivity	A3	Must	5	5
US-42	See my earnings itemised by preview and purchase so that I trust the accounting	A3	Must	5	5
US-43	See how far I am from eligibility so that the gate feels reachable	A3	Should	3	5
US-63	Publish an article I have bought when I am ready, rather than the purchase publishing it for me, so that my masthead stays mine to decide	A4	Must	5	5

ID	User story	Actor	Priority	Points	Sprint
US-64	Have the articles I publish appear on my public profile under my masthead so that licensing buys me a shelf and not just a file	A4	Must	5	5

E8: Moderation and administration

ID	User story	Actor	Priority	Points	Sprint
US-44	Report content so that abuse has a channel	A2	Must	3	5
US-45	Work a queue of reports so that moderation is systematic	A5	Must	5	5
US-46	Remove an article or ban an author so that decisions are enforceable	A5	Must	3	5
US-47	Grant eligibility manually so that a promising writer is not held back by a counter	A5	Must	3	5
US-48	Search and filter users so that I can act on the right account	A5	Should	5	5
US-56	Block an account so that I stop seeing someone without having to explain myself	A2	Must	5	7

E9: Quality and operations (technical stories)

ID	User story	Actor	Priority	Points	Sprint
US-49	Prove that every balance equals its ledger sum, after every money-moving operation	none	Must	8	1
US-50	Bring the whole stack up with one command, reproducibly	none	Must	8	1
US-51	Fail the build on a lint error, a type error, a failing test or schema drift	none	Must	5	1
US-52	Apply migrations automatically before the application starts	none	Must	5	6
US-53	Seed a believable corpus so that the product can be demonstrated cold	none	Must	5	6
US-54	Lower the eligibility thresholds under a flag so that the gate can be crossed live	none	Must	2	6
US-55	Serve the product at a public URL over TLS	none	Must	8	6